О. В. ШИНДОР,
Д. Е. ЧИКРИН,
П. А. КОКУНИН

LabVIEW В ЦИФРОВОЙ ОБРАБОТКЕ СИГНАЛОВ

УЧЕБНОЕ ПОСОБИЕ



ЛАНЬ
САНКТ-ПЕТЕРБУРГ • МОСКВА • КРАСНОДАР
2025

УДК 004.92
ББК 32.81я73

Ш 62 **Шиндор О. В.** LabVIEW в цифровой обработке сигналов : учебное пособие для вузов / О. В. Шиндор, Д. Е. Чикрин, П. А. Кокунин. — Санкт-Петербург : Лань, 2025. — 100 с. : ил. — Текст : непосредственный.

ISBN 978-5-507-51490-8

Данное пособие включает в себя практикум по цифровой обработке сигналов с использованием среды графического программирования LabVIEW. В пособии изложены теоретические основы создания виртуальных приборов в среде LabVIEW, приведены практические примеры разработки виртуальных приборов, рассмотрены методы анализа сигналов: фильтрация, спектральный анализ, вейвлет-преобразование.

Учебное пособие предназначено для студентов вузов, аспирантов и преподавателей.

УДК 004.92
ББК 32.81я73

Рецензенты:
Э. И. БЕЛЯЕВ — кандидат технических наук, доцент,
директор Института цифровых технологий и экономики
Казанского государственного энергетического университета;
Е. С. ДЕНИСОВ — кандидат технических наук, доцент,
доцент кафедры радиоэлектроники и информационно-измерительной техники
Казанского национального исследовательского
технического университета им. А. Н. Туполева.

Обложка
П. И. ПОЛЯКОВА

© Издательство «Лань», 2025
© Коллектив авторов, 2025
© Издательство «Лань»,
художественное оформление, 2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 5

ГЛАВА 1. ОСНОВЫ ПРОГРАММИРОВАНИЯ В СРЕДЕ LabVIEW 6

1.1. Концепция LabVIEW и области применимости 6
1.2. Виртуальные измерительные приборы: структура и принципы создания 7
1.3. Лицевая панель виртуального прибора 12
1.4. Блок-диаграмма виртуального прибора 13
1.5. Принцип потока данных 16
1.6. Примеры виртуальных приборов: преобразование температуры из градусов Цельсия в температуру по Фаренгейту 16
1.7. Числовые функции в LabVIEW 21
1.8. Примеры виртуальных приборов: вычисление с помощью формул 22
1.9. Циклы. Цикл с фиксированным числом итераций For Loop 25
1.10. Цикл по условию While Loop 26
1.11. Сдвиговые регистры 26
1.12. Массивы 27
1.13. Примеры виртуальных приборов: вычисление чисел Фибоначчи 29
1.14. Примеры виртуальных приборов: генератор случайных чисел 31

ГЛАВА 2. ИНТЕГРАЦИЯ ТЕКСТОВЫХ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ В LabVIEW 33

2.1. Интеграция скрипта Python в LabVIEW 33
2.2. Интеграция скрипта MATLAB в LabVIEW 36
2.3. Интеграция кода C++ в LabVIEW 37

ГЛАВА 3. ГЕНЕРАЦИЯ И ЦИФРОВАЯ ОБРАБОТКА СИГНАЛОВ 41

3.1. Примеры виртуальных приборов: генерация сигналов в LabVIEW с помощью формул 41
3.2. Анализатор спектра в LabVIEW. Быстрое преобразование Фурье 44
3.3. Электрические шумы. Виды шумов. Характеристики шумов 48
3.4. Структура варианта (Case) 49
3.5. Примеры виртуальных приборов: статистические характеристики шумов 50
3.6. Фильтрация сигналов. Цифровые фильтры 56
3.7. КИХ-фильтры: особенности и характеристики 57
3.8. Пример виртуального прибора: фильтрация сигнала с помощью цифрового КИХ-фильтра 60
3.9. БИХ-фильтры: типы и частотные характеристики 62
3.10. Пример виртуального прибора: фильтрация сигнала с помощью цифрового БИХ-фильтра 63
3.11. Фильтрация сигнала с помощью Express VI 65
3.12. Вейвлет-анализ 66
3.13. Пример виртуального прибора: анализ сигнала с помощью непрерывного вейвлет-преобразования 69

3.14. Пример виртуального прибора: выделение полезного сигнала
из зашумленного сигнала с помощью дискретного вейвлет-преобразования 72
3.15. Звуковые сигналы. Виртуальный прибор считывания звукового сигнала
из файла и его спектральный анализ....77
3.16. Вибродиагностика 80
3.17. Пример виртуального прибора: спектральный анализ
вибрационного сигнала гидрогенератора 80

ГЛАВА 4. ПЛАТФОРМА NI ELVIS III 84
4.1. Назначение платформы NI ELVIS III, архитектура аппаратных средств 84
4.2. Генерация сигнала в NI ELVIS III 87

Приложения 91
Приложение 1. Типы данных....92
Приложение 2. Математические функции 93
Приложение 3. Операции структуры Formula Node 94
Приложение 4. Классификация вейвлетов 95

ЛИТЕРАТУРА 98

ВВЕДЕНИЕ

LabVIEW (*Laboratory Virtual Instrument Engineering Workbench*) с начала 2000-х гг. в России используется в учебном процессе для выполнения лабораторных практикумов по физике, механике, аналоговой и цифровой электронике, схемотехнике. В научных исследованиях многоплатформенная среда *LabVIEW* позволяет осуществлять сбор, анализ, обработку данных модельных и реальных устройств, создавать электронную аппаратуру с широкими функциональными возможностями. В промышленности и инженерной практике — на объектах космического, воздушного, надводного и подводного флота, нефтедобычи и многих других. Стоит отметить, что первая версия *LabVIEW* была выпущена фирмой National Instruments в 1986 г. С тех пор она активно развивается и находит применение все в больших сферах.

Данное учебное пособие включает в себя практикум по цифровой обработке сигналов с использованием среды графического программирования *LabVIEW*.

В пособии в первой главе рассматриваются основы программирования, принципы разработки виртуальных приборов, использование математических функций в разработке приборов. Во второй главе показано включение скриптов текстовых языков программирования. В третьей главе приведены методы анализа сигналов: фильтрация, спектральный анализ, вейвлет-преобразование, а также рассмотрен пример спектрального анализа реального вибродиагностического сигнала. В четвертой главе рассмотрена платформа *NI ELVIS III* и ее возможности для генерации сигналов.

Учебное пособие может использоваться при проведении практических занятий по дисциплинам «Цифровая обработка сигналов» направлений подготовки: 09.03.04 «Программная инженерия», 15.03.06 «Мехатроника и робототехника».

ГЛАВА 1
ОСНОВЫ ПРОГРАММИРОВАНИЯ В СРЕДЕ LABVIEW

1.1. Концепция LabVIEW и области применимости

Концепция *LabVIEW* как среды разработки лабораторных виртуальных приборов принципиально отличается от традиционных языков программирования, так как представляет собой среду графического программирования и языка *G*, легкую и интуитивно понятную разработчику оболочку. Процесс программирования в среде *LabVIEW* позволяет решать поставленную задачу через создание графической блок-диаграммы, которая компилирует алгоритм в машинный код.

Пользовательский интерфейс *LabVIEW* позволяет превратить процесс программирования в увлекательное занятие, при этом значительно сокращая время по сравнению с написанием традиционного программного кода. Многофункциональные измерительные системы на основе *LabVIEW*, кроме того, обладают низкой себестоимостью наряду с широкой функциональностью и гибкостью. Данные факторы и возможности послужили широкому применению и использованию программного продукта *LabVIEW* и сопутствующих устройств в различных сферах деятельности.

Созданный в *LabVIEW* измерительный комплекс позволит измерить некоторую физическую величину (температуру, давление, скорость, объем и т. д.), обработать и представить в виде, удобном для восприятия, например, оператора. Для графического вывода информации имеются широкие возможности. Следующим вопросом встает возможность управления неким физическим процессом, который также можно реализовать с помощью среды *LabVIEW*. Таким образом, задачи диагностики, мониторинга, контроля сложных технических систем возможно решить с помощью среды разработки лабораторных виртуальных приборов. Создаваемые системы могут быть трансформированы в файл с расширением *.exe* и, соответственно, установлены на любом персональном компьютере.

После запуска *LabVIEW* появляется стартовое окно (рис. 1), разделенное на две части. В левой части окна можно выбрать создание нового ВП (*Blank VI*), нового проекта (*Empty project*), создать ВП по шаблону (*VI from Template*), а также открыть ранее созданные проекты или приборы.

Правая часть окна позволяет узнать последние новости сайта *National Instruments* (*ni.com*) и реализовать онлайн-поддержку (*Online Support*), получить доступ к справочным материалам (*Help*) или найти примеры ВП (*Find Example*).

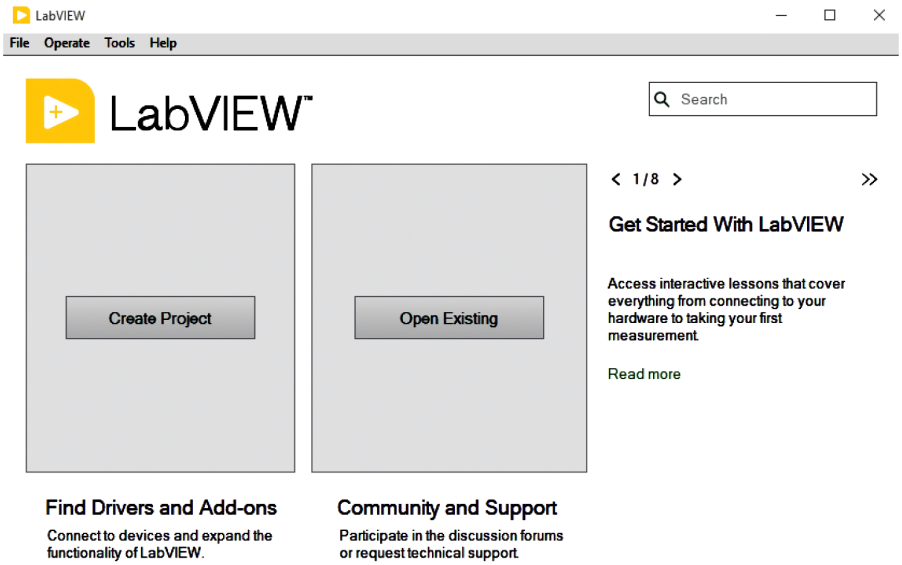


Рис. 1

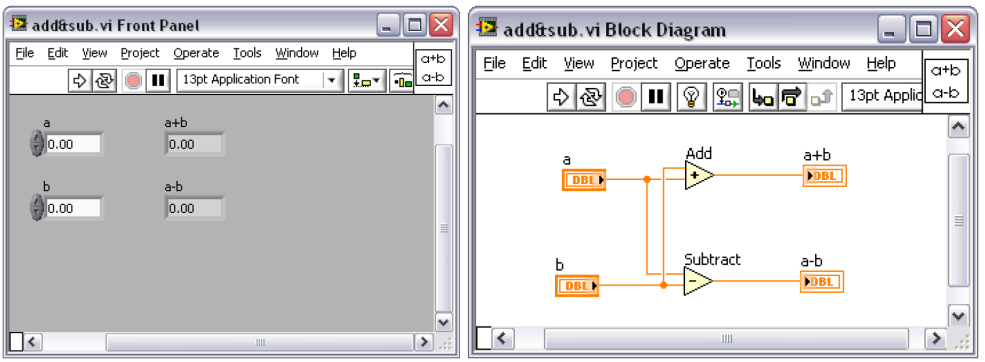
Диалоговое окно запуска LabVIEW

1.2. Виртуальные измерительные приборы: структура и принципы создания

Программы, созданные в среде *LabVIEW*, называются виртуальными приборами (ВП). Почему виртуальными приборами? Они имитируют работу реального прибора, например, осциллографа, термометра, мультиметра и представляются в виде некоторого прибора. Любой виртуальный прибор (рис. 2) состоит из:

1) лицевой панели *Front Panel* (рис. 2*а*);

2) блок-диаграммы или структурной схемы *Block Diagram* (рис. 2*б*).



а *б*

Рис. 2

Виртуальный прибор:

а — лицевая панель; *б* — блок-диаграмма.

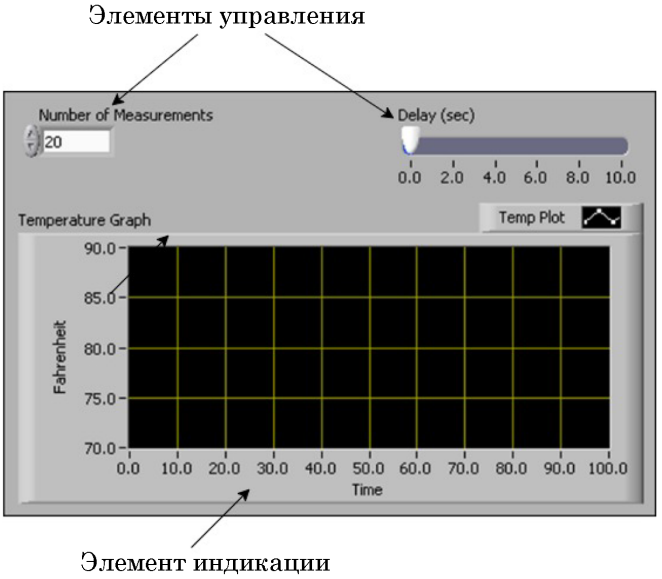


Рис. 3

Лицевая панель виртуального прибора с элементами управления и индикации

На лицевой панели ВП, как на любом реальном приборе, располагаются элементы управления (кнопки, переключатели, регуляторы), которые являются входными элементами прибора, а также элементы индикации (экраны, табло, индикаторы, диаграммы), являющиеся выходными элементами прибора (рис. 3).

Блок-диаграмма представляет собой область создания исходного кода будущего виртуального прибора. Здесь размещаются узлы и терминалы элементов ввода, управления и индикации с лицевой панели, блоки выполнения различных операций, в самом простейшем случае операций суммирования или вычитания (рис. 2*б*), операторы циклов, условий. Все они соединяются между собой проводниками, узлами и терминалами (рис. 4). Блок-диаграмму можно назвать «сердцем» виртуального прибора, именно она создается с помощью графического языка программирования *G*.

В правом верхнем углу лицевой панели и блок-диаграммы ВП располагается иконка *Icon* — графическое представление виртуального прибора и панель подключения *Connector Pane* (рис. 5). Они необходимы в случае, когда ВП является частью некоторой системы или большого прибора и представляет собой подприбор *sub VI*.

В верхней части лицевой панели и блок-диаграммы располагаются главное меню и инструментальная панель (рис. 6).

Главное меню содержит пункты *File*, *Edit*, *View*, *Project*, *Operate*, *Tools*, *Windows* и *Help*. Инструментальная панель служит для запуска и редактирования ВП, при этом на панели блок-диаграммы также имеются инструменты отладки.

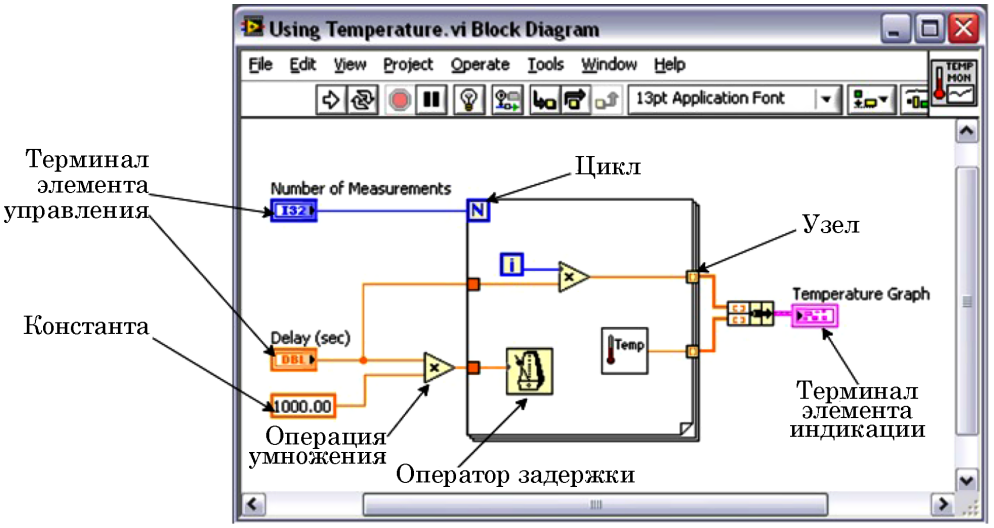
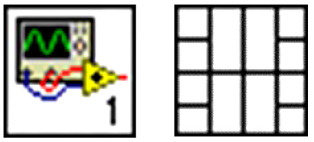


Рис. 4

Блок-диаграмма виртуального прибора Block Diagram



а *б*

Рис. 5

Составляющие виртуального прибора:

а — иконка *Icon*; *б* — панель подключения *Connector Pane.*

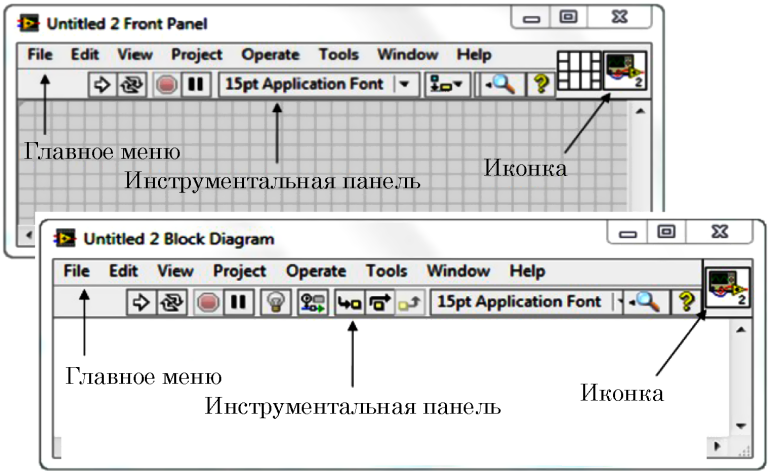


Рис. 6

Главное меню и инструментальная панель лицевой панели и блок-диаграммы ВП

Описание функций кнопок инструментальных панелей приведены в таблице 1.

Таблица 1

Описание функций кнопок инструментальных панелей

	Кнопка запуска ВП *Run*
	Вид кнопки запуска при работе ВП
	Кнопка запуска ВП, выполнение происходит непрерывно *Run Continuously*
	Вид кнопки при непрерывном выполнении ВП
	Кнопка прерывания выполнения ВП *Abort Execution*
	Кнопка паузы для временного останова ВП *Pause*
	Меню изменения размера и вида шрифта
	Кнопка выравнивания элементов управления лицевой панели *Align Objects*
	Кнопка распределения элементов управления лицевой панели *Distribute Objects*
	Кнопка изменения размеров объектов *Resize Objects*
	Кнопка изменения порядка расположения элементов на лицевой панели Reorder
	Кнопка анимации потока данных при выполнении ВП *Highlight Execution*
	Кнопка сохранения последнего значения, переданного по проводу и по окончании выполнения ВП *Retain Wire Values*
	Кнопка пошагового выполнения ВП *Start Single Stepping*
	Кнопка выполнения ВП через шаг *Step Over*
	Кнопка очистки блок-диаграммы *Clean Up Diagram*
	Окно поиска
	Кнопка вызова контекстной помощи

Переключаться между окнами можно через меню *Window* → *Show Front Panel (Show Block Diagram)*, также в *LabVIEW* предусмотрены сочетания горячих клавиш *Ctrl* + *E*.

Программирование любого виртуального прибора в *LabVIEW* начинается с лицевой панели, на которой размещаются входные элементы, переключатели, средства управления и индикации, которые отображают результат работы виртуального прибора. Процесс же функционирования прибора реализуется на блок-диаграмме.

Для навигации, перемещения, соединения элементов между собой используется палитра инструментов *Tools* (рис. 7), она располагается в меню *View* → *Tools Palette*.



Рис. 7
Палитра инструментов Tools

Краткое описание функций элементов палитры инструментов приведено в таблице 2. Самым простым способом выбора инструмента является нажатие на него ЛКМ.

Таблица 2

Описание функций элементов палитры инструментов

Рабочий инструмент	**Английское название**	**Русское название (смысловой эквивалент)**	**Функции инструмента**
	*Automatic Tool selection*	Автоматический выбор инструмента	При наведении курсора на объект автоматически выбирает соответствующий инструмент из палитры *Tools*
	*Operate Value*	Инструмент управления «*Рука*»	Размещает объекты на лицевой панели и блок-схеме, позволяет изменять значения элементов управления и вводить текст
	*Position/ Size/Select*	Инструмент управления *«Стрелка»*	Выбор, перемещение, размещение и изменение размеров объектов
	*Edit Text*	Инструмент *«Ввод текста»*	Редактирует тексты меток объектов и создает метки
	*Connect Wire*	Монтажный инструмент *«Катушка»*	Подключает «проводом» объекты друг к другу на блок-диаграмме

Продолжение табл. 2

Рабочий инструмент	Английское название	Русское название (смысловой эквивалент)	Функции инструмента
	*Object Shortcut Menu*	Инструмент *«Вызов объектного меню»*	Вызывает объектное меню
	*Scroll Window*	Инструмент *«Прокрутка»*	Прокручивает окно без использования слайдеров
	*Set/ClearBreakpoint*	Инструмент *«Контрольная точка»*	Устанавливает контрольные точки на элементах, функциях, узлах, проводах, циклах, структурах и останавливает в них выполнение программы
	*Probe Data*	Инструмент *«Пробник»*	Создает пробные индикаторы на проводах. Позволяет исследовать поток данных в проводах блок-диаграмм
	*Get Color*	Инструмент копирования цвета *«Пипетка»*	Копирует цвета для последующей вставки
	*Set Color*	Цветовой инструмент *«Кисть»*	Раскрашивает цвета лицевой панели, надписей и фона. Используется для цветового дизайна панели

1.3. Лицевая панель виртуального прибора

На палитре элементов управления *Controls* представлены данные различных типов: числовые, строковые, булевы. Они симулируют устройства ввода для разрабатываемого измерительного прибора и являются источниками данных для блок-диаграммы.

Экраны, осциллограммы всех видов, графическое представление данных, индикаторы, светодиоды симулируют устройства вывода прибора и отображают данные, сгенерированные в процессе работы ВП.

Таким образом, на лицевой панели размещаются:

- устройства ввода, управления;
- устройства индикации.

Практически все элементы и узлы в *LabVIEW* могут работать как элементы управления, так и элементы индикатора. Изменение производится нажатием ПКМ и выбором команды *Change to Indicator* или *Change to Control*.

Выбор элементов лицевой панели осуществляется с помощью палитры элементов управления *Controls* (рис. 8).

Для отображения палитры достаточно нажать правую кнопку мыши в любой области лицевой панели.

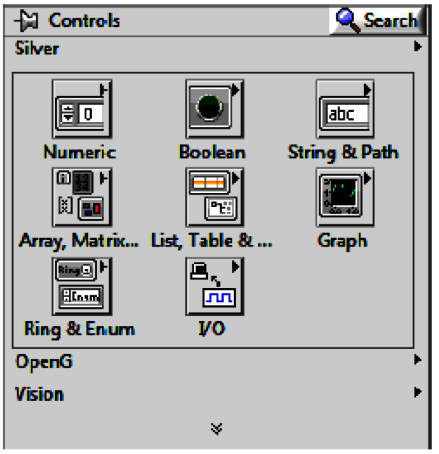


Рис. 8
Палитра элементов управления Controls

1.4. Блок-диаграмма виртуального прибора

На блок-диаграмме виртуального прибора размещается графический программный код, который позволяет реализовывать функциональные возможности прибора.

Основными элементами блок-диаграммы являются следующие.

1. Узлы (элементы) *Nodes* — аналогичны функциям, высказываниям, операторам и подпрограммам в текстовых языках программирования. На входы узлов (элементов) подаются некоторые переменные или постоянные данные, с выхода снимается информация, которая может подаваться на другой узел или на индикатор, дисплей. Пример узлов на блок-диаграмме представлен на рисунке 9. Количество входов и выходов определяется типом узла (элемента).

На рисунке 9 приведен пример узлов функций, они имеют по два входа и выход. Узлы не отображаются на лицевой панели. В соответствии со своим названием выполняют заранее запрограммированную функцию, которую можно узнать через контекстную справку (*Ctrl* + *H*). Более детализированное описание заложенных функций и назначение входов и выходов.

Выбор узлов выполняется с помощью одноименной палитры функции *Functions* (рис. 10).

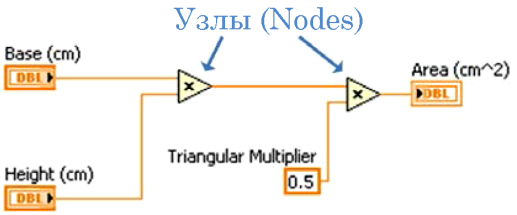


Рис. 9
Примеры узлов на блок-диаграмме

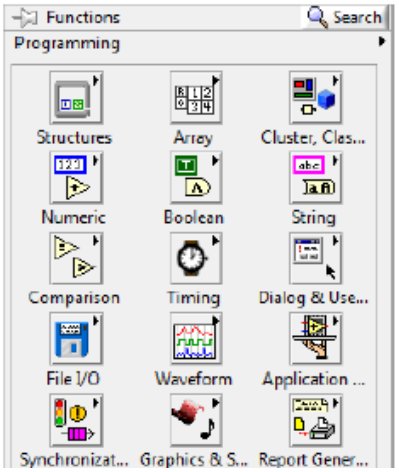


Рис. 10

Палитра функций Functions

Для отображения палитры достаточно нажать ПКМ в любой области блок-диаграммы.

Также имеются узлы, представляющие собой отдельные виртуальные приборы, имеющие свою лицевую панель и блок-диаграмму, они называются узлами *Sub VI*. Выполняемая ими функция, как правило, не так проста, как функция, выполняемая узлом функций. Такие узлы (элементы) можно найти, например, *Functions* → *Signal Processing*.

Еще один вид *Sub VI*, встречающийся в *LabVIEW*, представляет собой *Express VI*. Он отличается большим размером по сравнению с узлами функций и узлами *Sub VI*, выполнен в голубом цвете и требует минимальных соединений, имеет диалоговое окно, с помощью которого и осуществляется его конфигурация. Пример такого *Express VI* приведен на рисунке 11.



Рис. 11

Пример Express VI затраченного времени Elapsed Time

2. Терминалы — элементы, необходимые для передачи данных между объектами на лицевой панели и функциональными блоками. Для каждого объекта лицевой панели обязательно есть свой терминал на блок-диаграмме, для его нахождения достаточно выбрать инструмент «стрелка» и дважды щелкнуть по элементу на лицевой панели, автоматически откроется блок-диаграмма,

и терминал элемента будет выделен. Аналогичное действие можно выполнить на блок-диаграмме;

3. Проводники данных (провода) — необходимы для передачи информации от входных элементов к элементам вывода через узлы. С помощью проводников выполняется соединение всех элементов блок-диаграммы и передача между ними данных. Они имеют различный стиль, цвет, толщину, эти параметры зависят от типа данных. Примеры проводников приведены на рисунке 12. Линия одинарной толщины соответствует скалярной величине *Scalar*, линия потолще — одномерному массиву *1D Array*, двойная линия — двумерному массиву *2D Array*.

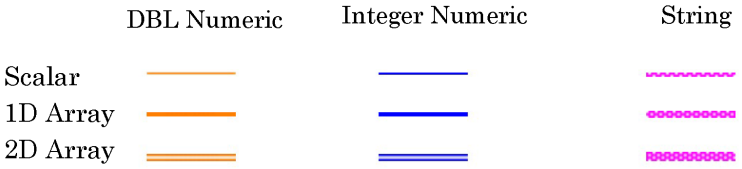
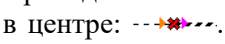


Рис. 12

Типы проводников

При соединении входных, выходных элементов и функциональных блоков с помощью проводов необходимо учитывать тип данных. Данные могут передаваться только в одном направлении: от источника к приемнику данных, т. е. от элементов контроля, управления через узлы, функции или *Sub VI* к элементам индикации.

Разорванный проводник выглядит в виде черной пунктирной линии с красным крестом в центре:  . Удалить все разорванные проводники на блок-диаграмме можно с помощью сочетания *Ctrl+B*. Для симметричного и информативного отображения проводника на блок-диаграмме используйте щелчок ПКМ по проводнику и выберите *Clean Up Wire*. Пример применения данной операции представлен на рисунке 13.

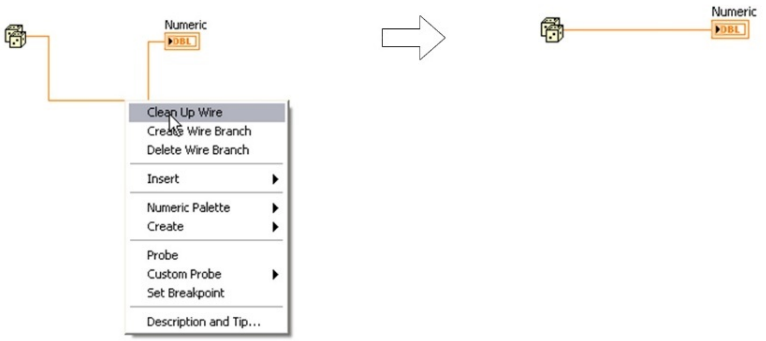


Рис. 13

Пример упорядочивания проводников на блок-диаграмме

Аналогичную операцию можно проводить к фрагменту блок-диаграммы, предварительно выделив необходимую часть либо применив соответствующий инструмент.

1.5. Принцип потока данных

LabVIEW выполняет код, написанный на графическом языке программирования. Вычисления начинаются тогда, когда на входы всех узлов поступают необходимые входные сигналы, при этом последовательность вычисления заранее не задана. В этом и заключается реализация принципа потока данных. Вычисления выполняются одновременно по параллельным каналам.

Сигналы на входах обрабатываются внутри узлов согласно их функциям и приводят к генерации данных на выходе. Таким образом программист, соединяя между собой входы и выходы узлов, реализует несколько потоков вычислений, которые будут выполняться параллельно и независимо друг от друга.

Мы не можем определить, какая из операций будет выполнена первой при реализации потока данных, но все операции будут выполнены точно после поступления всех входных сигналов.

В режиме блок-диаграммы можно наблюдать процесс потокового вычисления, для этого необходимо на инструментальной панели нажать на кнопку «Подсветка выполнения» *Highlight Execution*. В этом режиме также можно увидеть, на выполнение каких операций требуется большее время, на каких — меньшее.

1.6. Примеры виртуальных приборов: преобразование температуры из градусов Цельсия в температуру по Фаренгейту

Ниже приводится описание процесса разработки виртуального прибора, который выполняет преобразование температуры из градусов Цельсия в температуру по Фаренгейту. Как известно из курса физики, перевод осуществляется по формуле

$$t_F = 1{,}8 \cdot (t_C + 32), \qquad (1)$$

где t_F — значение температуры по Фаренгейту; t_C — температура в градусах Цельсия.

Таким образом, на лицевой панели прибора необходим элемент, позволяющий вводить значение температуры в градусах Цельсия и индикатор, показывающий значение температуры по Фаренгейту. На блок-диаграмме необходимы узлы, принимающие значения температуры, выполняющие вычисления по формуле и выводящие полученный результат на индикатор.

Разработку данного прибора начнем с лицевой панели.

1) Необходимо открыть новый виртуальный прибор *New VI*;

2) откройте палитру элементов управления *Controls* щелчком правой кнопки мыши (ПКМ) в любой области лицевой панели;

3) на палитре элементов управления *Controls* в подразделе Числовых элементов *Numeric* выберите цифровой элемент управления *Numeric Control* и поместите его на лицевой панели (рис. 14). Он будет использоваться для ввода данных температуры в градусах Цельсия;

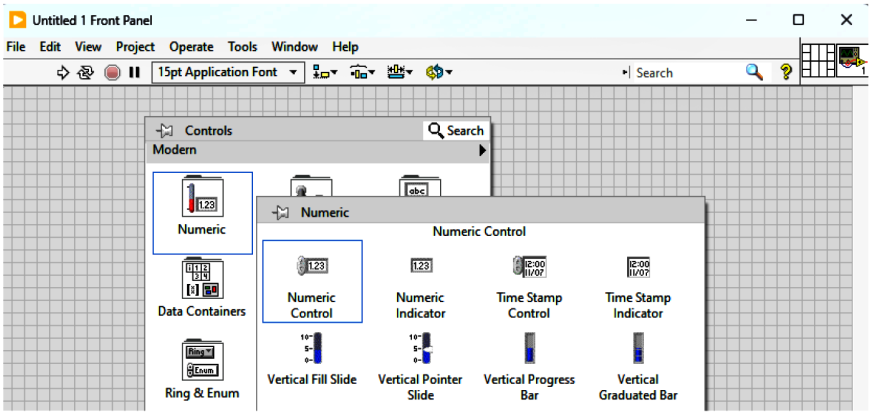


Рис. 14

Выбор цифрового элемента управления

4) введите название цифрового элемента управления — «Градусы, С» в поле собственной метки и щелкните ЛКМ в свободном месте экрана. Если не присвоить название, то *LabVIEW* присвоит элементу название по умолчанию. Изменить название элемента можно в любой момент с помощью панели инструментов *Tools Pallet*;

5) для отображения температуры по Фаренгейту выберите на панели управления цифровой элемент индикации *Controls* → *Numeric* → *Numeric Indicator* (рис. 15) и введите его название «Градусы, F»;

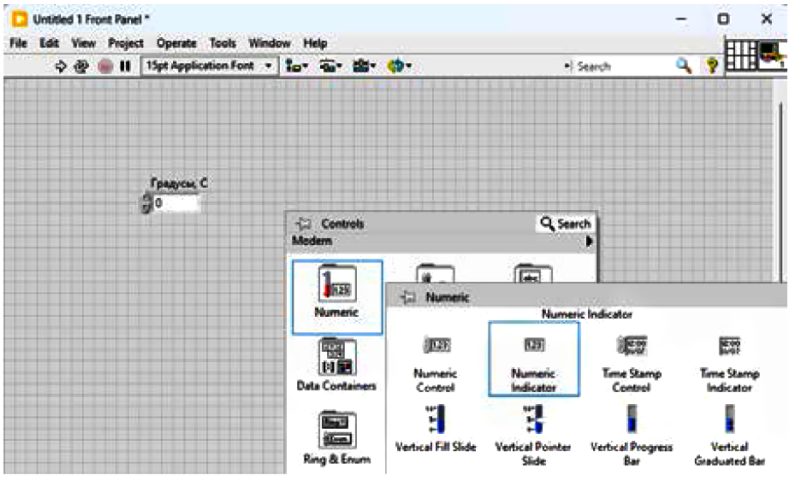


Рис. 15

Выбор цифрового элемента индикации

6) терминалы цифрового элемента управления и индикации автоматически появляются на блок-диаграмме (рис. 16). Для реализации принципа потока данных представлены их различные типы, которые отличаются характеристиками, цветом, значениями по умолчанию. Перечень типов данных приведен в Приложении 1. В данном ВП цифровому элементу «Градусы, С» соответству-

ет тип числовых данных двойной точности с плавающей запятой *DBL*, отображается оранжевым цветом;

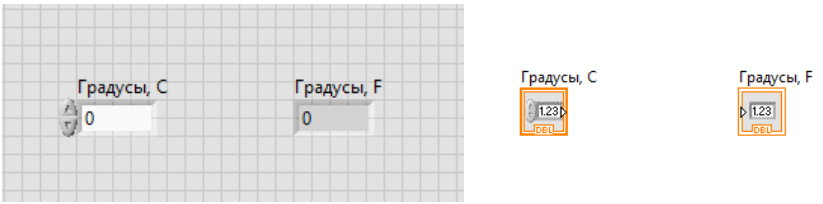


Рис. 16

Лицевая панель и блок-диаграмма ВП преобразования температуры

7) перейдите на блок-диаграмму (для перемещения между окнами можно использовать пункты главного меню *Window* → *Show Block Diagram* или сочетание клавиш *Ctrl* + *E*), откройте палитру функций *Functions*, щелкнув ПКМ в любой свободной области блок-диаграммы.

8) выберите функцию умножения *Multiply* из палитры *Functions* → *Programming* → *Numeric* и поместите ее на блок-диаграмму;

9) выберите функцию сложения *Add* аналогичным образом (*Functions* → *Programming* → *Numeric*) и поместите ее на блок-диаграмму;

10) для реализации формулы необходимо добавить постоянные величины 1,8 и 35 (константы). Для этого выберите *Functions* → *Programming* → *Numeric* → *Constant*. Поместите две константы на блок-диаграмму и введите их значения 1,8 и 35 соответственно (рис. 17). Ввод значений можно осуществлять сразу после помещения на диаграмму, либо позже, используя инструмент «Ввод текста», расположенный в палитре *Tools Palette* (*View* → *Tools Palette*);

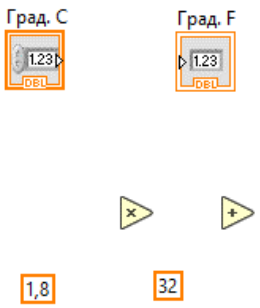


Рис. 17

Блок-диаграмма ВП преобразования температуры

11) соедините объекты диаграммы проводниками с помощью инструмента «Соединение», для этого следует нажать ЛКМ на одном из объектов и, не отпуская кнопку, перевести инструмент на другой объект. Последовательность соединения не имеет значения. Проводник данных можно изгибать, закрепив его щелчком ЛКМ и переместив курсор в перпендикулярном направлении. Нажав пробел, можно переключить направление движения проводника. Для

отмены начала соединения следует нажать *Esc*, либо нажать ПКМ, либо щелкнуть инструментом «Соединение» на поле источник соединения;

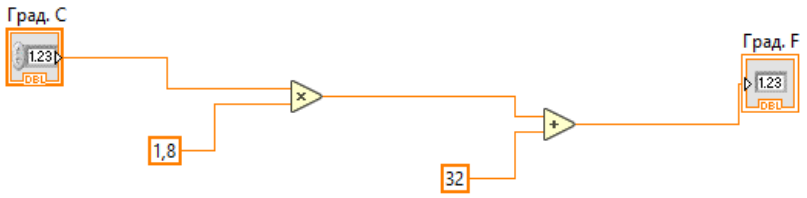


Рис. 18

Блок-диаграмма ВП преобразования температуры с проводниками

12) перейдите на лицевую панель ВП (*Window* → *Show Front Panel*);

13) сохраните ВП под именем «Преобразование C в F»;

14) запустите виртуальный прибор, введите несколько значений температуры, зафиксируйте результаты преобразования (рис. 19);

15) преобразуйте полученный виртуальный прибор в подпрограмму *Sub VI*. Для этого щелкните ПКМ по иконке *Icon*, расположенной в правом верхнем углу ВП (рис. 20*а*) и в контекстном меню выберите пункт «Редактирование иконки» *Edit Icon*. Появится диалоговое окно редактора иконки *Icon Editor*;

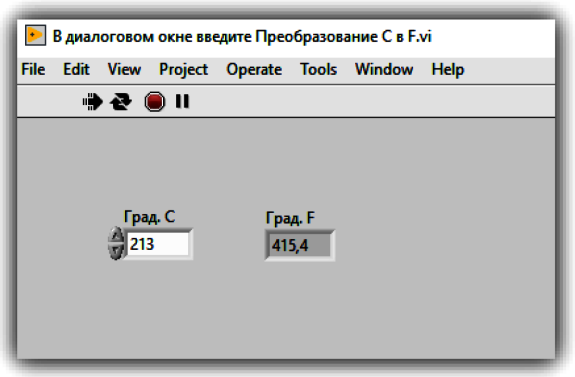


Рис. 19

Лицевая панель работающего виртуального прибора

16) дважды щелкните ПКМ по инструменту Выбор *Select*  и, нажав кнопку *Delete*, очистите область редактирования иконки;

17) дважды щелкните ЛКМ по инструменту Прямоугольник *Rectangle*  , чтобы обвести область редактирования границей выбранного цвета;

18) создайте следующую иконку, как на рисунке 20*б*;

19) перейдите на лицевую панель, щелкните ПКМ по Панели подключения *Connector Pane* (рис. 20*б*) и выберите пункт *Patterns* из контекстного меню. Количество отображаемых *LabVIEW* полей ввода/вывода данных соответствует количеству элементов на лицевой панели. Например, лицевая панель этого ВП

имеет два элемента «Градусы, С» и «Градусы, F», соответственно, выберите в соединительной панели два поля ;

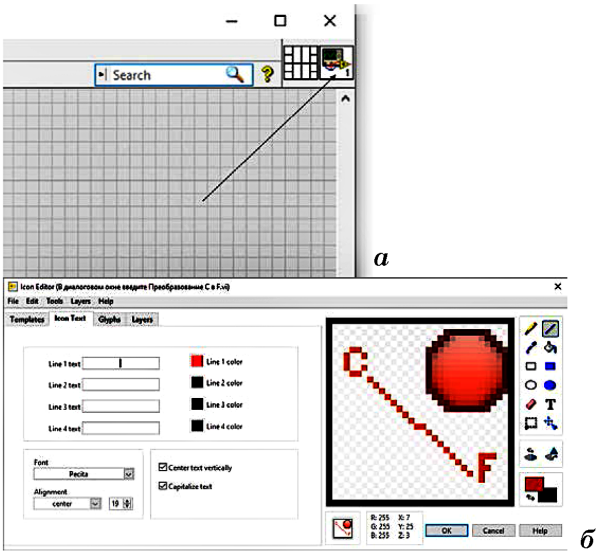


Рис. 20

Создание иконки ВП:

а — вызов меню; *б* — диалоговое окно редактирования.

20) для просмотра полученных соединений нажмите на кнопку контекстной помощи на панели инструментов или *Help* → *Show Context Help* (рис. 21);

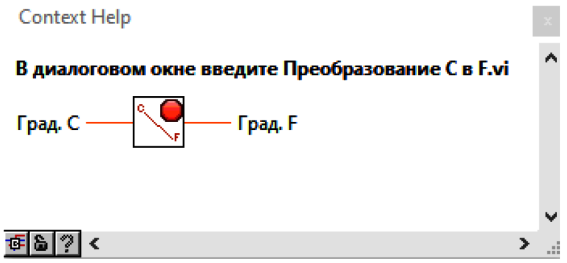


Рис. 21

Окно контекстной помощи иконки ВП

21) щелкните ЛКМ на левом поле соединительной панели, а затем ЛКМ по элементу «Градусы, С» (рис. 22). Левое поле соединительной панели окрасится в оранжевый цвет, что соответствует типу выбранных данных *DBL*. Щелкните курсором в любом свободном месте;

22) щелкните ЛКМ по правому полю соединительной панели и элементу «Градусы, F». Правое поле также станет оранжевым (рис. 23);

23) сохраните виртуальный прибор *File* → *Save as* под именем «C v F.vi».

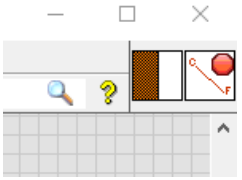


Рис. 22
Подключение соединительной панели ВП

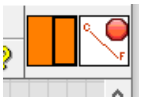


Рис. 23
Соединительная панель и иконка ВП

В представленном виртуальном приборе были рассмотрены принципы реализации в *LabVIEW* простейших математических вычислений.

Задание:

1. Измените виртуальный прибор таким образом, чтобы он выполнял перевод температуры из градусов по Цельсию в Кельвины.

1.7. Числовые функции в LabVIEW

Для выполнения математических операций над числами в *Lab VIEW* разработана специальная палитра Математика *Mathematics*. Она располагается в палитре функций *Functions* → *Mathematics* (рис. 24).

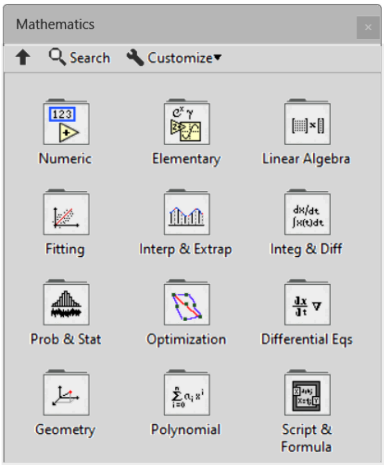


Рис. 24
Палитра математических функций

В ее состав входят числовые функции (*Numeric*), элементарные функции (*Elementary*), линейная алгебра (*Linear Algebra*), аппроксимация (*Fitting*), интерполяция и экстраполяция (*Interp & Extrap*), интегрирование и дифференцирование (*Integ & Diff*), вероятность и статистика (*Prob & Stat*), оптимизация

(*Optimization*), дифференциальные вычисления (*Differential Eqp*), геометрия (*Geometry*), полиномы (*Polynomial*), структуры скриптов и формул (*Script & Formula*).

С помощью данной палитры, как следует из перечисленных вкладок, можно выполнить разнообразные математические вычисления.

В Приложении 2 представлено описание некоторых математических функций *LabVIEW*. Исчерпывающую информацию можно найти в контекстной помощи любого узла, функции, элемента.

При выполнении вычислений с большим количеством переменных или в случае сложных расчетов рекомендуется использовать структуру «Узел формул» *Formula Node* (рис. 25).

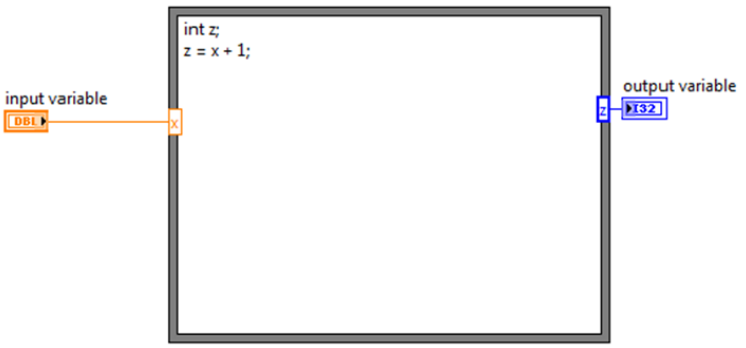


Рис. 25
Структура узла формул Formula Node

Программирование в данной структуре осуществляется по принципу текстовых языков программирования, при этом значения переменных на вход и на выход подаются с помощью графического языка программирования. Для добавления входной величины необходимо щелкнуть правой кнопкой мыши по рамке структуры и добавить вход (*Add input*) или выход (*Add output*), соответственно. На рамке структуры возникнут входные или выходные переменные, в которые нужно вписать их название. Названия переменных должны совпадать с именами переменных в поле структуры с учетом регистра. Запись каждой строки в поле структуры заканчивается символом «;».

Для программирования используются функции и операторы, представленные в Приложении 3.

1.8. Примеры виртуальных приборов: вычисление с помощью формул

Разработаем виртуальный прибор, который выполняет вычисление двух эквивалентных числовых формул $y1 \sim y2$, $z1 \sim z2$ с помощью математических функций и с помощью узла формул. Вычисление числовых формул $y1$, $z1$ выполнить с помощью структуры узла формул *Formula Node*, $y2$, $z2$ выполнить с помощью математических функций.

Исходные данные: $m = 2, 3$; $\alpha = 0, 23$; $\beta = 1, 2$.
Формулы:

$$y_1 = \frac{m^2 - m - 6 - (m+3)\sqrt{m^2-4}}{m^2 + m - 6 - (m-3)\sqrt{m^2-4}}, \; y_2 = \frac{-\sqrt{m+2}}{\sqrt{m-2}},$$

$$z_1 = (\cos\alpha - \cos\beta)^2 + (\sin\alpha - \sin\beta)^2, \; z_2 = 4\sin^2\frac{\alpha - \beta}{2}.$$

1. Разработаем лицевую панель прибора, выбрав три простых цифровых элемента с возможностью ввода значения (*Controls* → *Classic* → *Numeric* → *Simple Numeric*) и четыре цифровых элемента индикации (рис. 26), присвоим им названия. Соответствующие им терминалы появляются на блок-диаграмме.

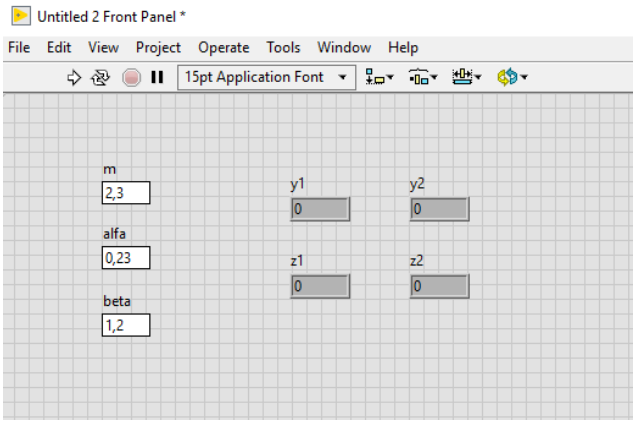


Рис. 26
Лицевая панель ВП вычисления с помощью формул

2. Выполним вычисления y_2, z_2 с помощью формул. Для этого поместим на блок-диаграмму математические функции: сложение, вычитание, деление, квадратный корень, функция *sin*, возведение в квадрат, также числовые константы.

3. Для добавления узла формул на блок-диаграмму выберите на панели функций *Functions* → *Programming* → *Structures* → *Formula Node*.

4. Для добавления входных и выходных значений щелкните по рамке узла и выберите добавить вход *Add input* и добавить выход *Add output*, назовите входные и выходные константы (рис. 27).

5. Для узла формул будем использовать следующие выражения:

```
y1=(m**2-m-6-(m-3) *sqrt(m**2-4));
y1=y1/(m**2+m-6-(m-3) *sqrt(m**2-4));
z1=pow(cos(a)-cos(b),2) +pow(sin(a)-sin(b),2);
```

6. Блок-диаграмма ВП представлена на рисунке 28.

7. Запустите ВП, зафиксируйте значения функций y_1, y_2, z_1, z_2, сравните результаты вычислений с помощью узла формул и с помощью математических функций.

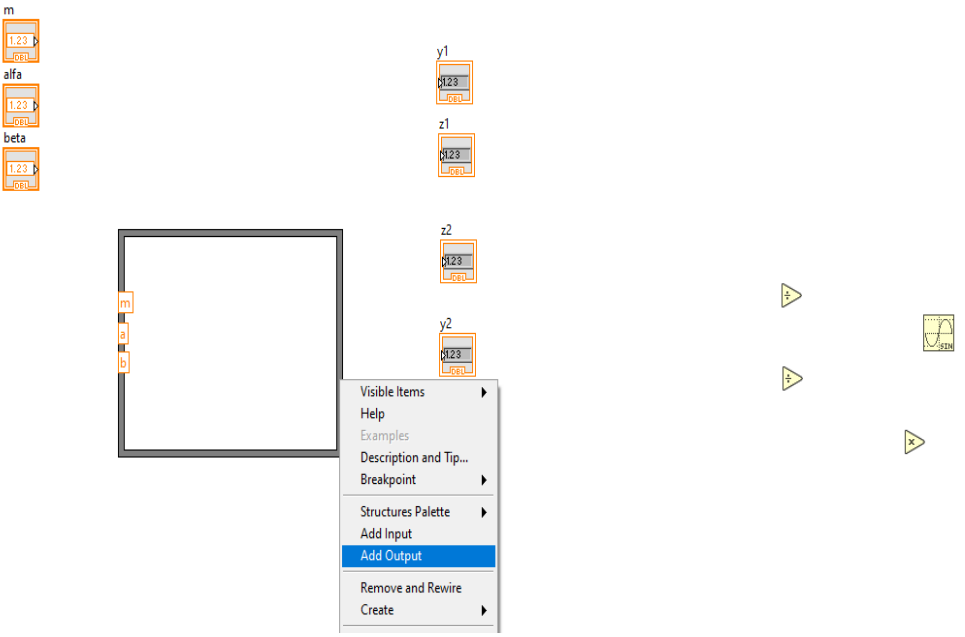


Рис. 27

Создание блок-диаграммы ВП

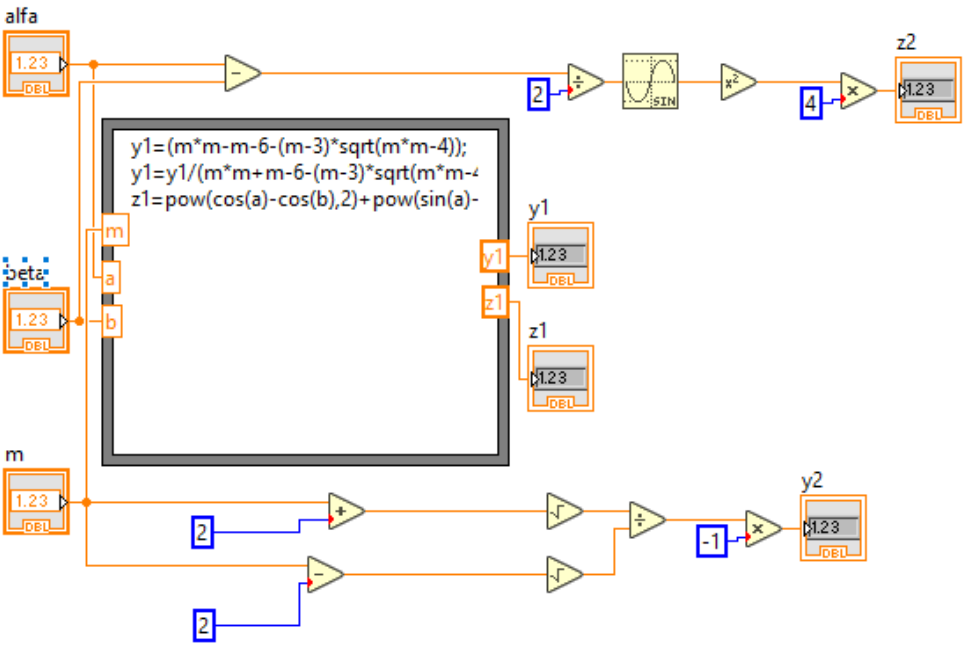


Рис. 28

Блок-диаграмма ВП вычисления по формулам

1.9. Циклы. Цикл с фиксированным числом итераций For Loop

В *LabVIEW* имеются несколько видов структур, которые являются графическим представлением операторов цикла, аналогичных текстовым языкам программирования. Структуры позволяют выполнять операции над потоком данных в определенной последовательности или по условию. Среда *LabVIEW* содержит пять структур: цикл по условию *While Loop*, цикл с фиксированным числом итераций *For Loop*, структура варианта *Case Structure*, структура последовательности *Sequence Structure*, структура события *Event Structure*, а также узел формулы *Formula Node*, который был рассмотрен выше. Все они находятся в палитре функций *Functions* → *Programming* → *Structures*.

Цикл с фиксированным числом итераций *For Loop* выполняет повторяющиеся операции над потоком данных определенное количество раз (по заданному значению).

Аналог текстового кода, иллюстрирующего работу данного цикла, можно представить следующим образом:

```
N = 100;
I = 0;
Until i = N;
   Repeat (code, i = i + 1);
End.
```

Изображение цикла приведено на рисунке 29, где:

1) тело цикла — содержит инструкции, которые необходимо повторять;

2) *N* — максимальное число итераций цикла, определяет количество операций над потоком данных;

3) *i* — счетчик цикла, начальное значение равно 0;

4) выходные терминалы для отображения массива значений данных после выполнения всех итераций цикла.

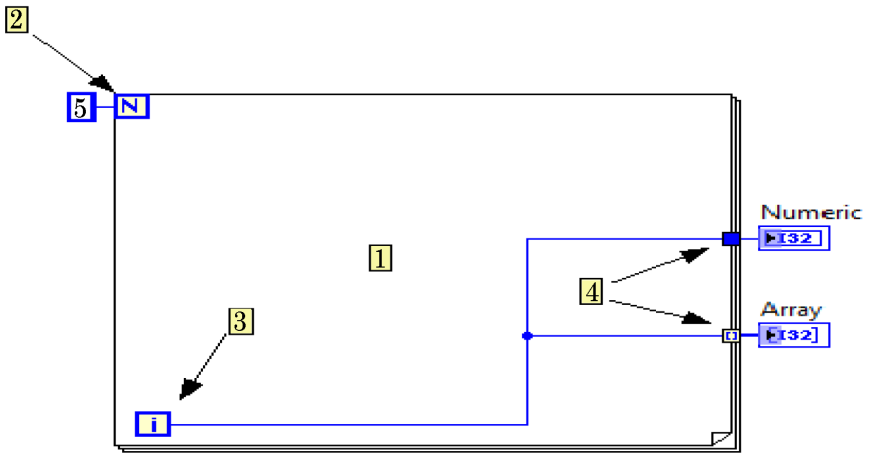


Рис. 29

Цикл с фиксированным числом итераций For Loop

1.10. Цикл по условию While Loop

Цикл по условию *While Loop* работает до тех пор, пока не выполнится логическое условие выхода из цикла. Цикл *While Loop* аналогичен циклам *Do* и *Repeat Until*, используемым в текстовых языках программирования.

Аналог текстового кода, иллюстрирующего работу данного цикла, можно представить следующим образом:

```
Repeat (code);
Until Conditional met;
End.
```

Изображение цикла приведено на рисунке 30, где:

1) *i* — счетчик итераций цикла;

2) терминал условия выхода (завершения цикла). По умолчанию цикл выполняется до поступления на терминал значения «Истина», в этом случае остановка цикла выполняется, если условие на входе цикла истинно (*Stop if True*), например, происходит нажатие на кнопку *Stop* на лицевой панели прибора. Терминал можно переключить в режим завершения работы, если условие ложно *Continue if True*. Для этого достаточно щелкнуть по терминалу ПКМ;

3) тело цикла содержит инструкции операций над данными, которые необходимо повторять;

4) выходные терминалы, на которых появляется значение после выполнения цикла. По умолчанию на выходе терминал скалярного вида.

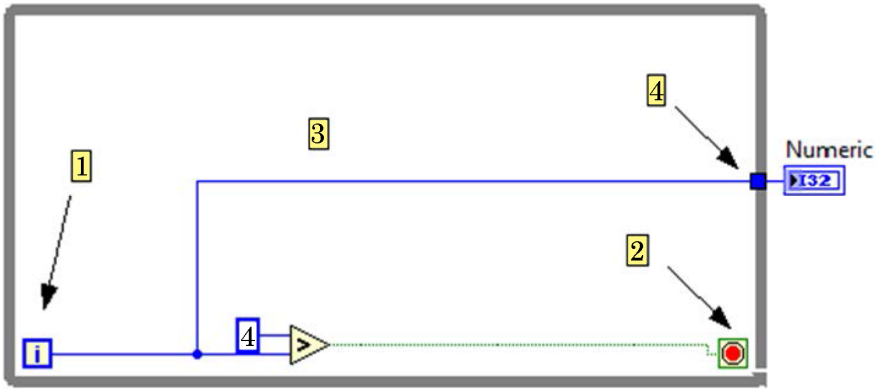


Рис. 30
Цикл по условию While Loop

1.11. Сдвиговые регистры

Для передачи значений данных от текущей итерации цикла к следующей осуществляется сдвиговыми регистрами. Они аналогичны статическим переменным в текстовых языках программирования.

В ВП с циклами часто требуется использовать значения данных, полученные в предыдущих итерациях, например, в случае ВП для измерения температуры. Для отображения текущего значения температуры на графике необхо-

димо использовать значения предыдущих итераций. Именно в этом случае используются сдвиговые регистры *Shift Register*.

Сдвиговый регистр создается щелчком ПКМ по границе цикла, и выбором из контекстного меню пункта добавить сдвиговый регистр *Add Shift Register*. Представляют они собой два терминала, расположенные на противоположных вертикальных границах цикла. Правый терминал содержит стрелку «вверх», сохраняет данные по завершении текущей итерации и передает данные с этого регистра в следующую итерацию цикла. Может передавать любой тип данных, и автоматически принимает вид поданных на него данных.

Для инициализации сдвигового регистра необходимо подать на его левый терминал (стрелка «вниз») необходимое значение вне тела цикла. Если значение не подано, то регистр использует значение, записанное в него во время последнего выполнения цикла или значение по умолчанию для этого типа данных.

Чтобы сохранить информацию о состоянии между последующими запусками ВП, следует оставить вход левого терминала сдвигового регистра неопределенным, т. е. не подавать на него никакие данные. В этом случае по завершении цикла в регистр запишется последнее значение, которое при последующей передаче данных будет передано через правый терминал. В одном цикле можно создавать несколько сдвиговых регистров.

На рисунке 31 приведен цикл по условию и сдвиговые регистры.

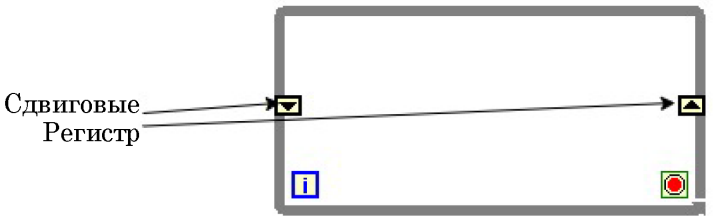


Рис. 31
Цикл по условию и сдвиговые регистры

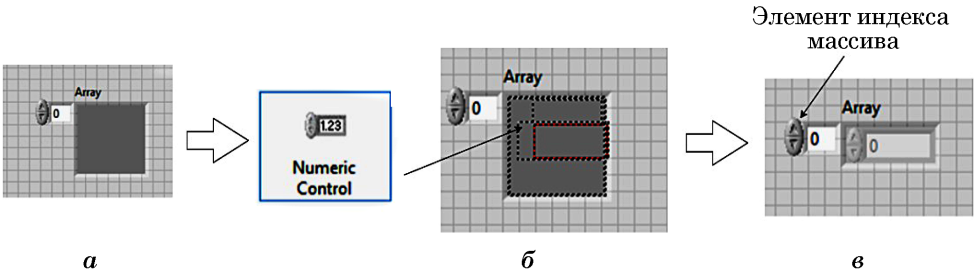
1.12. Массивы

Массив представляет собой набор данных (элементов) определенной размерности, расположенных в памяти непосредственно друг за другом. Массив является структурой с произвольным доступом, т. е. программист может осуществлять доступ к любому элементу по его индексу в произвольном порядке. Массивы бывают одномерными (с одним индексом), двумерные (с двумя индексами) и т. д. В математике двумерный массив представляется в виде матрицы.

Основными характеристиками массива является его размерность (одномерный, двумерный), тип данных (целочисленный, логический или строковый), размер (количество элементов, например, по строкам и столбцам). Массивы идеально подходят для хранения данных, полученных из графиков или накопленных во время работы циклов, причем одна итерация цикла создает один элемент массива.

Массивы в *LabVIEW* являются динамическими и не требуют предварительного определения хранимых объемов данных. Максимальное количество элементов на каждую размерность составляет 2^{31}. Значения индексов лежат в диапазоне от 0 до N–1, где N — количество элементов массива.

Массивы располагаются на лицевой панели на палитре элементов управления *Controls* → *Data Containers* → *Array*. При добавлении на лицевую панель массив выглядит так, как представлено на рисунке 32*а*. Следующим шагом необходимо добавить элемент, из которых будет состоять массив (рис. 32*б)*. В результате массив примет вид рисунка 32*в*.



а *б* *в*

Рис. 32

Создание массива на лицевой панели:

а — добавление из палитры; *б* — добавление элемента массива;
в — вид одномерного массива.

Таким образом, при создании целочисленного массива данных необходимо в серое поле массива добавить числовой элемент управления *Numeric Control* (рис. 32*б*). Тогда одномерный массив примет вид как на рисунке 32*в*. Вид массива определяется типом данных, добавленных в его поле. Для получения массива.

Для его преобразования в двумерный массив необходимо по элементу индекса массива щелкнуть правой кнопкой мыши и выбрать из контекстного меню Добавить размерность *Add Dimension*.

Для создания массива констант на блок-диаграмме необходимо выбрать *Functions* → *Array* → *Array Constant* и поместить внутрь массива константу нужного типа, например, *Functions* → *Numeric* → *Numeric Constant*.

Для первого опыта работы с массивами создайте виртуальный прибор (рис. 33).

Для этого используйте шаги, представленные на рисунке 32.

Массивы могут создаваться с помощью циклов. Цикл с фиксированным числом итераций *For Loop* и цикл по условию *While Loop* автоматически накапливают массивы и проводят их индексацию на своих границах. Это свойство называется автоиндексацией. Для создания массива необходимо соединить терминал данных массива с терминалом выхода из цикла. В этом случае каждая итерация цикла будет создавать новый элемент массива (рис. 34).

Автоиндексация автоматически включена всегда в цикле *For Loop*, так как данный цикл часто используется для создания массивов. Для отключения автоиндексации необходимо щелкнуть правой кнопкой мыши на терминале

входа/выхода из цикла и выбрать в открывшемся меню Отключить автоиндексацию *Disable Indexing*.

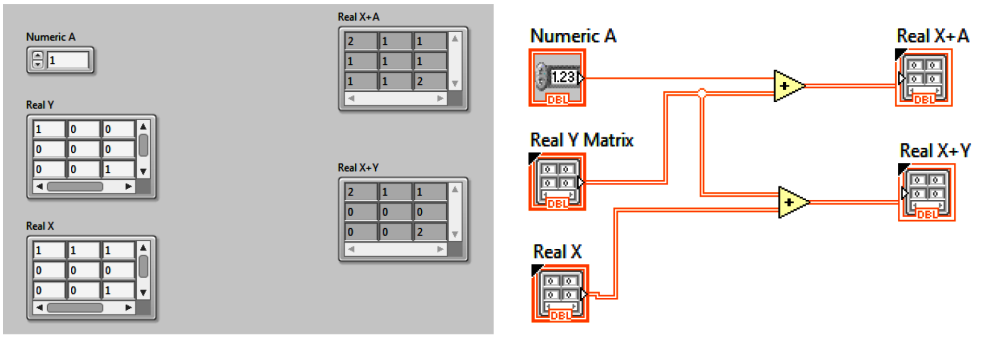


Рис. 33
ВП работы с массивами

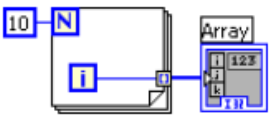


Рис. 34
Создание массива с помощью цикла

Для создания двумерных массивов необходимо использовать два цикла, один внутри другого.

1.13. Примеры виртуальных приборов: вычисление чисел Фибоначчи

Числа Фибоначчи — это числа, вычисляемые по алгоритму: каждое последующее число вычисляется как сумма двух предыдущих. Первые два числа, как правило, равны 0 и 1. Разработаем ВП, вычисляющий числа Фибоначчи.

Последовательность чисел: 0, 1, (0 + 1 = 1), (1 + 1 = 2), (1 + 2 = 3), (2 + 3 = 5), (3 + 5 = 8) и т. д.

Для реализации ВП используем цикл с фиксированным числом итераций и сдвиговые регистры.

1. Разместите на блок-диаграмме цикл с фиксированным числом итераций *For Loop*.

2. На лицевую панель добавьте цифровой элемент управления *Numeric Control* и введите его название «*N*», с его помощью будет задаваться номер числа Фибоначчи. Рядом разместите цифровой индикатор *Numeric Indicator*, на который будет выводиться значение числа.

3. На блок-диаграмме соедините терминал цифрового элемента управления «*N*» с максимальным числом итераций *N* цикла.

4. Добавьте сдвиговый регистр *Add Shift Register*, далее добавьте еще один левый терминал сдвигового регистра. Для этого щелкните ПКМ на левом терминале и выберите Добавить элемент *Add Element*.

5. Для инициализации сдвигового регистра на левый терминал подайте первые значения чисел Фибоначчи: 0 и 1. Для этого создайте константы.

6. В теле цикла разместите сумматор *Add* и подайте на него значения левых терминалов сдвигового регистра, сигнал с выхода сумматора подайте на цифровой элемент индикации *Numeric*, а также на правый терминал сдвигового регистра.

7. Запустите виртуальный прибор. Лицевая панель и блок-диаграмма ВП, выполняющего вычисление чисел Фибоначчи, приведены на рисунке 35.

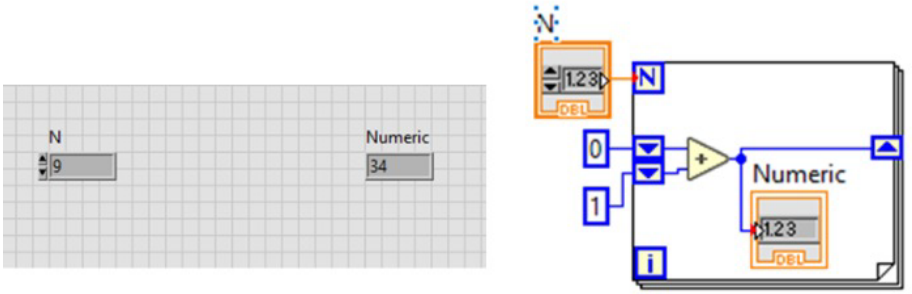


Рис. 35

Виртуальный прибор вычисления чисел Фибоначчи

8. Внесите изменения в ВП, добавив функцию ожидания *Wait* в тело цикла (рис. 36), *Function* → *Programming* → *Timing*.

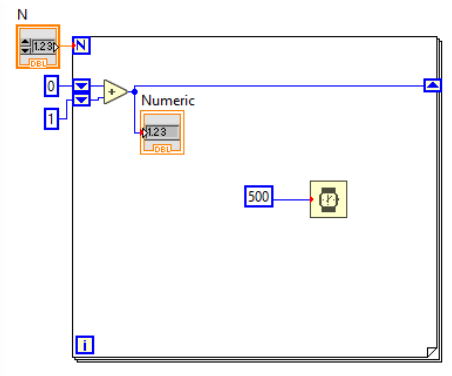


Рис. 36

Виртуальный прибор вычисления чисел Фибоначчи с ожиданием

Если с выхода цикла подать сигнал на массив, то можно отобразить на лицевой панели значения нескольких чисел одновременно, например, первых десяти чисел.

9. Для создания массива выберите элемент Массив *Array* (*Controls* → *Data Containers*) и разместите на лицевой панели. Внутрь него поместите цифровой элемент индикации *Numeric Indicator* и растяните его вниз на десять элементов. Выполните соединения на блок-диаграмме. Тогда лицевая панель и блок-диаграмма ВП будут выглядеть следующим образом (рис. 37).

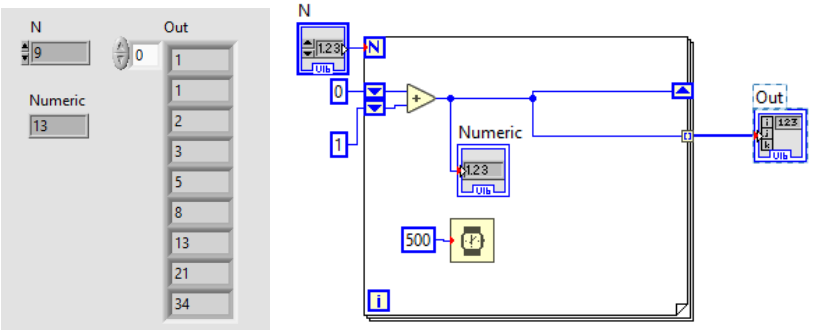


Рис. 37

Виртуальный прибор вычисления чисел Фибоначчи и их отображение в виде массива

На данной блок-диаграмме также изменены типы данных. На рисунке 36 цифровые элементы управления и индикации определены как числовой тип данных с плавающей запятой двойной точности *DBL*.

10. Для изменения типа данных щелкните правой кнопкой мыши по элементу управления *N* и выберите пункт контекстного меню Представление *Representation* → Беззнаковое 16-битовое целое число *U16*. Выполните аналогичные действия для элемента индикации *Numeric Indicator*. Цвет терминалов элементов изменится на синий.

11. Запустите виртуальный прибор.

Задание:

1. Доработайте ВП таким образом, чтобы он вычислял сумму *N* чисел Фибоначчи.

1.14. Примеры виртуальных приборов: генератор случайных чисел

В соответствии со своим названием ВП генерирует случайные числа в заданном симметричном интервале и выполняет их отображение на индикаторах.

1. Создайте лицевую панель и блок-диаграмму согласно рисунку 38.

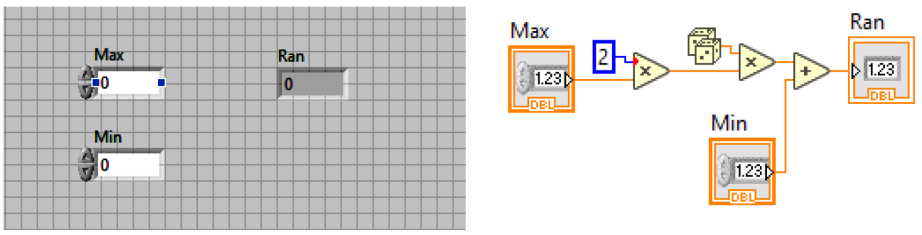


Рис. 38

Генератор случайных чисел: лицевая панель и блок-диаграмма

2. Цифровые индикаторы *Min* и *Max* задают пределы интервала значений, в котором генерируются случайные числа. Стоит учитывать, что интервал должен быть симметричным, например, от –2 (*Min*) до 2 (*Max*) или от –3 (*Min*) до 3 (*Max*).

3. Генератор случайных чисел расположен на палитре *Functions* → *Mathematics* → *Numeric* → *Random Number* (*0–1*). Генератор случайных чисел генерирует числа в интервале от 0 до 1. В соответствии с блок-диаграммой ВП, пределы генерируемых чисел определяются заданным интервалом значений.

4. Запустите виртуальный прибор.

5. Выполните модернизацию виртуального прибора, добавив возможность ввода количества случайных чисел и их вывода в виде массива (рис. 39).

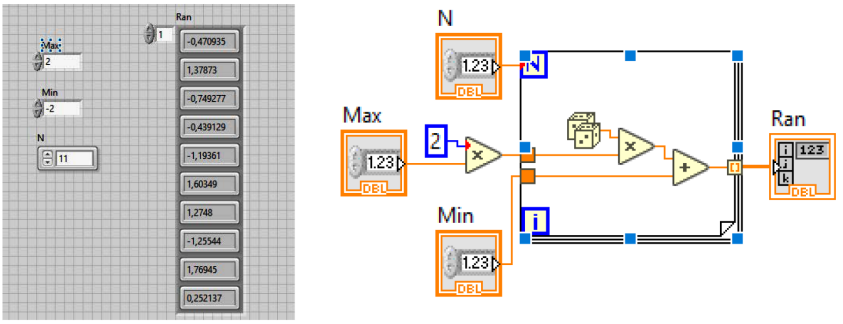


Рис. 39

Модернизация генератора случайных чисел

6. Для остановки ВП добавьте на лицевую панель кнопку Стоп *Stop Button* (*Controls* → *Modern* → *Boolean*).

7. Добавьте на блок-диаграмму функцию Стоп *Stop* (*Functions* → *Programming* → *Application Control*). Соедините проводником с терминалом кнопки Стоп. Блок-диаграмма изменится следующим образом (рис. 40).

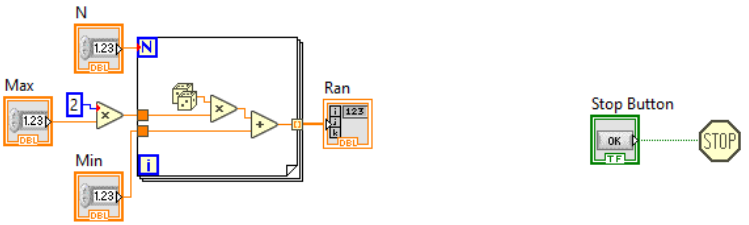


Рис. 40

Блок-диаграмма генератора случайных чисел

8. Преобразуйте ВП в файл с расширением .exe, который можно запускать на любом компьютере. Меню *Tools* → *Built Application (EXE) from VI...*

Задание:

1. Доработайте лицевую панель ВП, оформив ее с помощью вкладки *Controls* → *Decorations*. Выполните необходимые подписи, чтобы по лицевой панели были понятны функции ВП.

ГЛАВА 2
ИНТЕГРАЦИЯ ТЕКСТОВЫХ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ В LABVIEW

2.1. Интеграция скрипта Python в LabVIEW

Среда программирования *LabVIEW* позволяет интегрировать текстовые языки программирования в виртуальные приборы. Это позволяет расширить возможности *LabVIEW* за счет имеющихся библиотек интегрируемых языков. Кроме того, если у пользователей уже имеются *API* на *Python,* они могут их легко добавить в ВП, так как разрабатывать лицевую панель прибора в *LabVIEW* удобнее.

Для удобства разработчика в *LabVIEW* предоставлен узел *Python*, представляющий собой простой *API* для вызова скриптов на *Python* в соответствии с кодом *LabVIEW*. Первоначально открывается сеанс *Python* (*Open Python Session*), где выбирается интерпретатор *Python*, который будет использован. На данный момент поддерживаются все основные версии интерпретаторов *Python*.

Затем выполняется переход к узлу *Python* в коде *LabVIEW* и вызов скрипта *Python Node*, в котором можно использовать любую функциональность *Python*. Также можно осуществлять передачу и получать данные между *LabVIEW* и *Python*. По завершении работы с задачей на *Python*, сеанс закрывается *Close Python Session*.

Одной из мощных особенностей этого потока является возможность использовать несколько экземпляров интерпретаторов *Python* параллельно друг другу. Таким образом, процессы в *LabVIEW* могут взаимодействовать с различными сессиями *Python*, что позволяет выполнять параллельные задачи и использовать множество экземпляров языка в одном и том же процессе *LabVIEW*.

Такая интеграция между *LabVIEW* и *Python* предоставляет пользователю удобный инструмент для решения разнообразных задач и повышает эффективность рабочего процесса.

Разработаем ВП, выполняющий суммирование двух целых чисел и демонстрирующий результат вычислений.

1. Создайте лицевую панель ВП согласно рисунку 41.

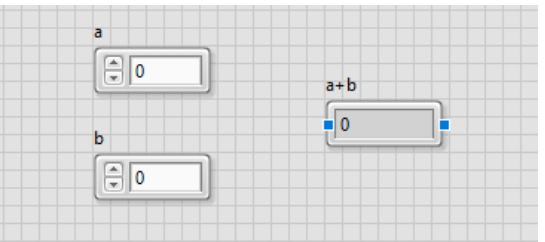


Рис. 41
Лицевая панель ВП

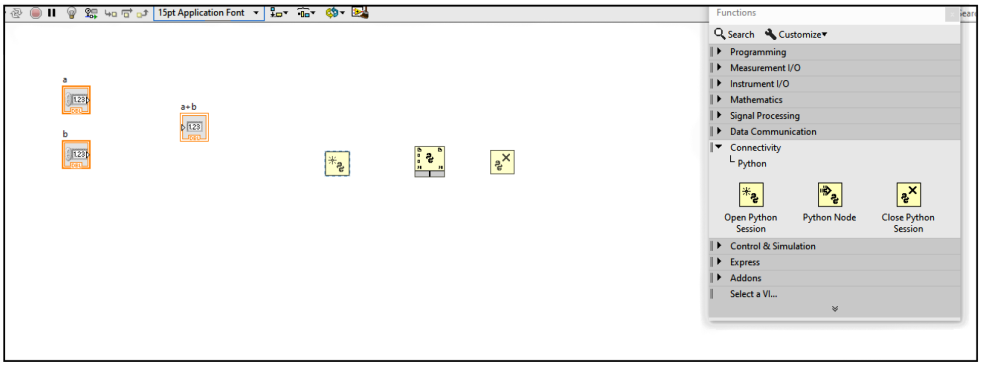


Рис. 42

Добавление узлов для интеграции скрипта Python в LabVIEW

2. На блок-диаграмме разместите узлы интеграции *Python* (рис. 42): Открытие сессии *Open Python Session*, узел скрипта *Python Node* и Закрытие сессии *Close Python Session* (*Functions* → *Connectivity* → *Python*).

3. Для открытия сессии *Python* необходимо указать версию, в которой написан программный код. Для этого необходимо щелкнуть ПКМ в левой верхней части узла, вход — версия языка *Python version* (рис. 43), выбрать Создать константу *Create Constant*, ввести в созданном поле номер версии. Номер версии должен совпадать с версией, в которой создан код. Константа имеет формат строковых данных.

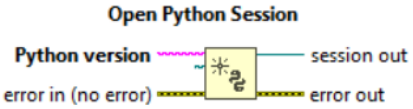


Рис. 43

Обозначение входов и выходов узла открытие сессии Open Python Session

4. На узле скрипта *Python Node* найдите вход имя функции *function name*, щелкните ПКМ и создайте константу *Create Constant*, введите название сложение *Add* (совпадает с оператором def в файле кода). Блок-диаграмма примет вид, как на рисунке 44.

5. Создайте константу *Functions* → *Numeric* → *DBL Numeric Constant*, значение константы 0, соедините ее со входом *return type* узла *Python Node*.

6. Выполните соединение входных элементов управления и выходного элемента индикации согласно рисунку 44. Для их соединения растяните вниз узел скрипта *Python Node* на два дополнительных элемента.

7. Скрипт Python, интегрируемый в LabVIEW, представлен на рисунке 45. Он должен быть сохранен в виде отдельного файла.

8. Для указания пути к скрипту Python щелкните ПКМ по входу *module path* узла *Python Node* и создайте константу. Введите в поле путь к файлу со скриптом *Python* (рис. 46).

9. Для передачи информации об ошибках, происходящих во время выполнения программ, в *LabVIEW* реализован кластер ошибок. Он представляет

собой специальный тип данных: кластер, состоящий из логического элемента статус *status*, длинного целого кода *132 code* и строки источник *source*. Добавим кластеры ошибок входа и выхода ВП. Для этого выберем *Controls* → *Modern* → *Data Containers* → *Error In 3D.ctl* и *Error Out 3D.ctl*. Выполним соединения на блок-диаграмме, как показано на рисунке 47. Также соединим узлы *Python* между собой.

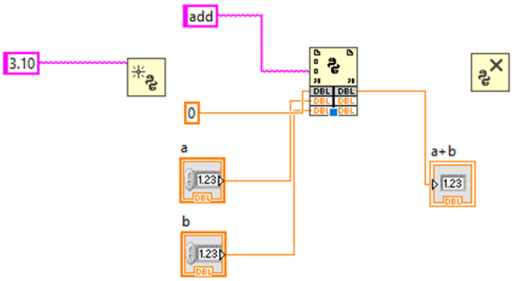


Рис. 44
Выполнение соединений узла Python Node

```
def add(a, b):
    return a + b
```

Рис. 45
Скрипт Python, интегрируемый в LabVIEW

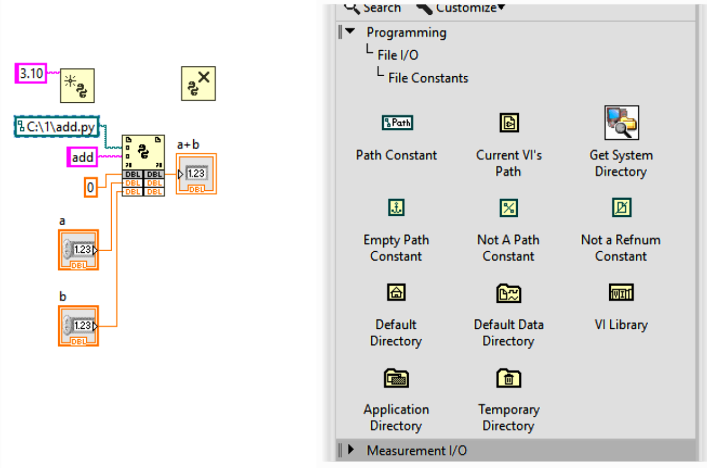


Рис. 46
Добавление пути к файлу со скриптом Python

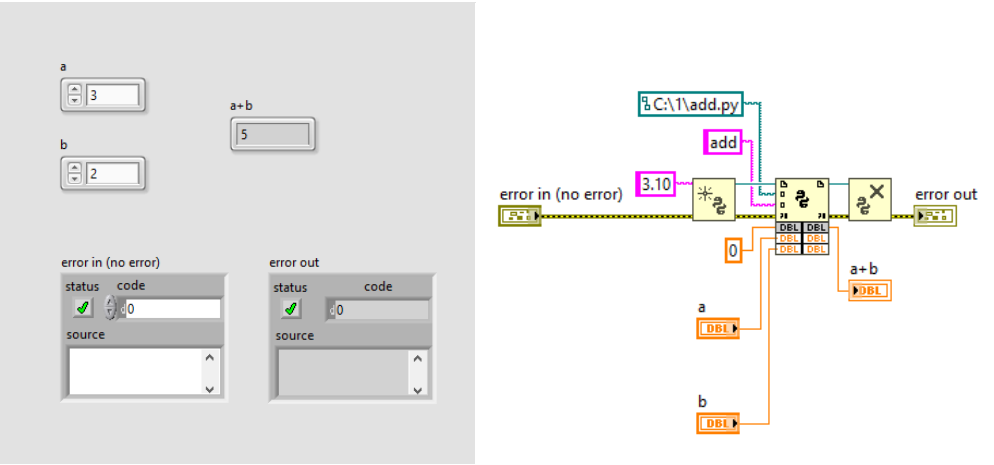


Рис. 47

Виртуальный прибор с интеграцией скрипта Python в LabVIEW: лицевая панель и блок-диаграмма

10. Запустите виртуальный прибор, введите значение $a = 3$, $b = 2$, получите значение суммы, введите другие значения чисел a и b, зафиксируйте результаты.

Задание:

1. Доработайте ВП таким образом, чтобы он также выполнял дополнительно еще умножение и деление двух чисел.

2.2. Интеграция скрипта MATLAB в LabVIEW

Создадим ВП, выполняющий аналогичную математическую операцию, как и в п. 2.1, но с использованием скрипта MATLAB.

1. Лицевую панель ВП создадим аналогично рисунку 41.

2. Добавление скрипта *MATLAB* осуществляется на блок-диаграмме через меню *Functions* → *Mathematics* → *Scripts & Formulas* → *Script Nodes* → *MATLAB script*.

3. Щелкните правой кнопкой мыши по левой области рамки скрипта *MATLAB*, выберите Добавить вход *Add Input*, введите название входа в появившемся поле *A*, аналогично добавьте еще один вход *B*.

4. Щелкните по правой части рамки скрипта *MATLAB*, выберите Добавить выход *Add Output*, введите название выхода в появившемся поле *C*. Выполните соединения с помощью проводников входов и выходов скрипта *MATLAB* с элементами управления и индикации.

5. Введите код в области скрипта *MATLAB*:

```
A=double(A);
B= double(B);
C=A+B;
```

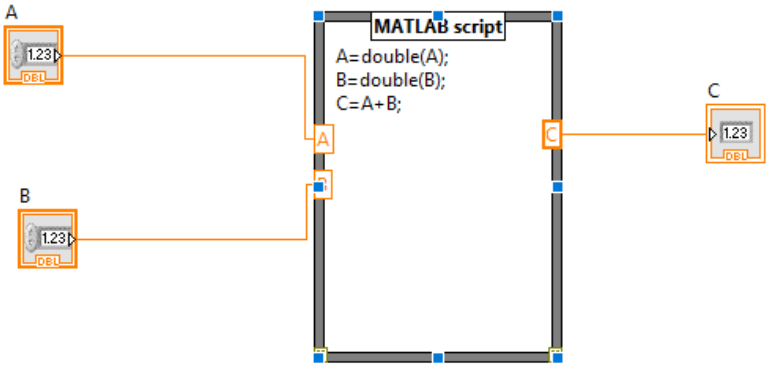


Рис. 48
Блок-диаграмма ВП со скриптом MATLAB

6. Блок-диаграмма ВП должна соответствовать рисунку 48.

7. Запустите виртуальный прибор, введите значение $A = 1$, $B = 2$, получите значение суммы, введите другие значения чисел A и B, зафиксируйте результаты. Для работы ВП необходима установка *MATLAB*. Также возможна интеграция скрипта *MATLAB* аналогично *Python*, т. е. с предварительным созданием файла *MATLAB*.

2.3. Интеграция кода C++ в LabVIEW

Для интеграции кода C++ в *LabVIEW*, как правило, используются следующие два способа:

1) использование *Call Library Function Node*;

2) использование *CIN* (*Code Interface Node*) — узла, позволяющего писать код непосредственно на блок-диаграмме. Данный узел не идет в стандартной комплектации *LabVIEW* и требует дополнительной установки наборов узлов (*Toolkit*). Использование узла *CIN* (*Code Interface Node*) аналогично рассмотренной в предыдущем параграфе интеграции скрипта *MATLAB*.

Рассмотрим первый способ интеграции кода C++ в *LabVIEW*. Разработаем ВП, аналогичный п. 1.16 и 1.17, заменим лишь интегрируемый скрипт с использованием узла *Call Library Function Node*, т. е. интегрируя в *LabVIEW* уже созданный код C++.

1. Лицевую панель ВП создадим аналогично рисунку 41.

2. Создайте C++ библиотеку, экспортирующую функции, которые вы хотите вызвать из *LabVIEW*. Для этого в программе *Visual Studio* создайте новый проект по типу «Мастер классических приложений *Windows*», в настройках проекта выберите «Библиотека динамической компоновки (*DLL*)» (рис. 49).

3. Выполните настройки, как показано на рисунке 50.

4. Создайте код согласно рисунку 51 и сохраните файл под именем *Project.dll*. Данный код представляет собой объявление функции с помощью

спецификаторов extern «C» и _declspec (dllexport). Этот код используется в C++ для определения функции, которая будет экспортирована из динамической библиотеки (*DLL*) и доступна для вызова из других программ или языков программирования.

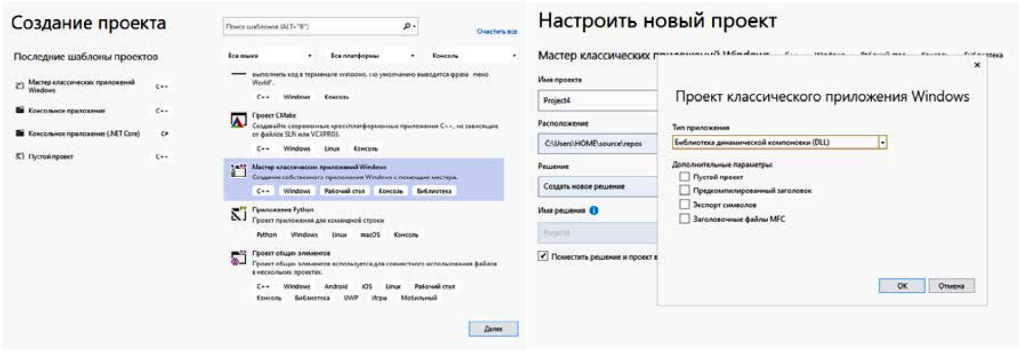


Рис. 49

Создание нового проекта в среде Visual Studio

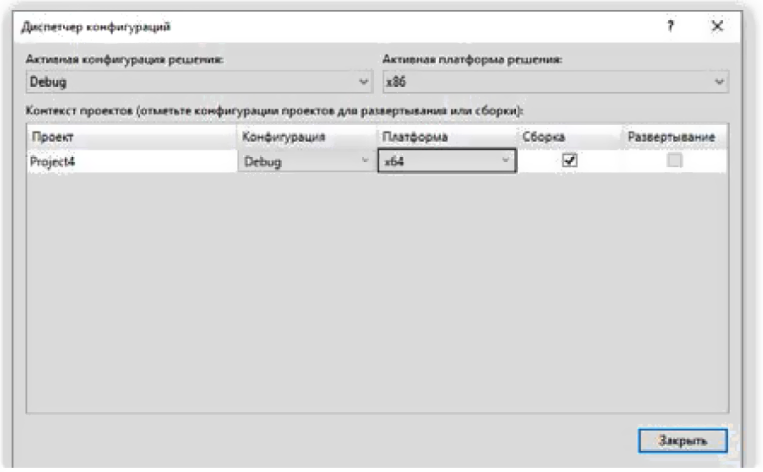


Рис. 50

Настройки проекта в среде Visual Studio

5. Добавьте на блок-диаграмму узел, отвечающий за инициацию кода C++ *Functions* → *Connectivity* → *Libraries & Executables* → *Call Library Function Node*.

6. Выполните настройку узла, для этого щелкните по нему дважды ЛКМ. Укажите путь к файлу с кодом, название функции суммирования *Add*, на вкладке Параметры укажите входные и выходную переменные, их тип и размер. Параметры настройки приведены на рисунке 52.

7. Выполните соединения на блок-диаграмме (рис. 53).

8. Запустите виртуальный прибор, введите значение $a = 1$, $b = 7$, получите значение суммы, введите другие значения чисел a и b, зафиксируйте результаты.

```
1   // dllmain.cpp : Определяет точку входа для приложения DLL.
2   #include "framework.h"
3   extern "C" _declspec(dllexport) int Add(int a, int b);
4   BOOL APIENTRY DllMain( HMODULE hModule,
5                          DWORD  ul_reason_for_call,
6                          LPVOID lpReserved
7                        )
8   {
9       switch (ul_reason_for_call)
10      {
11      case DLL_PROCESS_ATTACH:
12      case DLL_THREAD_ATTACH:
13      case DLL_THREAD_DETACH:
14      case DLL_PROCESS_DETACH:
15          break;
16      }
17      return TRUE;
18  }
19
20  int Add(int a, int b)
21  {
22      return a + b;
23  }
```

Рис. 51
Интегрируемый код C++

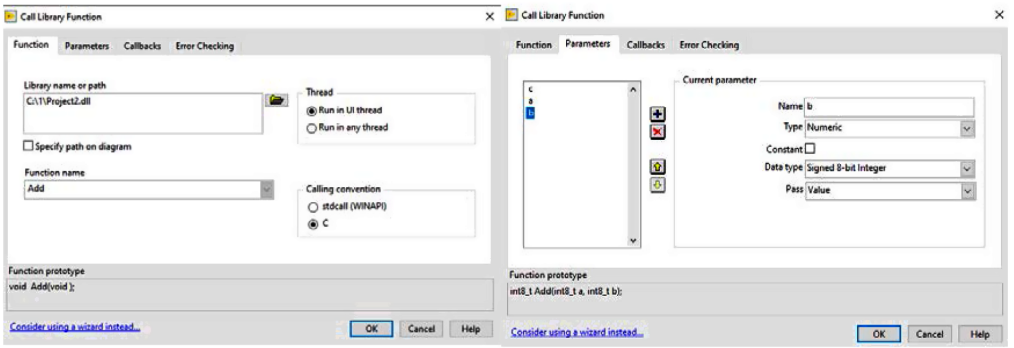


Рис. 52
Параметры настройки узла
Call Library Function Node

9. Выполните соединения на блок-диаграмме (рис. 53).

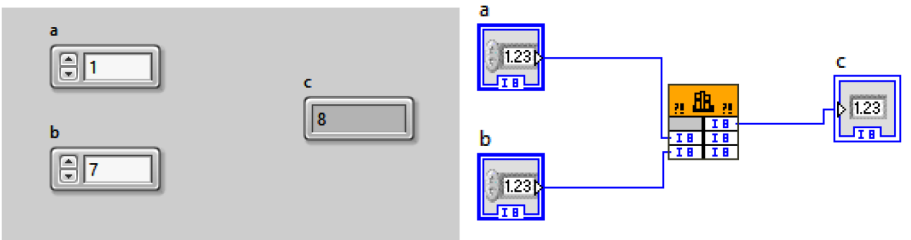


Рис. 53
Виртуальный прибор
с интеграцией кода C++

10. Запустите виртуальный прибор, введите значение $a = 1$, $b = 7$, получите значение суммы, введите другие значения чисел a и b, зафиксируйте результаты.

Задание:

1. Доработайте ВП таким образом, чтобы он выполнял сложение трех чисел, а также выполнял дополнительно еще умножение и деление двух чисел.

ГЛАВА 3
ГЕНЕРАЦИЯ И ЦИФРОВАЯ ОБРАБОТКА СИГНАЛОВ

3.1. Примеры виртуальных приборов: генерация сигналов в LabVIEW с помощью формул

В *LabVIEW* обеспечен широкий пул возможностей для генерации сигналов и их последующей обработки.

Сигнал на вход виртуального прибора может подаваться с реальных датчиков либо может быть сгенерирован программным образом. Рассмотрим способ генерации сигналов с использованием математических функций, в частности, гармонических сигналов с помощью функции Эйлера.

1. На лицевой панели разместите два цифровых элемента управления «Частота» и «Амплитуда» *Horizontal Pointer Slide* (*Controls* → *Modern* → *Numeric*). Выберите тип данных *I32* (32-разрядное целое число со знаком), для этого щелкните ПКМ по элементу, выберите *Representation* → *I32*.

2. Для отображения сигналов будем использовать осциллограмму *Waveform Graph*. Добавьте ее на лицевую панель.

3. На блок-диаграмму добавим цикл с фиксированным числом итераций *For Loop*, количество итераций цикла выберем $N = 200$, что будет соответствовать числу отсчетов сигнала. Щелкните ПКМ по N → *Create Constant*, введите значение 200.

4. Начальную фазу сигнала зададим с помощью математической функции деления *Divine*, на один вход которой подадим значение константы π (*Functions* → *Mathematics* → *Numeric* → *Math & Scientific Constants* → *Pi*), на второй вход целое число 50 (*Functions* → *Programming* → *Numeric* → *DBL Numeric Constant*).

5. Для формирования синусоидального и косинусоидального сигналов с помощью формулы Эйлера необходимо выполнить ряд вычислительных операций. В поле структуры разместите узлы *Compound Arithmetic* (*Functions* → *Programming* → *Numeric*), *Polar To Complex* (*Functions* → *Mathematics* → *Numeric* → *Complex*), *Complex To Re/Im* (*Functions* → *Mathematics* → *Numeric* → *Complex*). Спецификация входов и выходов узлов приведена на рисунке 54.



Рис. 54

Спецификация входов и выходов математических элементов

6. Увеличьте размер узла *Compound Arithmetic* на одну строку вниз и измените выполняемую математическую операцию, для этого щелкните ПКМ по узлу, выберите *Change Mode* → *Multiply*.

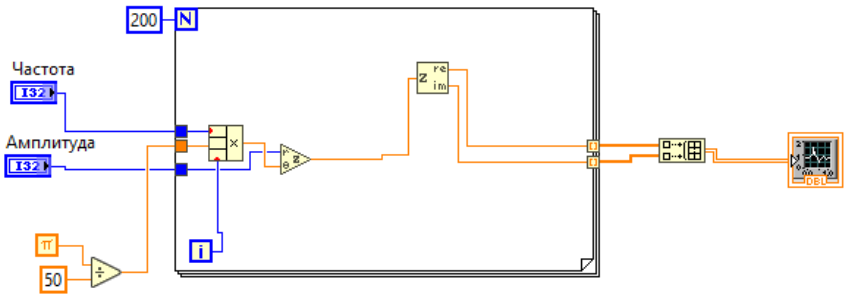


Рис. 55

Блок-диаграмма синусоидального и косинусоидального сигналов

7. Выполните соединения на блок-диаграмме (рис. 55).

8. Добавьте узел для построения массива *Built Array* (*Functions* → *Array*). На входы узла подайте реальную и мнимую части комплексной функции, соедините с терминалом графика.

9. Для возможности изменения частоты и амплитуды сигналов при отображении результатов сразу на графике, добавим на блок-диаграмму структуру *While Loop*.

10. На лицевую панель добавьте кнопку *Stop* и соедините ее терминал с терминалом структуры (рис. 56).

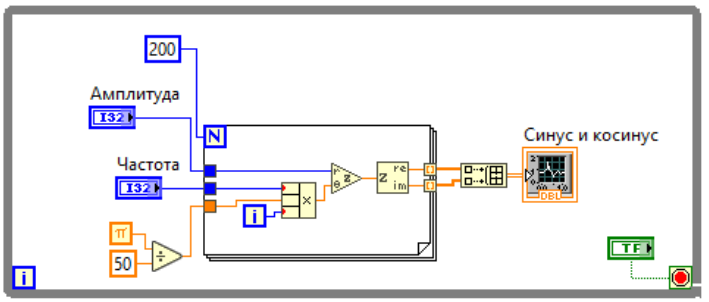


Рис. 56

Блок-диаграмма синусоидального и косинусоидального сигналов с возможностью останова работы

11. Выполним настройку графика на лицевой панели. По осям введем названия «Номер отсчета», «Амплитуда», в названии графика — «Синус и косинус». У осциллограмм есть ряд отображаемых составляющих: название, легенда, курсор и т. д. Щелкните ПКМ по осциллограмме, уберите «галочку» в поле *Visible* → *Label*. Щелкните ПКМ на графике и выберите раздел Свойства *Properties*.

12. На вкладке *Scales* в разделе *Grid Style and Colors* выберите «без сетки» (рис. 57). Повторите операцию, выбрав другую ось «Амплитуда (*Y-Axis*)». На

вкладке *Plots* выберите цвет линии графика *Line* для одного *Plot 0* — синий цвет, для *Plot 1* — красный цвет.

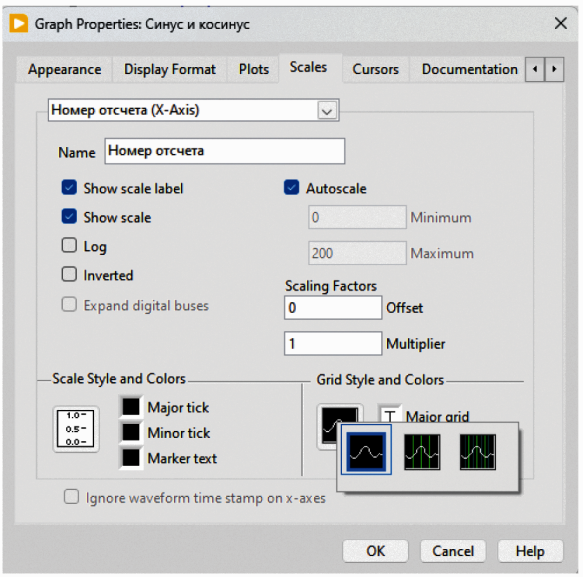


Рис. 57

Диалоговое окно настройки осциллограммы

13. На панели инструментов *Tools Palette* выберите инструмент Кисть, щелкните ЛКМ по фону, выберите белый цвет и щелкните по области осциллограммы.

14. Используйте декоративные элементы для обрамления лицевой панели на вкладке *Controls* → *Modern* → *Decoration*. Пример оформления лицевой панели приведен на рисунке 58.

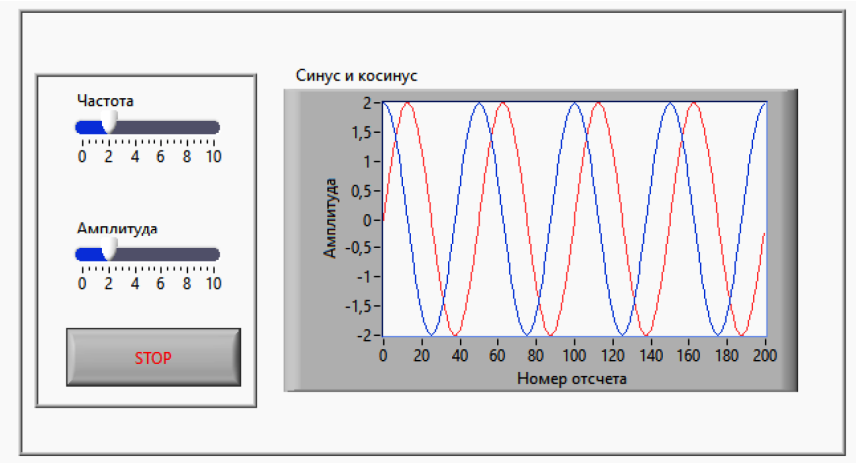


Рис. 58

Лицевая панель виртуального прибора генерации периодических сигналов с помощью формулы Эйлера

15. Запустите ВП, нажав кнопку *Run*, выполните изменения частоты и амплитуды, понаблюдайте за изменениями сигнала. Определите гармонический сигнал, изменяющийся по закону синуса и гармонический сигнал по закону косинуса.

16. Остановите виртуальный прибор, нажав кнопку *Stop* на лицевой панели.

Задание:

1. Измените ВП таким образом, чтобы сигналы отображались на разных осциллограммах, а не на одной.

3.2. Анализатор спектра в LabVIEW. Быстрое преобразование Фурье

Представление сигнала во временной области позволяет получить данные о его мгновенных значениях во времени. Очень часто такой информации недостаточно и требуется знать гармонические составляющие сигнала, т.е. необходимы данные о поведении сигнала в частотной области. Преобразование Фурье позволяет представить сигнал в виде информации об амплитуде и начальной фазе каждой гармонической составляющей. Таким образом мы получаем представление сигнала в частотной области. Алгоритм, по которому выполняется данное преобразование, называется дискретным преобразованием Фурье (ДПФ). Для выборки на N отсчетов потребуется выполнить N^2 комплексных операций, что занимает достаточно большое количество времени и вычислительных возможностей. Соответственно, быстрое преобразование Фурье (БПФ) позволяет оперативно вычислить ДПФ, которое определяется уравнением:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j(\frac{2\pi nk}{N})} . \qquad (2)$$

Разработаем ВП, вычисляющий спектр сигнала с помощью быстрого преобразования Фурье периодического и импульсного сигналов.

1. Добавьте на блок-диаграмму цикл по условию *While Loop* (*Functions* → *Structures*), на лицевую панель кнопку *Stop* (*Controls* → *Boolean*), замените ее название на «Стоп». Соедините с помощью проводника кнопку «Стоп» с терминалом условия выхода из цикла. В данном случае цикл будет останавливаться, когда кнопка «Стоп» будет нажиматься, соответственное ее значение будет «Истина» *Stop If True*.

2. Для генерации сигналов добавьте на блок-диаграмму базовый функциональный генератор (*Functions* → *Signal Processing* → *Waveform Generation* → *Basic Function Generator*). Обозначение входов и выходов узла приведены на рисунке 59.

3. Для формирования цифровых элементов управления щелкните ПКМ у входа *amplitude* функционального генератора, выберите *Create Controls*. Повторите действия для входов задания фазы, постоянной составляющей и частоты. На лицевой панели поменяйте названия на «Амплитуда, В», «Частота, Гц», «Фаза, град», «Постоянная составляющая».

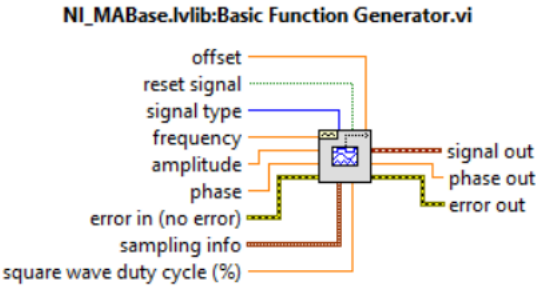


Рис. 59

Базовый функциональный генератор — спецификация входов и выходов

4. Для выбора вида анализируемого сигнала щелкните ПКМ на входе *signal type* и выберите *Create Control*, перейдите на лицевую панель и поменяйте название элемента управления на «Тип сигнала». Возможна генерация синусоидального сигнала *Sine Wave*, пилообразного сигнала *Sawtooth Wave*, прямоугольного сигнала *Square Wave* и треугольного сигнала *Triangle Wave*.

5. Добавьте на лицевую панель осциллограмму для отображения генерируемого сигнала и осциллограмму для представления спектра сигнала (*Controls* → *Graph* → *Waveform Graph*).

6. Для вычисления быстрого преобразования Фурье добавьте на блок-диаграмму узел *FFT Spectrum* (*Mag-Phase*) (*Functions* → *Signal Processing* → *Waveform Measurements*). Щелкните правой кнопкой мыши по входу *window*, выберите *Create Constant*, в появившемся элементе выберите прямоугольное окно *Rectangle*. На рисунке 60 приведена спецификация входов и выходов узла *FFT Spectrum* (*Mag-Phase*).

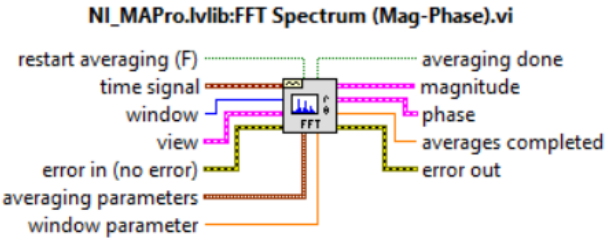


Рис. 60

Узел вычисления быстрого преобразования Фурье

7. Выполните соединения на блок-диаграмме, как показано на рисунке 61.

8. Выполните оформление лицевой панели виртуального прибора, для этого настройте графические элементы индикации — осциллограммы. Измените подписи осей на «Время, с» и «Напряжение, В» соответственно (достаточно дважды щелкнуть по тексту на оси, появится возможность его редактирования). Щелкните ПКМ по осциллограмме на лицевой панели, выберите опцию *Properties*. На вкладке *Plots* диалогового окна выберите толщину линии и синий цвет (рис. 62). На вкладке *Scales* выберите максимальное значение по оси времени 0,1 (*Maximum*). Для оси «Напряжение, В» оставьте *Autoscale* без измене-

ния. Выполните настройки сетки поля графика *Grid Style and Colors* аналогично предыдущему ВП (рис. 57). Завершите работу с диалоговым окном, нажав кнопку *Ok*. Выберите инструмент Кисть на панели *Tools*, щелкните по выбору цвета ЛКМ, установите белый цвет и щелкните ЛКМ по области графика, залив его белым цветом.

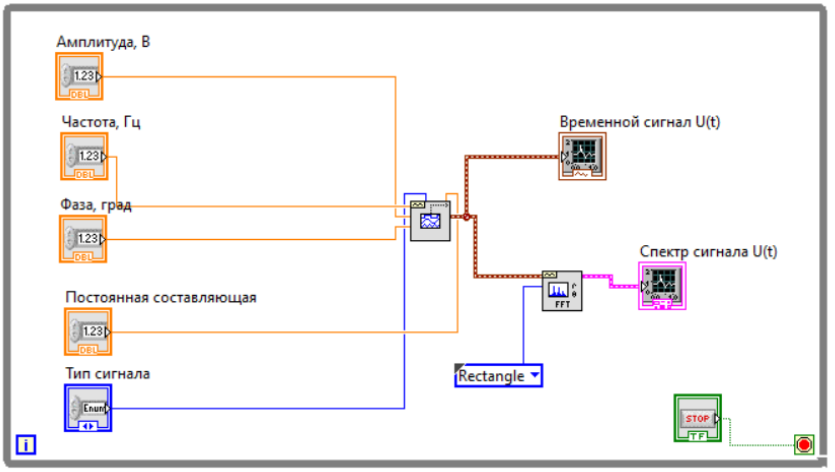


Рис. 61

Блок-диаграмма анализатора спектра

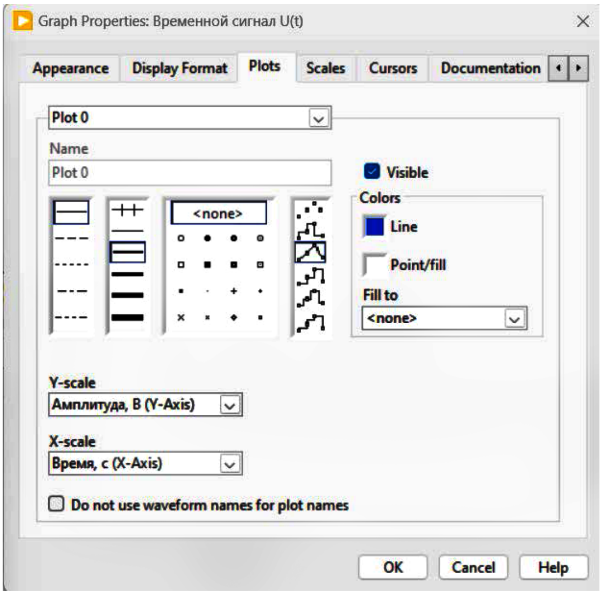


Рис. 62

Настройки отображения временного сигнала U(t)

9. Настройте график «Спектр сигнала *U(t)*» аналогичным образом, изменив лишь подпись оси *X* на «Частота, Гц», и оставьте без изменения максималь-

ное значение по данной оси *Autoscale*, т. е. максимальное значение будет определяться программным образом. Лицевая панель ВП приведена на рисунке 63.

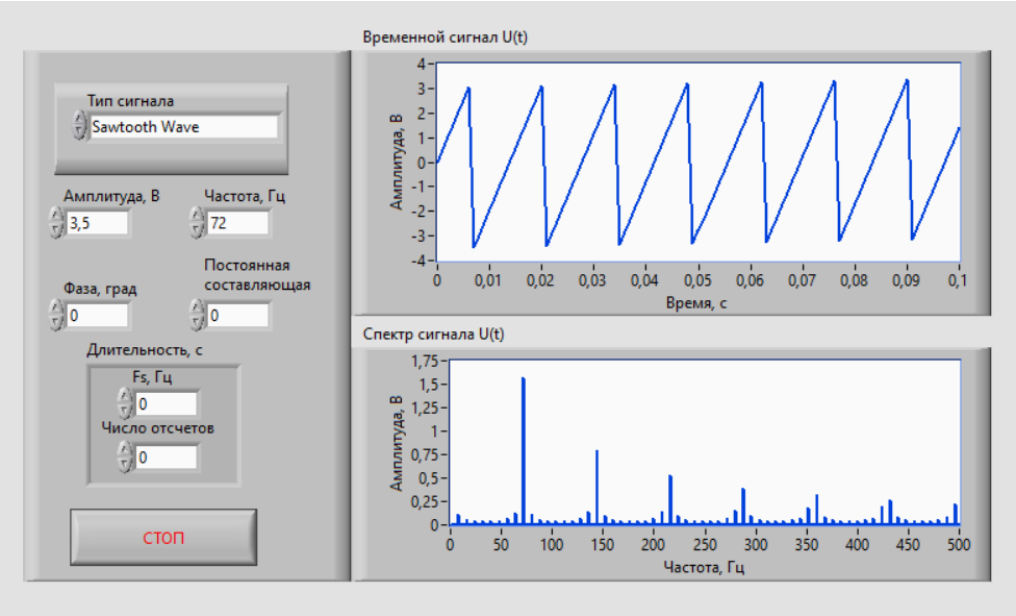


Рис. 63

Лицевая панель анализатора спектра

10. Введите значение амплитуды — 2 В, частоты — 50 Гц, фаза — 0 град, постоянная составляющая — 0. Запустите ВП, измените тип сигнала, наблюдайте за изменениями графического отображения сигнала и его спектра. Измените значения параметров управления генерацией сигнала, наблюдайте изменения в амплитудном спектре сигнала. Заметьте, что ВП будет продолжать работать при изменении параметров сигнала и его не нужно будет каждый раз перезапускать.

11. Для прекращения работы ВП нажмите на кнопку «Стоп» на лицевой панели.

12. Для задания длительности генерируемого сигнала на лицевой панели создадим кластер (*Controls* → *Data Containers* → *Cluster*). Разместим кластер на лицевой панели и добавим в него два цифровых элемента управления, так же как добавляются элементы в массив (*Controls* → *Numeric* → *Numeric Control*). Кластеры подобны структурам в текстовых языках программирования и объединяют элементы разных типов данных. В данном случае мы объединяем в кластер элементы одного типа данных. Один из числовых элементов управления будет задавать частоту дискретизации (F_s, *Гц*); второй — число отсчетов (объем выборки) (*s*). Введите названия элементов. Зная частоту дискретизации и объем выборки, можно определить длительность генерируемого сигнала по формуле

$$\Delta T = \frac{s}{F_S}. \quad (3)$$

13. Выполните соединение кластера *Cluster* со входом *sampling info* функционального генератора *Basic Function Generator*.

14. Запустите ВП, введите значения $F_s = 1000$, $s = 1000$. Измените значения частоты дискретизации и числа отсчетов сигнала, понаблюдайте за изменениями.

Задание:

1. Добавьте возможность визуализации фазового спектра сигнала в ВП. Для этого разместите элемент цифровой индикации на лицевой панели в виде графика и подключите его терминал к соответствующему выходу узла вычисления быстрого преобразования Фурье (спецификация выводов приведена на рис. 60).

2. Выполните сравнения частотного спектра каждого из типов сигнала для трех значений частоты: 15 Гц, 50 Гц, 75 Гц и амплитуды 1,5 В.

3.3. Электрические шумы. Виды шумов. Характеристики шумов

Электрическим шумом называют флуктуации тока и напряжения. Случайным процессом, шумом, флуктуациями называют такое изменение наблюдаемой величины во времени, когда по значению этой величины в текущий и в предыдущие моменты времени нельзя точно предсказать ее будущее поведение. На практике шумом считают достаточно сложное нерегулярное, хаотическое изменение. Причина возникновения шума может быть обусловлена механизмом протекания электрического тока или флуктуациями других неэлектрических величин.

До определенного времени считалось, что шумы являются нежелательными сигналами, засоряющими полезные сигналы. С течением времени было определено, что шумы несут в себе диагностическую информацию о состоянии аппаратуры или устройств.

Электрические шумы являются случайными сигналами, описание которых осуществляется на основе ансамбля реализаций. Свойства случайных сигналов оцениваются с помощью вероятностных характеристик.

С точки зрения физической природы выделяют: тепловой шум (шум Джонсона или Найквиста); дробовой шум; фликкер-шум (шум мерцания, избыточный шум, шум типа 1/*f*); генерационно-рекомбинационный шум; взрывной (телеграфный) шум.

Так как шумы описываются вероятностными характеристиками, принято их классифицировать на основе спектрального анализа на: белый/цветной шум, гауссовский шум, узкополосный/широкополосный шум.

В *LabVIEW* представлен ряд виртуальных приборов, позволяющих генерировать шумы с различными характеристиками.

Они располагаются на вкладке *Waveform Generation* (*Functions* → *Signal Processing*), основные из них приведены на рисунке 64:

1) равномерный белый шум (*Uniform White Noise Waveform*) — ВП генерирует псевдослучайный белый шум с равномерным законом распределения;

2) гауссовский белый шум (*Gaussian White Noise Waveform*) — последовательность с гауссовским (нормальным) законом распределения;

3) периодический случайный шум (*Periodic Random Noise Waveform*) — генерирует выходной массив как результат суммирования синусоид с одинаковыми амплитудами и случайными фазами;

4) пуассоновский шум (*Poisson Noise Waveform*) — ВП генерирует псевдослучайную последовательность значений, представляющих число событий ординарного пуассоновского процесса;

5) биномиальный шум (*Binomial Noise Waveform*) — последовательность с биномиальным амплитудным распределением;

6) шум 1/*f* (*Inverse f Noise Waveform*) — ВП генерирует непрерывный шумовой сигнал со спектральной плотностью мощности, которая обратно пропорциональна частоте в указанном частотном диапазоне.

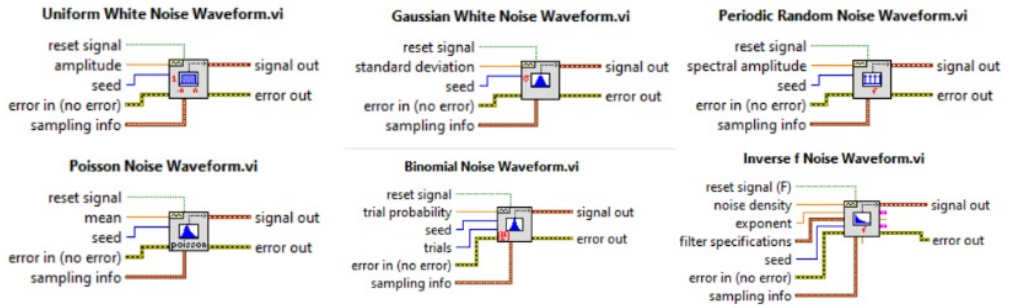


Рис. 64

Элементы генерации шумов

Основными вероятностными характеристиками сигнала, представленными случайной величиной, не изменяющихся во времени, являются: функция распределения, плотность распределения вероятности, математическое ожидание, дисперсия, среднеквадратическое значение, среднеквадратическое отклонение (СКО), мода, медиана.

3.4. Структура варианта (Case)

Операция ветвления в *LabVIEW* реализуется с помощью структуры варианта *Case Structure*. Структура может иметь минимум два или более возможных подвариантов программы, в зависимости от задаваемого условия или оператора выбора. Данная структура аналогична операторам варианта (*switch...case*) или логическим операторам (*if...else*) в текстовых языках программирования.

В любой момент времени выполняется один из вариантов в зависимости от значения (числового, логического или строкового), поданного на терминал выбора. Структура варианта *Case Structure* представлена на рисунке 65, где:

1 — отображает название ветви;

2 — тело структуры: внутри каждой вкладки содержится подпрограмма, которая будет выполняться;

3 — элемент выбора ветви.

Если элемент выбора ветви представляет собой логическую переменную, то структура будет иметь две подпрограммы: «Ложь» и «Истина». В том случае, если элемент выбора ветви — цифровая или строковая переменная, то количество подпрограмм может быть практически неограниченное.

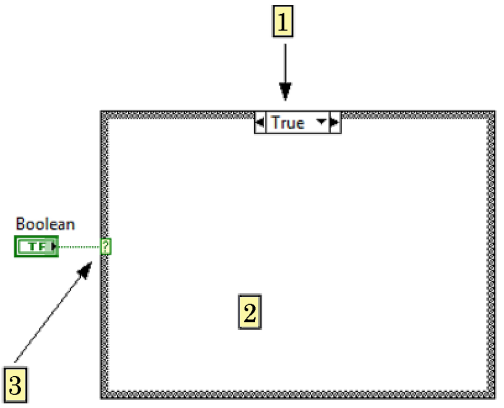


Рис. 65
Структура варианта Case Structure

При создании структура имеет две вкладки по умолчанию: Истина *True*, Ложь *False*. Добавить дополнительные вкладки можно, щелкнув ПКМ по рамке структуры. Их можно добавлять «До вкладки» *Before Case* или «После вкладки» *After Case*, также можно удалять пустые вкладки или дублировать. Виртуальный прибор будет готов к работе только после добавления подпрограмм на все вкладки. Также следует помнить, что одна из вкладок должна быть назначена по умолчанию.

3.5. Примеры виртуальных приборов: статистические характеристики шумов

Разработаем виртуальный прибор, позволяющий определять статистические характеристики электрических шумов. В приборе будет доступна возможность выбора типа шума: равномерный белый шум, гауссовский белый шум, периодический случайный шум, пуассоновский шум, шум 1/*f*.

1. Создайте новый ВП *New VI*, сохраните его под именем *Noise_statistics*.

2. Для выбора вида шума на лицевой панели разместите цифровой переключатель перечня (*Controls* → *Modern* → *Ring & Enum* → *Enum*), в названии переключателя укажите «Вид шума». Введите в поле переключателя «Равномерный белый шум». Щелкните по переключателю правой кнопкой мыши, выберите *Add Item After*, введите «Гауссов шум». Аналогичным образом добавьте еще 3 поля переключателя: «Периодический случайный шум», «Пуассоновский шум», «Шум 1/f».

3. Добавьте на лицевую панель цифровой элемент управления (*Controls* → *Modern* → *Numeric* → *Numeric Control*), введите название «Амплитуда».

4. Для задания длительности генерируемого сигнала на лицевой панели создайте кластер (*Controls* → *Modern* → *Data Containers* → *Cluster*). Добавьте в него два цифровых элемента управления, так же как добавляются элементы в массив (*Controls* → *Modern* → *Numeric* → *Numeric Control*). Один из числовых элементов управления будет задавать частоту дискретизации (F_s, Гц); второй — число отсчетов (объем выборки), *s*. Введите названия элементов: «F_s, Гц», «Число отсчетов».

5. Для визуализации генерируемого шума поместите на лицевой панели осциллограмму (*Controls* → *Modern* → *Graph* → *Waveform Graph*). Введите названия осей: «Время, с», «Амплитуда, В». Оформите график по своему усмотрению, минимальные и максимальные значения по осям оставьте автоматическими *Autoscale*.

6. Добавьте на блок-диаграмму структуру варианта (*Functions* → *Structures* → *Case Structure*). По умолчанию в структуре присутствует две вкладки (ветви), для реализации ВП нам необходимо 5 вкладок. Переименуйте вкладку *True* в «Равномерный белый шум», *False* в «Гауссов шум». Далее добавьте три дополнительные вкладки, как показано в предыдущем разделе и назовите их: «Периодический случайный шум», «Пуассоновский шум», «Шум 1/*f*».

7. Соедините с помощью проводника элемент выбора ветви (вкладки) структуры и терминал переключателя перечня. Название вкладок и названия переключателя перечня должны совпадать.

8. Для уменьшения занимаемого места терминалов на блок-диаграмме щелкните по нему ПКМ, уберите галочку на пункте *View As Icon*. Выполните данное действие для всех терминалов управления и индикации.

9. Поместите на вкладку «Равномерный белый шум» структуры узел генерации сигнала *Uniform White Noise Waveform* (*Functions* → *Signal Processing* → *Waveform Generation*).

10. Расположите элементы на блок-диаграмме и соедините их между собой согласно рисунку 66.

11. На вкладке «Гауссов шум» добавьте узел *Gaussian White Noise Waveform* (*Functions* → *Signal Processing* → *Waveform Generation*) и соедините его вход *standard deviation* с элементом управления «Амплитуда». Остальные соединения аналогичны рисунку 67.

12. На следующих вкладках добавьте одноименные генераторы сигналов шума, выполните соединения (рис. 67).

13. На вкладке «Шум 1/*f*» добавьте кластер констант (*Functions* → *Programming* → *Cluster, Class & Variant* → *Cluster Constant*) и поместите в него три константы с числовыми значениями: 0,01; 400; 5. Соедините кластер со входом *filter specifications* (рис. 68).

14. Для вычисления статистических характеристик электрических шумов добавьте на блок-диаграмму элемент статистики (*Functions* → *Mathema-*

tics → *Probability & Statistics* → *Statistics*). Данный узел относится к ВП типа *Express VI*, позволяет вычислять несколько статистических функций одновременно. Диалоговое окно выбора функций появляется автоматически при помещении узла на блок-диаграмму. Выберем статистические функции (рис. 69).

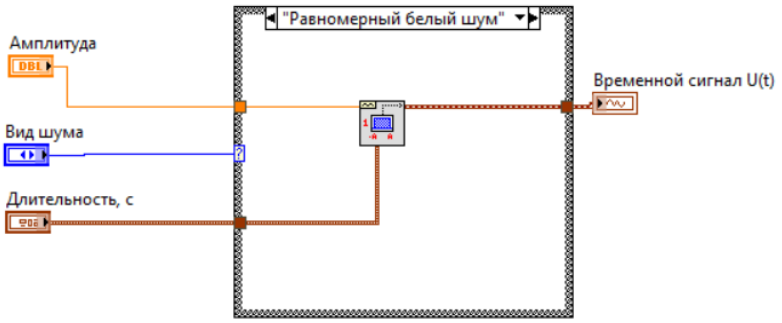


Рис. 66
Элемент блок-диаграммы ВП генерации электрических шумов: генерация равномерного белого шума

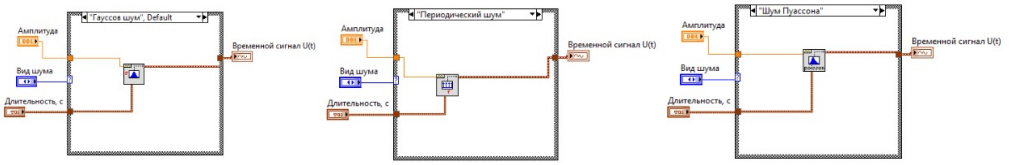


Рис. 67
Элемент блок-диаграммы ВП генерации электрических шумов

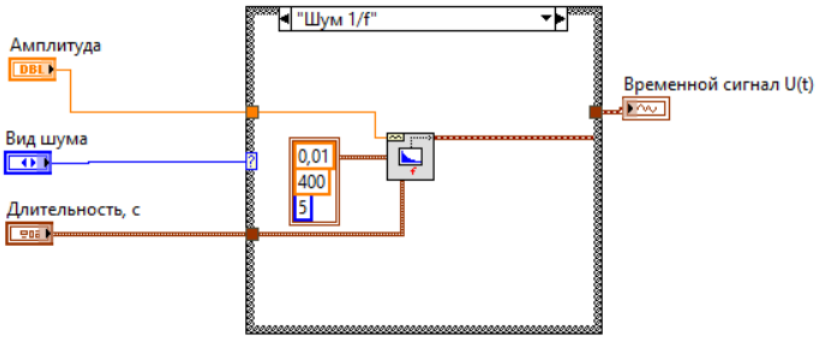


Рис. 68
Элемент блок-диаграммы ВП генерации электрических шумов: генерация шума 1/f

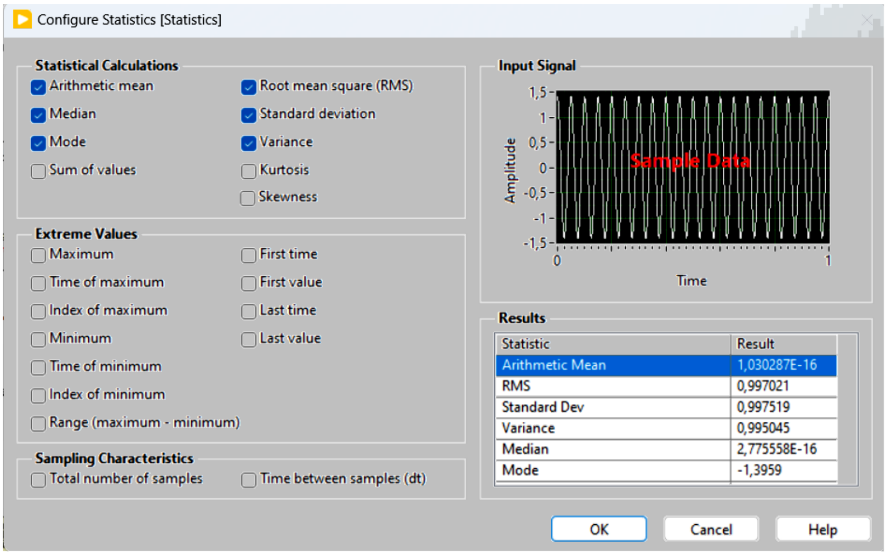


Рис. 69
Диалоговое окно элемента статистики *Statistics*

15. Разместите на лицевой панели прибора цифровые индикаторы для отображения значений: математическое ожидание, среднеквадратическое значение, СКО, дисперсия, медиана, мода.

16. Выполните соединения на блок-диаграмме (рис. 70).

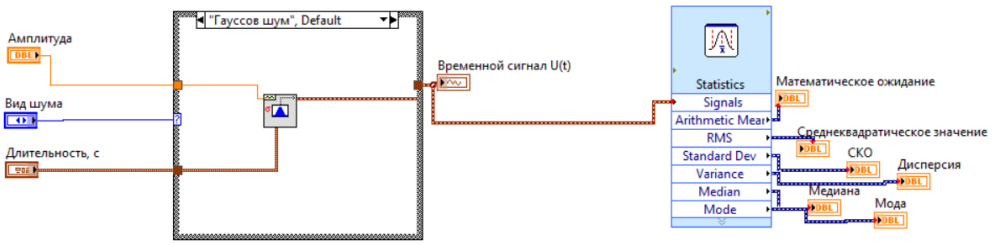


Рис. 70
Генерация электрического шума
и вычисление его статистических характеристик

17. Для определения функции распределения шумового сигнала добавьте на блок-диаграмму элемент гистограммы (*Functions* → *Mathematics* → *Prob & Stat* → *Histogram*). На вход *X* подайте временной сигнал *U*(*t*), на вход *intervals* подайте значение константы 20.

18. Для визуализации функции распределения в виде гистограммы на лицевую панель добавьте двухкоординатный график *XY Graph* (*Controls* → *Modern* → *Graph*).

19. Для подключения двухкоординатного графика понадобится выполнить сборку кластера. Поместите *Bundle* на блок-диаграмме (*Functions* → *Pro-*

gramming → *Cluster, Class & Variant* → *Bundle*), увеличьте его на два входа, потянув вниз.

20. Выполните соединения (рис. 71).

21. Оформите лицевую панель ВП в соответствии с рисунком 72.

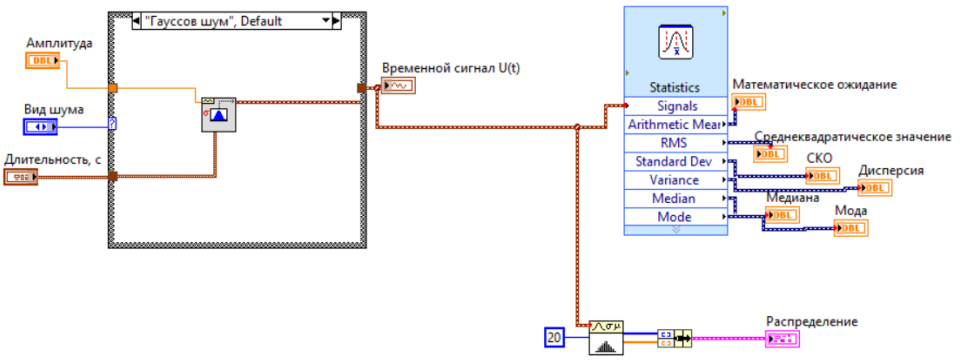


Рис. 71

Генерация шума, вычисление статистических характеристик и функции распределения

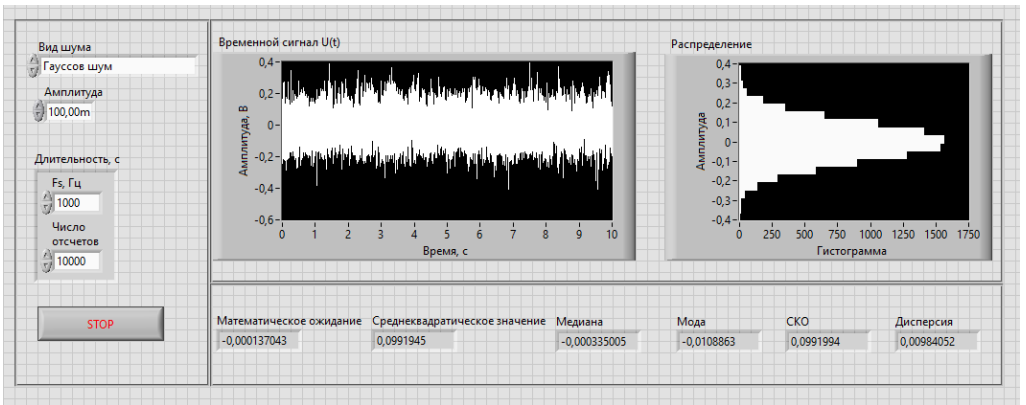


Рис. 72

Лицевая панель ВП определения статистических характеристик шумов

22. Для оформления элементов используйте вкладку *Decorations* меню *Controls*. Для размещения элементов декорирования за элементами управления или индикации выделите элемент, выберите на панели инструментов кнопку *Reorder* и один из необходимых пунктов «Поместить вперед», «Поместить назад» (рис. 73).

23. Запустите ВП, наблюдайте за изменением сигнала электрического шума, для уменьшения скорости изменения сигнала добавьте узел задержки на блок-диаграмму (*Functions* → *Programming* → *Timing* → *Wait* (*ms*)), создайте константу для регулирования времени задержки, установите значение 200 мс,

запустите ВП, наблюдайте изменения. Установите значение константы 500 мс, запустите ВП, наблюдайте изменения.

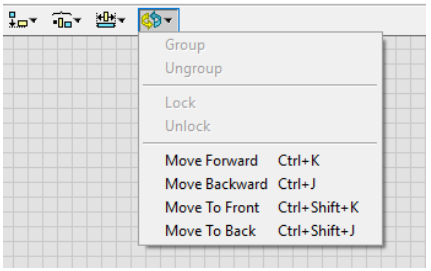


Рис. 73
Меню размещения элементов на лицевой панели

24. Для возможности изменения типа электрического шума и отсутствия необходимости постоянного перезапуска ВП добавьте структуру по условию *While Loop*. На лицевую панель поместите кнопку *Stop*, выполните необходимое соединение на блок-диаграмме (рис. 74).

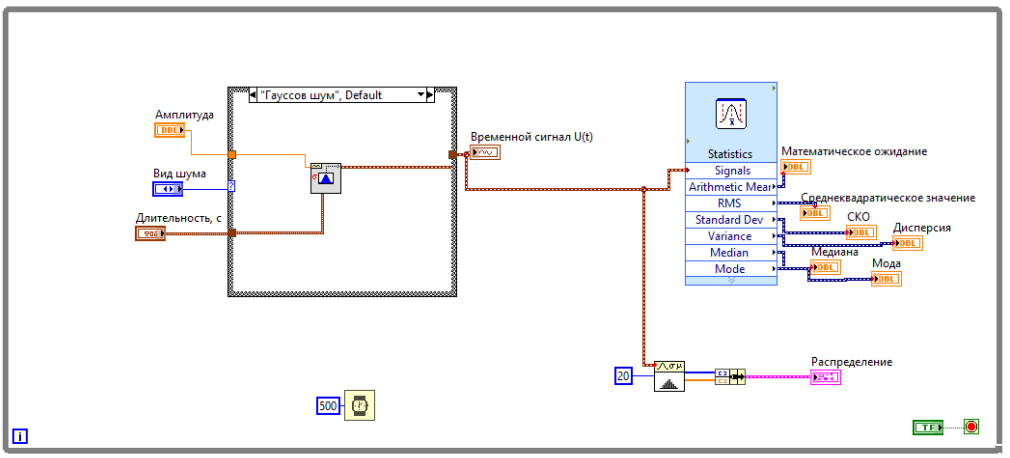


Рис. 74
Блок-диаграмма ВП определения статистических характеристик шумов

Задание:

1. Доработайте ВП, добавив в него биномиальный шум и шум Бернулли. Шум Бернулли *Bernoulli Noise* — псевдослучайный шум из единиц и нулей.

2. Добавьте в ВП периодический гармонический сигнал, выполните его суммирование с шумом, отобразите результаты на осциллограмме. Задайте возможность изменения амплитуды и частоты сигнала.

3. Важной характеристикой при обработке сигналов является отношение сигнал/шум. Отношение сигнал/шум (*SNR*) — это отношение среднеквадратического значения величины входного сигнала к среднеквадратическому значению величины шума, выраженное в децибелах:

$$SNR = 20 \cdot \log \frac{V_{signal(rms)}}{V_{noise(rms)}}. \quad (4)$$

4. Вычислите среднеквадратическое значение периодического сигнала с помощью функции *RMS* (*Functions* → *Mathematics* → *Prob & Stat*). Определите отношение сигнал/шум, выведите его значение на лицевую панель ВП.

5. Доведите отношение сигнал/шум до 2; 1; 0,5; 0,1; 0,01. Зафиксируйте в виде скринов внешний вид гармонического сигнала + электрического шума.

6. Повторите действия для другого типа шума. Зафиксируйте полученные результаты. Сделайте выводы.

3.6. Фильтрация сигналов. Цифровые фильтры

Фильтрация сигнала представляет собой процесс, при котором изменяется частотный спектр сигнала. Использование фильтров позволяет выделить нужный сигнал из смеси с другими сигналами, снизить уровень шума. При этом считается, что составляющие полезного сигнала отличаются от других составляющих в частотной области.

Фильтры, имеющие на входе и выходе аналоговые сигналы, называются аналоговыми фильтрами. В современной обработке сигналов предпочтения отдают цифровым фильтрам, так как они имеют ряд преимуществ перед аналоговыми:

- легкие в настройке и проверке, так как имеют программное обеспечение;
- требуют только арифметических действий: сложения, вычитания, умножения;
- не изменяют свои параметры при изменении условий окружающей среды;
- не требуют элементной базы высокой точности;
- не изменяют свойств в процессе эксплуатации;
- не подвержены старению компонентов;
- соотношение себестоимость/характеристики является наилучшим.

В связи с перечисленными преимуществами цифровые фильтры получили широкое применение.

Цифровые фильтры имеют несколько классификаций в зависимости от их характеристик.

В зависимости от диапазона частот, который либо пропускают, либо ослабляют, фильтры классифицируются на следующие типы:

1) низкочастотные фильтры (*Lowpass*) пропускают низкие частоты, ослабляют высокие;

2) высокочастотные фильтры (*Highpass*) пропускают высокие частоты, ослабляют низкие;

3) полосовой фильтр (*Bandpass*) пропускает определенную полосу частот;

4) заграждающий фильтр (*Bandstop*) ослабляет определенную полосу частот или вырезает ее.

Частотные характеристики фильтров приведены на рисунке 75.

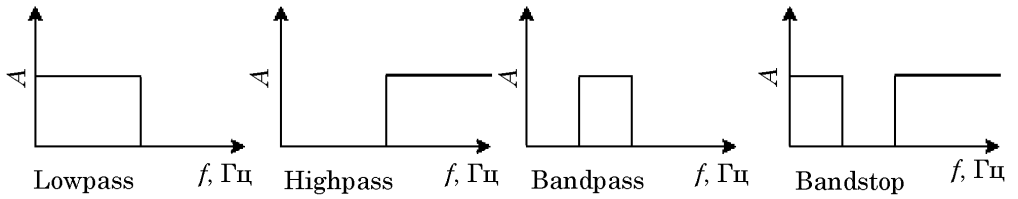


Рис. 75

Частотные характеристики фильтров

Полоса пропускания частот — диапазон частот, составляющих входного сигнала, которые проходят через фильтр без изменения амплитуд. Полоса задержания фильтра — частотный диапазон составляющих сигнала, которые не проходят через фильтр.

Фильтры характеризуются импульсной характеристикой $h(k)$, коэффициентом передачи фильтра $H(n)$.

По виду импульсной характеристики цифровые фильтры можно разделить на: фильтры с конечной импульсной характеристикой — КИХ-фильтры (*FIR-filters*) и фильтры с бесконечной импульсной характеристикой — БИХ-фильтры (*HR-filters*). Выходной сигнал КИХ-фильтра зависит только от отсчетов входного сигнала, поступившего на вход фильтра в предыдущие и рассматриваемый моменты времени. Выходной сигнал БИХ-фильтра зависит от отсчетов входного сигнала в текущий и предыдущие моменты времени и от отсчетов выходного сигнала в предыдущие моменты. В *LabVIEW* представлены как КИХ-фильтры, так и БИХ-фильтры.

3.7. КИХ-фильтры: особенности и характеристики

КИХ-фильтры также известны как нерекурсивные (трансверсальные) фильтры или фильтры свертки.

Особенности КИХ-фильтров:

- позволяют реализовать линейную ФЧХ;
- устойчивость;
- выполняют фильтрацию сигнала на основе свертки.

В *LabVIEW* представлены все четыре типа КИХ-фильтров в зависимости от диапазона пропускаемых частот. Их можно найти на вкладке *Filters* (*Functions* → *Signal Processing*). Кроме того, имеется универсальный фильтр, который позволяет выбрать полосу пропускания частот.

Создадим КИХ-фильтр и определим его характеристики: импульсную характеристику, АЧХ, ФЧХ.

1. Создайте новый ВП и добавьте на его блок-диаграмму оконный КИХ-фильтр (*Functions* → *Signal Processing* → *Filters* → *FIR Windowed Filter*). Его спецификация приведена на рисунке 76.

Коэффициенты данного фильтра определяются значениями на входах: частота дискретизации (*sampling freq:* f_s), нижняя частота среза (*low cutoff freq:* f_l), верхняя частота среза (*high cutoff freq:* f_h) и число отводов (*taps*). Значение отво-

дов определяет общее число КИХ-коэффициентов, по умолчанию оно равно 25. Для значений на входах должно выполняться следующее условие:

$$0 < f_l < f_h < 0{,}5 \cdot f_s. \quad (5)$$

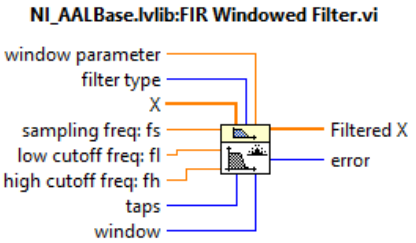


Рис. 76

Спецификация входов и выходов оконного КИХ-фильтра (FIR Windowed Filter)

2. Задайте значение частоты дискретизации постоянной и равной 1.

3. Верхнюю и нижнюю частоты среза, отводы задайте с помощью цифровых элементов управления на лицевой панели. Выполните соединения на блок-диаграмме.

4. Вход *window* определяет тип сглаживающего окна, которое уменьшает пульсации гармонических составляющих в полосе пропускания фильтра и улучшает подавление составляющих в полосе заграждения. В фильтре доступны следующие варианты: без окна, точное Блэкмана, Ханна, Блэкмана — Хэрриса, Хэмминга, Кайзера — Бесселя, треугольное, плосковершинное. Для возможности переключения типа сглаживающего окна щелкните ПКМ на соответствующем входе и выберите *Create Control*.

5. Вход *filter type* позволяет выбрать тип фильтра в зависимости от фильтруемых частот. Щелкните правой кнопкой мыши на соответствующем входе и выберите *Create Control*.

6. Разместите элементы управления на лицевой панели ВП, подпишите их.

7. Для получения импульсной характеристики фильтра подадим на его вход единичный импульс, генерируемый ВП *Impulse Pattern* (*Functions* → *Signal Processing* → *Signal Generation*). Задайте следующие значения на входе генератора: *samples* — 128, *amplitude* — 1, *delay* — 0.

8. Сигнал с выхода единичного импульса подайте на вход *X* фильтра.

9. Сигнал с выхода фильтра подайте на осциллограмму *Waveform Graph*.

10. Частотные характеристики фильтра вычисляются как БПФ от импульсной характеристики. Добавьте на блок-диаграмму узел вычисления БПФ (*Functions* → *Signal Processing* → *Transforms* → *FFT*).

11. Для визуализации АЧХ и ФЧХ разделим массив значений, полученный в предыдущем пункте, на два подмассива. Добавьте на блок-диаграмму узел *Split 1D Array* (*Functions* → *Programming* → *Array*). На его вход *array* подайте сигнал с выхода узла вычисления БПФ.

12. Значения первого подмассива подайте на узел *Complex To Polar* для перевода комплексных значений в значения в полярных координатах (*Functions* → *Mathematics* → *Numeric* → *Complex*).

13. Для построения АЧХ и ФЧХ необходимо выполнить сборку кластера с помощью узла *Bundle* (*Functions* → *Programming* → *Cluster, Class & Variant*).

14. Для получения непрерывной зависимости фазы от частоты добавьте на блок-диаграмму узел развертки фазы *Unwrap Phase*. Данный ВП удаляет разрывы, значения которых превышают π.

15. Для отображения АЧХ и ФЧХ добавьте на лицевую панель осциллограммы, выполните соединения и оформите ВП. Блок-диаграмма и пример оформления лицевой панели приведены на рисунке 77.

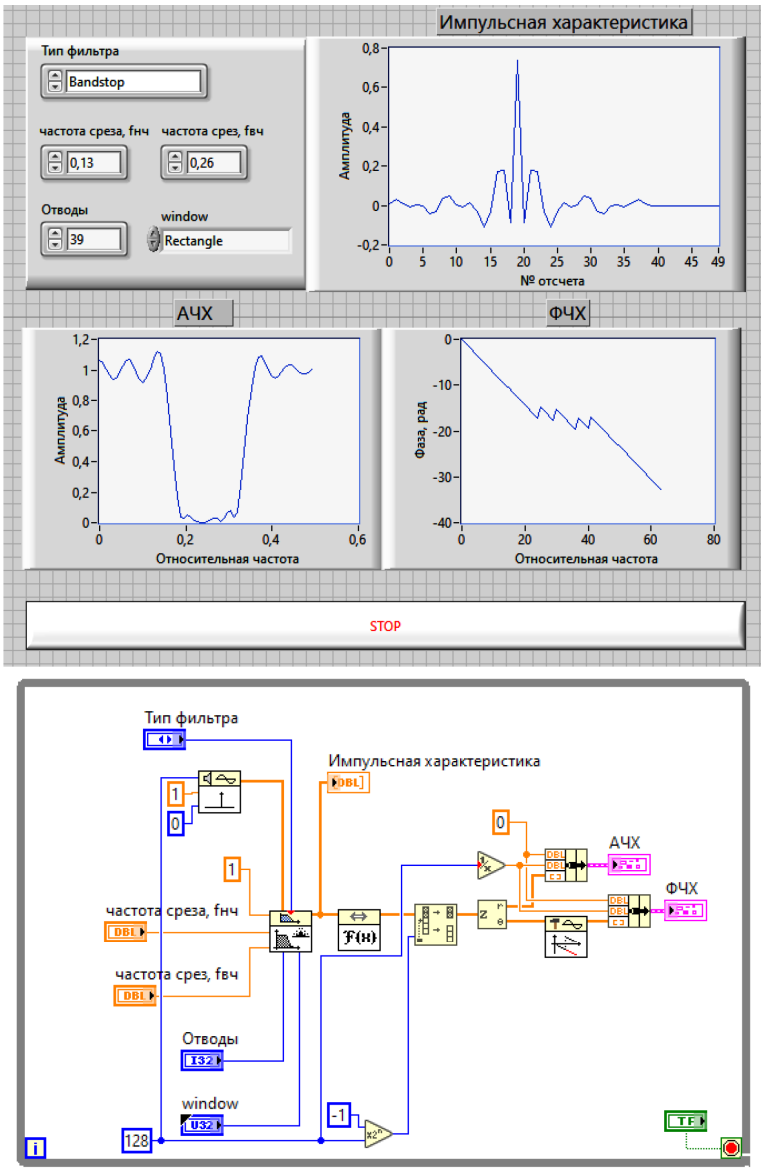


Рис. 77
Характеристики КИХ-фильтра

16. Сохраните ВП под именем: «КИХ_фильтры», он пригодится в следующем параграфе.

3.8. Пример виртуального прибора: фильтрация сигнала с помощью цифрового КИХ-фильтра

Выполним генерацию сигнала, требующего фильтрации. Для этого к периодическому синусоидальному сигналу добавим электрический шум и применим к смеси сигналов оконный КИХ-фильтр.

1. Создайте ВП.

2. Добавьте на блок-диаграмму генератор гармонического и шумового сигналов *Tones and Noise Waveform* (*Functions* → *Signal Processing* → *Waveform Generation*).

3. Создайте элементы управления гармоническим сигналом и уровнем шума. Разместите их на лицевой панели.

4. Сигнал с выхода генератора подайте на цифровой КИХ-фильтр (*Functions* → *Signal Processing* → *Waveform Condition* → *Digital FIR Filter*). Спецификация входов и выходов узла приведены на рисунке 78.

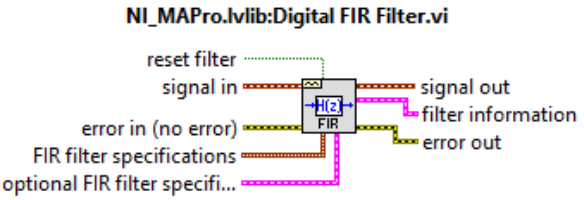


Рис. 78

Спецификация входов и выходов цифрового КИХ-фильтра

5. Создайте кластер управления параметрами фильтра. Для этого щелкните ПКМ по входу *FIR filter specifications*, выберите *Create Control*. Среди параметров для конструирования КИХ-фильтра:

- Топология фильтра: без фильтрации; КИХ-фильтр, заданный спецификацией; КИХ-фильтр с равномерными пульсациями, оконный КИХ-фильтр.
- Тип фильтра: фильтр нижних частот, фильтр верхних частот, полосовой фильтр, режекторный фильтр.
- Отводы задают число коэффициентов КИХ-фильтра.
- *f* нч1 — нижняя из двух частот полосы пропускания. По умолчанию 100 Гц.
- *f* вч1 — верхняя из двух частот полосы пропускания. По умолчанию 0.
- *f* нч2 — нижняя из двух частот полосы заграждения. По умолчанию 200 Гц.
- *f* вч2 — верхняя из двух частот полосы заграждения. По умолчанию 0.

6. С выхода *filter information* получим данные об АЧХ и ФЧХ фильтра. Для этого используем функцию разделения кластера по имени *Unbundle By*

Name. Она располагается *Functions* → *Programming* → *Cluster, Class and Variant*. Для визуализации АЧХ и ФЧХ создайте на лицевой панели две осциллограммы *Waveform Graph*. Для представления АЧХ в дБ используем специальный ВП. Скопируйте его из файла *Lin_DB.vi*, расположенного в папке ЦОС на рабочем столе.

7. Для представления на одной осциллограмме сигнала до фильтрации и после добавьте на блок-диаграмму узел *Build Array* (*Functions* → *Programming* → *Array*). Добавьте на лицевую панель осциллограмму *Waveform Graph*.

8. Выполните необходимые соединения на блок-диаграмме, настройте представление графиков на осциллограммах лицевой панели. Лицевая панель и блок-диаграмма ВП приведены на рисунках 79, 80.

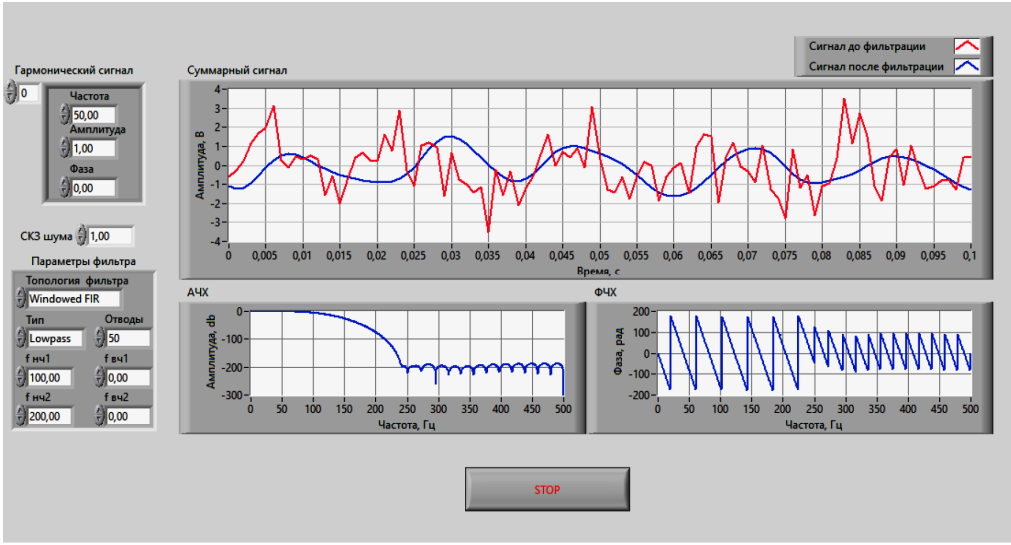


Рис. 79

Фильтрация сигнала с помощью КИХ-фильтра: лицевая панель

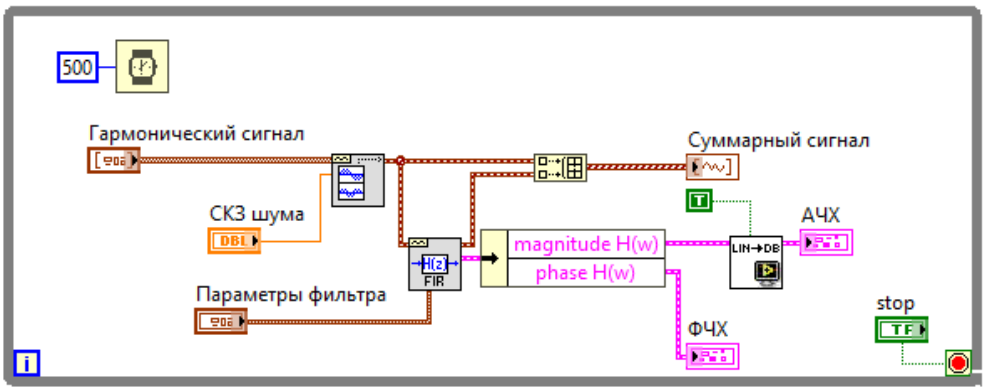


Рис. 80

Фильтрация сигнала с помощью КИХ-фильтра: блок-диаграмма

9. Добавьте на блок-диаграмму узел задержки (*Functions* → *Programming* → *Timing* → *Wait (ms)*).

10. Добавьте на лицевую панель кнопку *Stop*, выполните соединения на блок-диаграмме.

11. Введите значения элементов управления, как представлено на рисунке 79, наблюдайте полученные результаты.

12. Сохраните ВП под именем: «Фильтрация_КИХ», он пригодится в следующих параграфах.

Задание:

1. Измените среднеквадратическое значение шума до значения 3 с шагом 0,5. Наблюдайте изменения.

2. Проведите последовательно исследования фильтрации сигналов для различных топологий фильтров, типов и параметров фильтрации, параметров сигнала: частота (50 Гц) и шум (от 0,1 до 1,5 с шагом 0,5). Подберите параметры фильтрации таким образом, чтобы ВП работал и выполнял свои функции. Зафиксируйте результаты. При задании значений частот учитывайте неравенство (5).

3.9. БИХ-фильтры: типы и частотные характеристики

БИХ-фильтры или фильтры с бесконечной импульсной характеристикой, также известные как рекурсивные фильтры. Они используют в своих вычислениях предыдущие значения входных отсчетов и выходных отсчетов. Импульсная характеристика БИХ-фильтра никогда не достигает нулевого значения, в результате получается бесконечный по длительности отклик.

Среди БИХ-фильтров выделяют следующие типы:

- фильтры Баттерворта (*Butterworth Filter*);
- фильтры Чебышева (*Chebyshev Filter*);
- инверсные фильтры Чебышева (*Inverse Chebyshev Filter*);
- эллиптические фильтры или фильтры Кауэра (*Elliptic Filter*);
- фильтры Бесселя (*Bessel Filter*).

Среди достоинств БИХ-фильтров: большое быстродействие и небольшой объем памяти, который требуется. К недостаткам стоит отнести нелинейность ФЧХ.

АЧХ фильтра Баттерворта является гладкой на всех частотах и характеризуется монотонным спадом, начиная с частоты среза. На частоте среза мощность выходного сигнала уменьшается в два раза. Для фильтров Баттерворта характерна максимально плоская АЧХ в полосе пропускания частот и нулевое значение в полосе заграждения. Крутизна характеристики зависит от порядка фильтра. Повышение порядка фильтра приводит его характеристику к идеальной характеристике фильтра нижних частот.

Фильтры Чебышева имеют более крутой спад АЧХ по сравнению с фильтрами Баттерворта, который позволяет реализовать высокие скорости фильтрации и обеспечить меньшие абсолютные ошибки.

Инверсные фильтры Чебышева имеют те же преимущества перед фильтрами Баттерворта, что и фильтры Чебышева. Они подобны фильтрам Чебышева, при этом имеют перед ними ряд преимуществ: минимизируют пиковую ошибку в полосе задержания, а не в полосе пропускания; пульсации АЧХ в полосе задержания являются неизменными; АЧХ монотонно убывает в полосе пропускания, а не в полосе задержания.

Эллиптические фильтры позволяют минимизировать пиковые ошибки и в полосе пропускания, и в полосе заграждения. Пульсации АЧХ неизменны и в полосе пропускания, и в полосе заграждения. Имеют максимально крутой переход между полосой пропускания и заграждения по сравнению с фильтрами Баттерворта и Чебышева.

Максимально плоские АЧХ и ФЧХ у фильтров Бесселя. В полосе пропускания фильтров ФЧХ почти линейная.

Определение характеристик БИХ-фильтров в *LabVIEW* и их визуализация во многом совпадает с КИХ-фильтрами, отличие заключается лишь в видах фильтров.

Доработаем ВП по определению характеристик КИХ-фильтров таким образом, чтобы он позволял определять характеристики БИХ-фильтров.

1. Откройте ВП «КИХ_фильтры», выберите в меню *File* операцию *Save as* и в открывшемся диалоговом окне *Substitute copy for original* введите имя файла «БИХ_фильтры». В этом случае оригинальный файл останется без изменения.

2. На блок-диаграмме замените КИХ-фильтр на фильтр Баттерворта. Для этого щелкните ПКМ и выберите *Replace*, *Filters Palette* в открывшемся окне *Butterworth Filter*.

3. Элемент управления «Отводы» в названии замените на «Порядок». Он будет определять порядок фильтра. Установите значение по умолчанию 1 (*Data Operations → Make Current Value Default*).

4. Запустите ВП, изменяйте порядок фильтра, наблюдайте изменение АЧХ и ФЧХ фильтра.

Задание:

1. Доработайте ВП таким образом, чтобы можно было продемонстрировать на одной осциллограмме четыре АЧХ для различных порядков фильтра: 2, 3, 5, 10. Для объединения сигналов на одной осциллограмме используйте узел построения массива *Build Array*.

2. Добавьте в ВП возможность выбора фильтра: Баттерворта, Чебышева, инверсных Чебышева, эллиптического, Бесселя. Для этого используйте структуру варианта.

3.10. Пример виртуального прибора: фильтрация сигнала с помощью цифрового БИХ-фильтра

Для построения ВП, выполняющего фильтрацию с помощью БИХ-фильтра, воспользуемся ВП, полученным ранее и сохраненным под именем «Фильтрация_КИХ».

1. Откройте ВП и переименуйте его в «Фильтрация_БИХ».

2. Замените узел цифрового КИХ-фильтра *Digital FIR Filter* на цифровой БИХ-фильтр *Digital IIR Filter*.

3. Удалите кластер управления параметрами КИХ-фильтра, так как при замене фильтров появляется «оборванный провод». Создайте кластер управления параметрами БИХ-фильтра. Для этого щелкните ПКМ по входу *IIR filter specifications*. Среди параметров для конструирования БИХ-фильтра:

- Топология фильтра: без фильтрации; Баттерворта, Чебышева, инверсный Чебышева, эллиптический, Бесселя.
- Тип фильтра: фильтр нижних частот, фильтр верхних частот, полосовой фильтр, режекторный фильтр.
- Порядок фильтра.
- $F_{сн}$ — нижняя частота среза, должна соответствовать критерию Найквиста. По умолчанию 100 Гц.
- $F_{св}$ — верхняя частота среза. Данный параметр игнорируется в случае фильтра нижних или фильтра верхних частот.
- Пульсации — должен быть больше 0 дБ. Значение по умолчанию 1,0.
- Ослабление — определяет ослабление сигнала в полосе заграждения, должно быть больше 0 дБ. Значение по умолчанию 60,0.

4. Задайте следующие параметры фильтруемого сигнала: частота сигнала — 50 Гц, амплитуда — 1 В, СКЗ шума — 1,0. Параметры фильтра: фильтр Баттерворта нижних частот 3 порядка, $F_{сн}$ — 100 Гц, $F_{св}$ — 200 Гц, пульсации — 1, ослабление — 60. Запустите ВП, наблюдайте результаты работы ВП. Лицевая панель ВП приведена на рисунке 81.

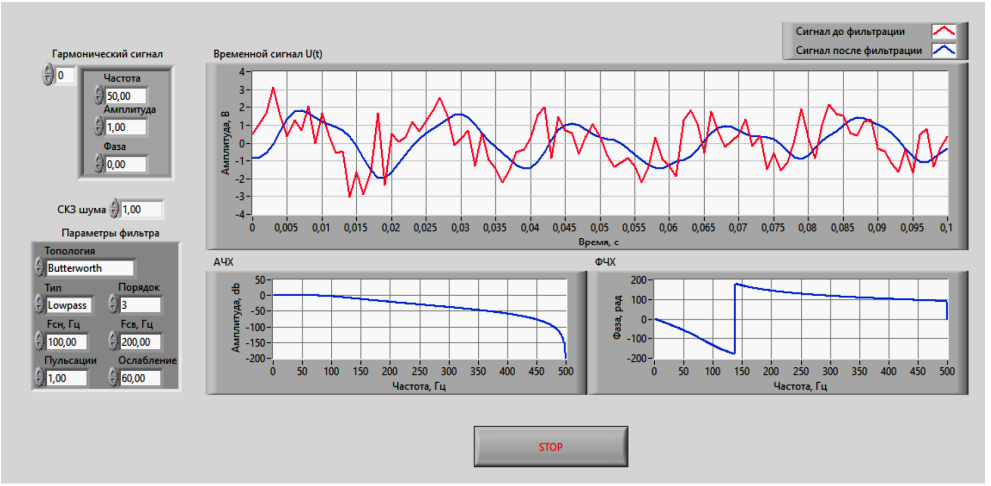


Рис. 81

Фильтрация сигнала с помощью БИХ-фильтра

Задание:

1. Проведите последовательно исследования фильтрации сигналов для различных топологий фильтров и различных порядков, типов и параметров фильтрации, параметров сигнала (частота) и шума (от 0,1 до 1,5 с шагом 0,5).

2. Для каждой из топологий фильтров рассмотрите не менее 10 фильтров различных порядков, не используйте фильтры выше 20 порядка. Проведите сравнительный анализ зависимости топологии фильтра, его порядка, полосы фильтруемых частот и качества фильтрации сигнала. Наблюдайте, чтобы сигнал после фильтрации максимально совпадал с сигналом до фильтрации. Зафиксируйте результаты в таблице:

№	Топология фильтра	Тип фильтра	Порядок	Осциллограмма временного сигнала *U(t)*

3.11. Фильтрация сигнала с помощью Express VI

В палитре функций фильтров находится ВП типа *Express VI — Filter* (*Functions → Signal Processing → Waveform Conditioning → Filter*), имеющий диалоговое окно настройки (рис. 82) и сочетающий в себе большое разнообразие фильтров.

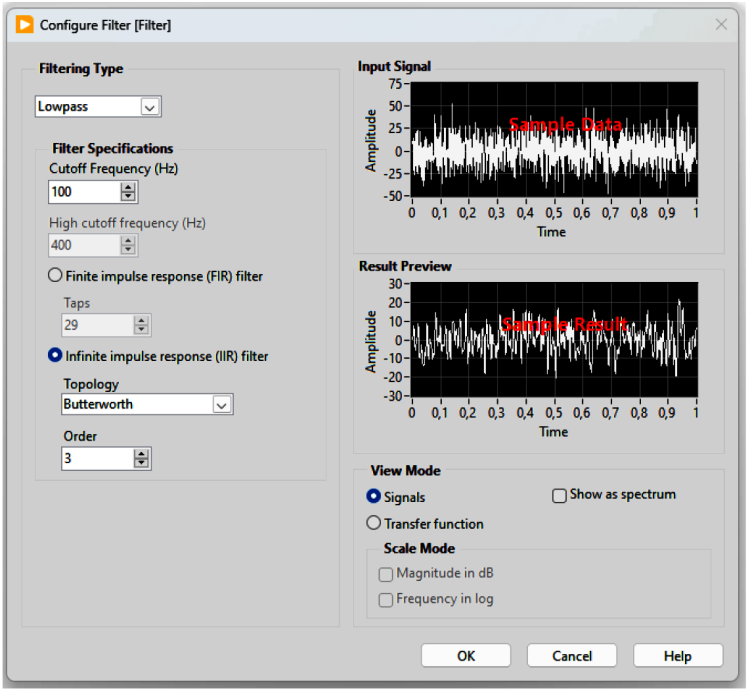


Рис. 82

Диалоговое окно настройки Express VI — Filter

Как видно из рисунка 82, фильтр позволяет выбрать полосу фильтруемых частот, тип фильтра: КИХ-фильтр или БИХ-фильтр, нижнюю и верхнюю частоты среза в зависимости от выбранного диапазона, для КИХ-фильтра число отводов, для БИХ-фильтра: топологию и порядок фильтра. В окне предварительного просмотра можно наблюдать входной и результирующий сигнал, либо частотные характеристики фильтра *Transfer function*.

Задание:

1. Создайте ВП, скопируйте на его блок-диаграмму узел формирования суммарного гармонического сигнала с шумом, осциллограммы для представления исходного и результирующего сигналов.

2. Добавьте экспресс ВП *Filter* на блок-диаграмму, в диалоговом окне выберите параметры фильтра: фильтр нижних частот, КИХ-фильтр, нижняя частота среза 150 Гц, отводов — 30.

3. Добавьте все необходимые элементы управления и индикации, запустите ВП. Зафиксируйте результаты фильтрации.

4. Щелкните дважды по узлу *Filter*, измените тип фильтра — БИХ-фильтр. Запустите ВП, наблюдайте результаты фильтрации.

5. Сделайте вывод об использовании узла *Express VI — Filter* при построении ВП.

3.12. Вейвлет-анализ

В случае стационарных сигналов, т. е. сигналов, параметры, которых не изменяются с течением времени, спектральный анализ позволяет получить исчерпывающую информацию о представлении сигнала в частотной области.

В случае нестационарных сигналов одним из эффективных методов анализа является вейвлет-преобразование. Теория вейвлетов является мощной альтернативой классическим методам анализа, так как дает более гибкую технику обработки сигнала.

Вейвлет-преобразование позволяет проводить анализ тонкой структуры сигналов, так как одинаково хорошо выделяет НЧ и ВЧ составляющие сигнала, что дает несравненно большое преимущество при анализе локальных особенностей сигнала, которое принципиально отсутствует у преобразования Фурье.

Вейвлет-анализ — это исследование сигнала при помощи вычисления коэффициентов с помощью базисных функций $\psi(t)$ (материнских вейвлетов) путем масштабирования и сдвигов:

$$\psi_{a,b} = \frac{1}{\sqrt{a}}\psi\left(\frac{t-b}{a}\right), \tag{6}$$

где a — масштабный коэффициент, отвечающий за ширину вейвлета, b — параметр сдвига, определяющий положение вейвлета на оси t. Разложение сигнала в многообразных вейвлетных базисах позволяет анализировать различный частотный диапазон сигнала.

В качестве материнских вейвлетов могут использоваться функции, удовлетворяющие ряду условий: они должны быть ограничены во времени и местоположению на оси времени, их спектр должен быть локализован на частотной оси. Для материнского вейвлета обязательно должно выполняться условие:

$$\int_{-\infty}^{\infty}\psi(t)dt = 0\,. \tag{7}$$

где $\psi(t)$ — вейвлет-функция, t — время. Таким образом, функция $\psi(t)$ должна быть равна нулю за пределами некоторого конечного интервала. Именно по-

этому в некоторой русскоязычной литературе можно встретить перевод вейвлета, как «маленькая волна».

При анализе сигнала происходит «покрытие» вейвлетом всего временного интервала исследуемого сигнала (выборки), для этого используется операция смещения материнского вейвлета $\psi(t)$ по оси времени: $\psi_b(t) = \psi(t - b)$. Для покрытия частотного диапазона применяется операция масштабирования вейвлета с непрерывным изменением переменной a: $\psi_a(t) = \psi(t/a)$. Таким образом, вейвлет перемещается по временной оси с помощью сдвига $(t - b)$. Зафиксировав материнский вейвлет на некотором значении временной оси, изменением масштабного коэффициента a можно проанализировать частотный спектр сигнала.

Исследователи называют вейвлет-анализ «математическим микроскопом», так как он позволяет получить информацию о тонкой структуре сигнала и зафиксировать те его особенности, которые получить другими методами просто невозможно.

Неоспоримые преимущества ВП позволяют находить широкое применение в науке и технике. Возможной трудностью при использовании ВП, препятствующим повсеместному внедрению преобразования в измерительной технике, является сложность интерпретации полученных данных, так как результатом преобразования является двумерное представление одномерного сигнала. Возникающая сложность может быть решена различными способами визуализации вейвлет-коэффициентов: скейлограмма, спектрограмма, временная реализация заданной частоты, пороговая обработка коэффициентов на различных масштабах.

В теории вейвлет-преобразования выделяют два основных вида: непрерывное вейвлет-преобразование (НВП) и дискретное вейвлет-преобразование (ДВП).

Непрерывное вейвлет-преобразование (НВП) осуществляется путем свертки анализируемого сигнала (функции) $f(t)$ с двухпараметрической вейвлетной функции $\psi_{a,b}(t)$ (материнским вейвлетом):

$$W(a,b) = \int_{-\infty}^{\infty} f(t) \cdot \psi^*_{a,b}(t) dt \,. \qquad (8)$$

где $\psi^*_{a,b}(t)$ — комплексно-сопряженная величина.

Непрерывное вейвлет-преобразование изображает пространство функции одной переменной в двумерное вейвлетное пространство. И соответственно, вейвлет-преобразование позволяет обнаружить изменение во времени составляющих сигнала по сдвигу и масштабу вейвлет-коэффициентов наибольшей амплитуды, т. е. позволяет проанализировать сигнал и во временной, и в частотной области, что не позволяет сделать преобразование Фурье.

НВП характеризуется избыточностью получаемой информации в связи с непрерывным изменением масштабного коэффициента и параметра сдвига, которая в данном случае часто является положительным моментом, нежели отрицательным, так как позволяет более полно проанализировать информацию, содержащуюся в данных. Кроме того, подобная избыточность может быть ликвидирована путем выбора параметров преобразования.

Таким образом, НВП осуществляется с одной материнской вейвлетной функцией. Покрытие всей временной оси за счет сдвигов вейвлета.

Непрерывный вейвлет-анализ в силу перечисленных свойств и возможностей ориентирован на выделение локальных особенностей сигнала. Основная трудность НВП связана с интерпретацией полученных результатов. Классическим методом визуализации результатов НВП является трехмерное представление вейвлет-коэффициентов, на основании которого можно получить зависимость амплитуды вейвлет-коэффициентов от времени для фиксированного масштабного коэффициента, т. е. проанализировать сигнал на конкретной частоте и определить локализацию особенностей сигнала.

В основе дискретного вейвлет-преобразования лежит разложение сигнала $f(t)$ на низкочастотную (сглаженную) и высокочастотную составляющие. Для такого разложения используются фильтры.

Возьмем сигнал $f(t)$, изменяющийся во времени, и применим к нему первый цифровой фильтр разложения h_n, который выделяет низкие частоты и фильтр g_n, который выделяет верхние частоты. На выходе фильтра h_n отсутствует верхняя половина частот, соответственно, частота дискретизации выходного сигнала уменьшается в два раза, т. е. выполняется децимация сигнала. На выходе фильтра g_n освобождается место в области низких частот. Таким образом, сигналы на выходе фильтров несут информацию о своей половине частот, при этом выходная информация представлена таким же количеством отсчетов, как и входная.

В результате применения НЧ фильтра h_n получается сглаженная составляющая A_k, ВЧ фильтра g_n — детализирующая составляющая сигнала D_k.

Массив A_k принято называть аппроксимирующими коэффициентами, в них заложена информация о трендовой составляющей сигнала, массив D_k — детализирующими коэффициентами, несущими информацию об особенностях сигнала.

Далее аналогичную процедуру фильтрации применяют к аппроксимирующим коэффициентам первого уровня A_1 для получения коэффициентов второго уровня разложения. Схематическое представление дискретного вейвлет-разложения представлено на рисунке 83.

Таким образом, при разложении, например, до пятого уровня получим детализирующие коэффициенты D_1, D_2, D_3, D_4, D_5 и аппроксимирующие коэффициенты только пятого уровня разложения — A_5.

Для уменьшения количества требуемых вычислительных операций С. Малла был предложен пирамидальный алгоритм, позволяющий раскладывать сигнал на НЧ и ВЧ компоненты с неполной двоичной выборкой.

На практике работают с дискретными значениями исследуемого сигнала $f(t)$, поэтому принимают имеющиеся значения сигнала в качестве вейвлет-коэффициентов нулевого уровня $a_{0,k}$ и дальнейшего вычисления коэффициентов, при этом на каждом уровне разложения одновременно выполняются операции фильтрации и децимации. Операция децимации необходима для сохранения количества вейвлет-коэффициентов, аналогичного нулевому уровню, так как при выполнении фильтрации происходит увеличение числа коэффициентов

в два раза. Данную разновидность вейвлет-преобразования называют «быстрым». На рисунке 84 представлена схематическая реализация пирамидального алгоритма Малла и разложения сигнала с помощью дискретного вейвлет-преобразования.

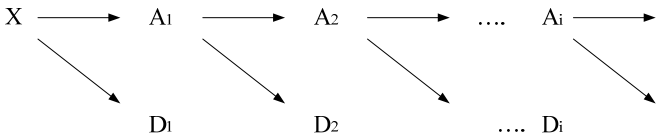


Рис. 83
Графическое представление ДВП

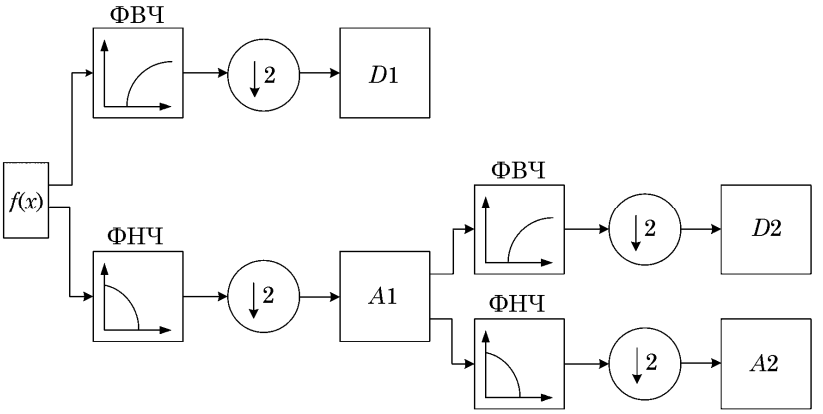


Рис. 84
Пирамидальный алгоритм Малла

ДВП применяется очень часто для выделения полезного сигнала из смеси сигнал + шум.

Важной особенностью вейвлет-преобразования является выбор материнского вейвлета. Каждый из материнских вейвлетов обладает определенными свойствами и характеристиками и подходит для анализа одного сигнала, при этом совершенно не принесет пользы при анализе другого сигнала. Использование того или иного типа вейвлета зависит, как правило, от опыта исследователя. Для применения материнских вейвлетов существуют лишь рекомендации, полученные из опыта применения вейвлетов исследователями разных стран. Классификация вейвлетов приведена в Приложении 4.

3.13. Пример виртуального прибора: анализ сигнала с помощью непрерывного вейвлет-преобразования

Разработаем ВП, реализующий непрерывное вейвлет-преобразование нестационарного сигнала. Для отображения результатов НВП будем использовать скейлограмму — график распределения интенсивности вейвлет-коэффициентов

в зависимости от параметра сдвига *b* и масштаба *a*. Параметр сдвига *b* является аналогом времени *t*. Масштабный коэффициент *a* имеет связь с частотой *f* и зависит от центральной частоты материнского вейвлета.

Для вейвлет-преобразования в *LabVIEW* имеется специальный тулкит *Advanced Signal Processing*, который требует дополнительной установки.

1. Создайте ВП, сохраните его под именем «Непрерывное ВП».

2. Разместите на блок-диаграмме узел, выполняющий функции генерации нестационарных сигналов *Data Samples* (*Functions* → *Signal Processing* → *Wavelet Analysis* → *Utilities*). Щелкните ПКМ и выберите *Create Control*, введите его название «Тип сигнала». В списке содержится 23 возможных сигнала. Он раскрывается при нажатии на элемент управления на лицевой панели ЛКМ.

3. Добавьте на лицевую панель осциллограмму (*Waveform Graph*) и соедините ее вход с выходом узла *Data Samples*. Настройте параметры графика.

4. Разместите на блок-диаграмме узел, вычисляющий непрерывное вейвлет-преобразование *Continuous Wavelet Transform* (*Functions* → *Signal Processing* → *Wavelet Analysis* → *Continuous Wavelet*). Спецификация входов и выходов узла приведена на рисунке 85.

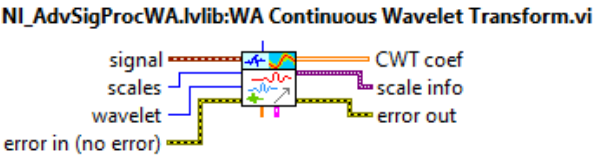


Рис. 85

Спецификация входов и выходов узла вычисления непрерывного вейвлет-преобразования

5. Создайте элемент управления параметром масштаба (*scale*) и материнским вейвлетом (*wavelet*). Для этого щелкните ПКМ на соответствующем входе узла и выберите *Create Control*.

6. Для визуализации результатов НВП воспользуемся скейлограммой (график интенсивности). Для ее настройки добавьте на блок-диаграмму узел настройки скейлограммы *Configure Scalogram Indicator* (*Functions* → *Signal Processing* → *Wavelet Analysis* → *Utilities*). Спецификация узла приведена на рисунке 86.

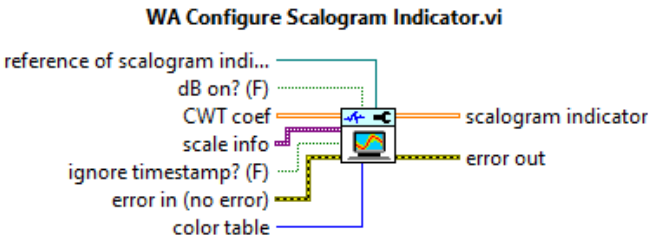


Рис. 86

Спецификация узла настройки скейлограммы

7. На вход узла настройки скейлограммы подайте сигналы с выхода узла вычисления вейвлет-коэффициентов.

8. Добавьте на лицевую панель график интенсивности *Intensity Graph* (*Controls* → *Modern* → *Graph*), введите его название «Скейлограмма», по осям введите «Сдвиг, *b*», «Масштаб, *a*». В параметрах графика по оси *X* добавьте умножение на 0,001.

9. Добавьте на блок-диаграмму узел *VI Server Reference* (*Functions* → *Programming* → *Application Control*), который позволяет изменять цветовую гамму графика *Intensity Graph* для работы совместно с элементом *Configure Scalogram Indicator*. Соедините данный узел со входом *reference of scalogram indicator*.

10. Добавьте на лицевую панель кнопку *Stop*, на блок-диаграмму структуру *While Loop*.

11. Значение масштабного коэффициента по умолчанию устанавливается 64, для представленных сигналов оно может быть меньше, больше выставлять не нужно, так как все «особенности» сигналов локализованы в области малых значений масштабного коэффициента.

12. Добавьте на лицевую панель график поверхности, который будет представлять вейвлет-коэффициенты в трехмерном представлении *3D Surface Graph* (*Controls* → *Modern* → *Graph* → *3D Graph*). Щелкните ПКМ по входу *color matrix* и создайте элемент управления *Create Control*. Данный элемент будет отвечать за цветовое представление трехмерного графика вейвлет-коэффициентов.

13. Для сравнения возможностей вейвлет-преобразования и спектрального анализа добавьте экспресс ВП *Spectral Measurements* (*Functions* → *Signal Processing* → *Waveform Measurements*). В диалоговом окне настройки экспресс ВП выберите тип амплитудного спектра *Magnitude* (*RMS*) и линейное представление спектра *Linear*.

14. Лицевая панель и блок-диаграмма ВП приведены на рисунке 87.

15. Запустите ВП, получите изображение скейлограммы, соотнесите анализируемый сигнал со скейлограммой вейвлет-коэффициентов. Какие особенности сигнала представлены на скейлограмме?

16. Для интерпретации величины вейвлет-коэффициентов изменяйте значение максимума масштабного коэффициента на скейлограмме, максимальному значению вейвлет-коэффициентов соответствует красный цвет области скейлограммы, минимальному — фиолетовый.

Задание:

1. Для каждого представленного в перечне типа сигналов постройте скейлограмму минимум для трех материнских вейвлетов.

2. Подумайте над интерпретацией вейвлет-коэффициентов и особенностями сигналов, зафиксируйте их в таблице:

№	Тип сигнала	Тип вейвлета	Интерпретация коэффициентов НВП

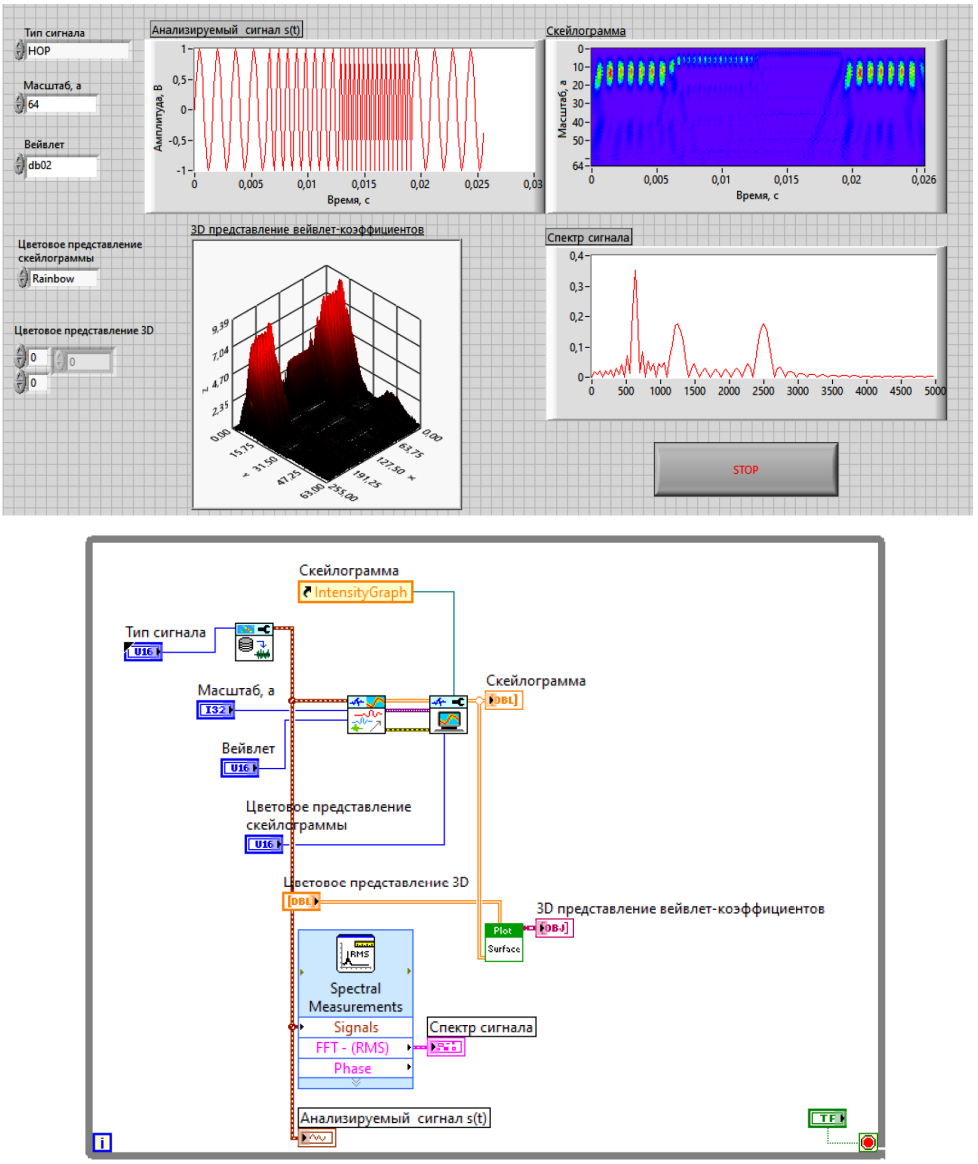


Рис. 87

Анализ сигнала методом непрерывного вейвлет-преобразования

3.14. Пример виртуального прибора: выделение полезного сигнала из зашумленного сигнала с помощью дискретного вейвлет-преобразования

Типовой метод выделения полезного сигнала из шума сводится к фильтрации зашумленного сигнала. В процессе фильтрации происходит удаление высокочастотных составляющих сигнала. Вейвлет-анализ в данном случае имеет

большие возможности, так как информацию о высокочастотных составляющих сигнала несут детализирующие коэффициенты. Таким образом, удаляя детализирующие коэффициенты, либо срезая некоторые их значения по заданному порогу, мы выделим полезный сигнал. Пороговые ограничения могут устанавливаться на разных уровнях разложения, в зависимости от характеристик шума и сигнала. Это позволяет создавать гибкие системы выделения полезного сигнала из шума, в зависимости от их особенностей.

При вейвлет-анализе, как отмечалось выше, сигнал раскладывается на аппроксимирующие коэффициенты A_k, которые представляют сглаженный сигнал, и детализирующие коэффициенты D_k, описывающие колебания. Следовательно, шумовая компонента больше отражается в детализирующих коэффициентах. Поэтому при удалении шума обрабатываются обычно детализирующие коэффициенты. Кроме этого, амплитуда шума, как правило, меньше полезного сигнала. Поэтому простейший способ удаления шума состоит в том, чтобы сделать нулевыми значения коэффициентов, большие некоторого порогового значения. Эта процедура называется пороговой обработкой коэффициентов (трешолдингом — от англ. *threshold*).

Таким образом, процедура выделения сигнала из шума состоит из трех шагов:

1) декомпозиция — производится разложение сигнала до уровня N;

2) пороговая обработка детализирующих коэффициентов: выбирается тип пороговой обработки, значения порогов;

3) реконструкция: проводится обратное вейвлет-преобразование на основе измененных детализирующих коэффициентов и первоначальных аппроксимирующих коэффициентов.

Говоря о методах пороговой обработки, стоит отметить, что широкое распространение получили такие методы, как жесткий трешолдинг (*hard threshold*) и мягкий трешолдинг (*soft threshold*).

При жесткой пороговой обработке (*hard threshold*) сохраняются неизменными все коэффициенты, большие или равные по абсолютной величине порога τ, а меньшие коэффициенты обращаются в нуль. При мягкой пороговой обработке (*soft threshold*) наряду с обращением в нуль коэффициентов, по модулю меньших, чем τ, происходит уменьшение по модулю остальных коэффициентов на величину τ. В случае обработки двумерного сигнала $f(x, y)$ (изображения) применяется жесткий трешолдинг, для одномерных сигналов $f(t)$ — мягкий трешолдинг.

От выбора порогового уровня фона (оценки дисперсии шума) зависит качество шумоподавления сигнала, оцениваемое в виде отношения сигнал/шум. Задание малых значений порога сохраняет фон в коэффициентах детализации и поэтому приводит лишь к незначительному увеличению отношения сигнал/шум. При больших значениях порога можно потерять коэффициенты, которые несут существенную информацию. Поиск оптимального значения τ_0 означает отыскание такого порога, который при наименьшем смещении восстановленного сигнала обеспечивает наибольшее отношении сигнал/шум.

Качество шумоподавления сигнала (отношение сигнал/шум) зависит от способа применения трешолдинга. Используются следующие способы пороговой обработки:

1) общий трешолдинг, осуществляемый с использованием фиксированного значения порога τ, значения, единого для всех уровней и коэффициентов детализации сигнала;

2) многоуровневый трешолдинг, осуществляемый с использованием порога τ, значения которого изменяются от уровня к уровню;

3) локальный трешолдинг, подразумевающий использование порога τ, переменного не только по уровню разложения, но также зависящего от позиции коэффициента детализации на данном уровне.

Разработаем ВП, осуществляющий разложение сигнала, представляющего сумму полезного сигнала и помехи, на аппроксимирующие и детализирующие коэффициенты, пороговую обработку коэффициентов и восстановление сигнала по аппроксимирующим коэффициентам *N* уровня и детализирующим коэффициентам с 1-го по *N* уровни.

1. Создайте ВП. Поместите на лицевой панели две осциллограммы *Waveform Graph*: одна осциллограмма будет отображать «Анализируемый сигнал *s(t)*», вторая — «Выделенный из шума сигнал *f(t)*».

2. На блок-диаграмме разместите генератор синусоидального сигнала *Sync Pattern* (*Functions* → *Signal Processing* → *Signal Generation*). Задайте постоянными значениями на входах *delay* — 0 и *delta t* — 0,1. Создайте элементы управления «Амплитуда», «Число отсчетов». Таким образом будет сформирован полезный сигнал.

3. Для формирования шума добавьте на блок-диаграмму структуру варианта (*Case Structure*). Создайте 3 вкладки структуры: «Гауссовский белый», «Равномерный белый шум», «Периодический случайный». Создайте переключатель «Тип шума» (*Controls* → *Modern* → *Ring & Enum* → *Enum*) и соедините его с выбором варианта структуры. Добавьте на соответствующие вкладки генераторы шума: *Gaussian White Noise*, *Uniform White Noise*, *Periodic Random Noise* (*Functions* → *Signal Processing* → *Signal Generation*). За пределами структуры создайте константу со значением 1, подключите ко входу *seed* генераторов. Также за пределами структуры создайте цифровой элемент управления «Амплитуда», соедините с соответствующими входами генераторов шума. У элемента *Gaussian White Noise* соедините со входом *standard deviation*. Вход *samples* генераторов соедините с элементом управления «Число отсчетов», созданным в п. 1.

4. На входы элемента суммы *Add* (*Functions* → *Numeric*) подайте полезный сигнал и шум, результат выведите на осциллограмму *Waveform Graph*.

5. Выделение полезного сигнала из зашумленного будем реализовывать с помощью функции *WA Denoise* (*Functions* → *Signal Processing* → *Wavelet Analysis* → *Feature Extraction*), в которой включено разложение сигнала на детализирующие и аппроксимирующие коэффициенты, пороговая обработка детализирующих коэффициентов и обратное вейвлет-преобразование. Спецификация элемента приведена на рисунке 88.

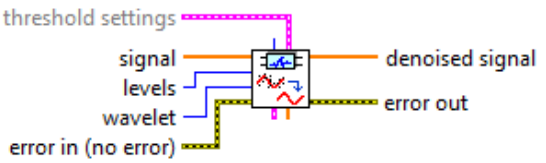


Рис. 88

Спецификация выводов элемента WA Denoise

6. Создайте элементы управления уровнем разложения (*levels*), материнским вейвлетом (*wavelet*), параметрами пороговой обработки (*threshold settings*). На вход *signal* подайте зашумленный сигнал, с выхода *denoised signal* на осциллограмму *Waveform Graph*. Элемент *WA Denoise* может работать в нескольких режимах, необходимо выбрать *DWT (Real Array)*.

7. Среди параметров обработки сигналов:

Переключатель «Пороговая обработка»:

• *SURE* — адаптивный выбор порога, используя принцип Штейна несмещенной оценки риска;

• *Hybrid* — эвристический вариант первого выбора, т. е. оптимальный вариант между адаптивным методом и универсальным. В случае низкого уровня помехи использует универсальный метод;

• *Universal* — порог sqrt(2log (length (*X*));

• *Minimaxi* — минимаксный порог.

Переключатель «Перемасштабирование»:

Если шум вне пределов [0, 1] или не белый, то порог должен быть перемасштабирован с использованием оценки уровня шума. Возможны следующие варианты:

• *One* — без перемасштабирования, когда используется базовый метод;

• *Single level* — для перемасштабирования порога с использованием оценки уровня шума на базе коэффициентов первого уровня;

• *Multiple level* — для перемасштабирования порога с использованием оценки уровня шума, зависящий от уровня разложения.

Переключатель жесткая/мягкая пороговая обработка: по умолчанию выбирается мягкая пороговая обработка, при нажатии переключателя переходит в режим жесткой пороговой обработки вейвлет-коэффициентов.

8. Добавим возможность просмотра полезного сигнала. Для этого добавьте на лицевую панель элемент управления вкладками *Tab control* (*Controls* → *Modern* → *Layout*). Разместите элемент управления вкладками на лицевой панели поверх ранее созданных осциллограмм. По умолчанию создаются две вкладки, нам понадобится только одна для данного прибора. Щелкните ПКМ по верхней части элемента, выберите *Remove Page*.

9. На блок-диаграмму добавьте структуру варианта *Case Structure*, подключите к элементу выбора ветви терминал элемента управления вкладками (*Controls* → *Modern* → *Ring & Enum* → *Enum*). Далее все элементы разместите внутри структуры варианта.

10. Настроить появление вкладки с полезным сигналом по щелчку кнопки поможет узел свойств *Property Node.* Щелкните ПКМ по терминалу элемента управления вкладками, выберите *Create → Property Node → Visible*.

11. Добавьте на лицевую панель элемент управления всплывающим окном *Rocker* (*Controls → Modern → Boolean*), укажите его название «Показать полезный сигнал». Соедините терминал элемента *Rocker* с узлом свойств элемента управления вкладками *Property Node*. Добавьте на вкладку осциллограмму *Waveform graph* с названием «Полезный сигнал», подайте на вход терминала сигнал с выхода синусоидального генератора.

12. Поместите все элементы блок-диаграммы внутрь цикла по условию *While Loop*, добавьте на лицевую панель кнопку *Stop*.

13. Для обработки ошибок при выполнении ДВП добавим на блок-диаграмму простой обработчик ошибок *Simple Error Handler* (*Functions → Programming → Dialog & User Interface*).

14. С выхода элемента вычисления ДВП *error out* подайте сигнал на элемент *Merge Error* (*Functions → Programming → Dialog & User Interface*) и затем на вход *error in* простого обработчика ошибок.

15. Для остановки прибора в случае возникновения ошибки добавьте логический элемент ИЛИ *Or* (*Functions → Programming → Boolean*).

16. Выполните соединения, как показано на блок-диаграмме (рис. 89).

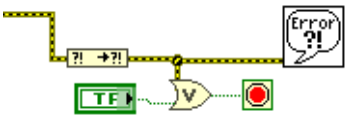


Рис. 89
Блок отработки ошибок

17. Лицевая панель и блок-диаграмма ВП приведены на рисунке 90. Оформите лицевую панель с помощью вкладки (*Controls → Modern → Decorations*).

18. Выставите значения параметров сигнала, шума и обработки, как показано на рисунке 90, запустите ВП, выполните нажатие на кнопку «Показать полезный сигнал», сравните полезный сигнал и выделенный из шума сигнал. Нажмите кнопку еще раз, остановите ВП.

Задание:

1. Последовательно с помощью переключателя «Уровень шума» установить следующие значения: для белого шума с равномерным распределением — 0,1, белого гауссовского шума — 0,1; для периодического случайного шума — 0,05. Изменяя тип вейвлета, число уровней разложения и параметры пороговой обработки, получите сигнал, наиболее приближенный к полезному.

2. Полученные данные и графики зафиксируйте в таблице:

№	Тип шума	Уровень шума	Параметры дискретного ВП: уровень разложения, вейвлет	Параметры пороговой обработки	Выделенный сигнал

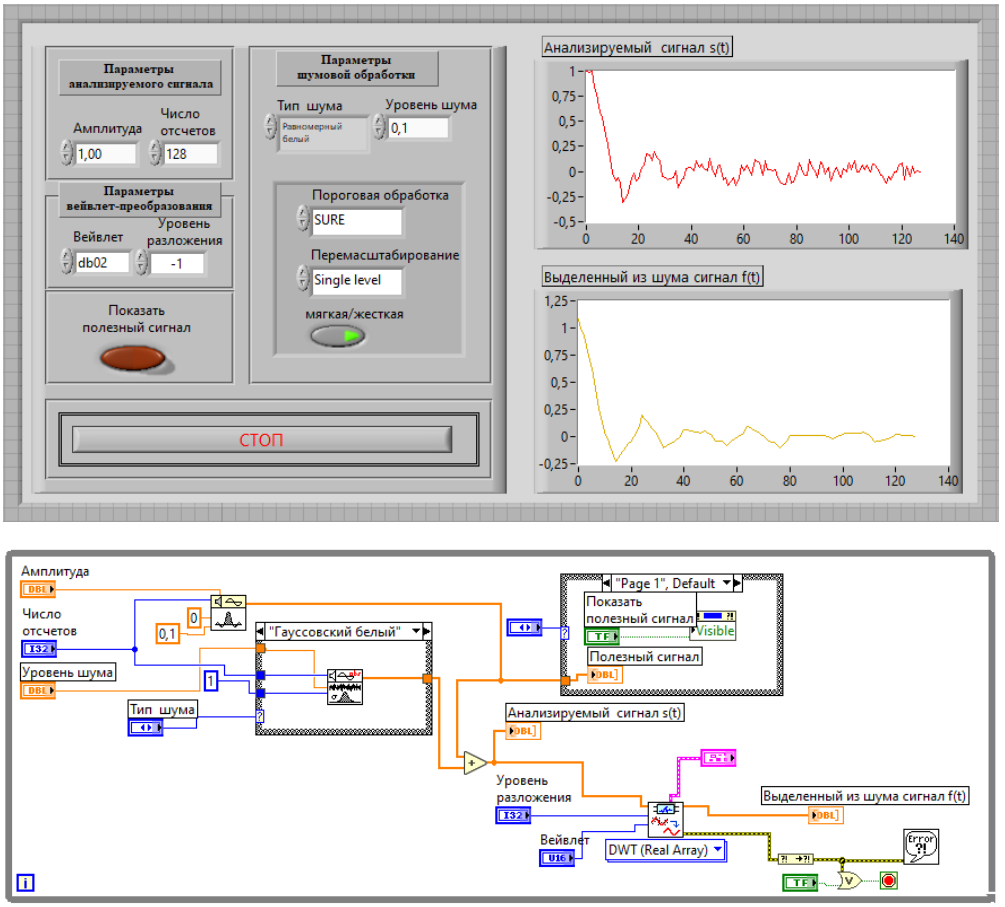


Рис. 90

ВП выделения полезного сигнала из шума

3.15. Звуковые сигналы. Виртуальный прибор считывания звукового сигнала из файла и его спектральный анализ

Звуковые сигналы — это представление звука, обычно использующее либо изменяющийся уровень электрического напряжения для аналоговых сигналов, либо серию двоичных чисел для цифровых сигналов.

В *LabVIEW* доступны функции записи, воспроизведения и обработки звуковых (аудио) сигналов. Функции располагаются на панели *Functions* → *Programming* → *Graphics and Sound* → *Sound*.

Для анализа и обработки звуковых сигналов используются методы, рассмотренные выше.

Разработаем виртуальный прибор, считывающий данные из файла, содержащий звуковой фрагмент, представляющий его в виде осциллограммы и выполняющий его спектральный анализ.

1. Создайте ВП и поместите на блок-диаграмму две осциллограммы для визуализации звукового сигнала в виде графического представления *Waveform Graph*.

2. Для считывания звукового файла добавьте на лицевую панель элемент управления Путь к файлу *File Path Control*, который *располагается Controls → Modern → String & Path*. Откройте файл *Program Files → National Instruments → LabVIEW 2021 → examples → Graphics and Sound → Sound → support → sample.wav*.

3. Для определения параметров звукового сигнала добавьте на блок-диаграмму элемент информации *Info* (*Functions → Programming → Graphics and Sound → Sound → Sound File Info*). С выхода *sound format* элемента создайте элемент индикации, назовите его «Параметры звукового сигнала». Параметры содержат информацию о частоте дискретизации (*sample rate S/s*), количестве каналов (*number of channels*), формате представления данных (*bits per sample*). Данные параметры не поддаются возможности переименования.

4. Функцию считывания файла из памяти осуществляет *Sound File Read Simple* (*Functions → Programming → Graphics and Sound → Sound → File*), разместите элемент на блок-диаграмме, спецификация его входов и выходов приведена на рисунке 91.

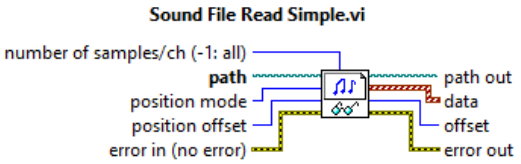


Рис. 91
Спецификация входов и выходов элемента считывания звуковых данных из файла

5. Соедините выход элемента пути к файлу со входом *path* элемента считывания. На выходе *offset* создайте элемент индикации для отображения количества отсчетов в сигнале.

6. Файл звукового сигнала содержит информацию о двух сигналах (запись в файл велась по двум каналам), для их разделения добавим на блок-диаграмму элемент *Index Array* (*Functions → Programming → Array*). Константа 0 будет определять массив данных первого, константа 1 — массив данных второго канала. Соответственно, каждый из подмассивов отдельно подается на осциллограмму. Соединения показаны на рисунке 92.

7. Выполним спектральный анализ звукового сигнала. Для этого добавьте на блок-диаграмму экспресс ВП вычисления спектра сигнала *Spectral Measurements* (*Functions → Signal Processing → Waveform Measurements*). В диалоговом окне Настройки экспресс ВП выберите Тип амплитудного спектра *Magnitude (RMS)* и Линейное представление спектра *Linear*.

8. Запустите ВП. Лицевая панель и блок-диаграмма ВП приведены на рисунке 93.

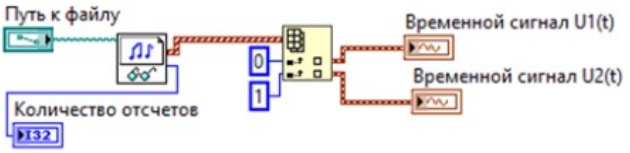
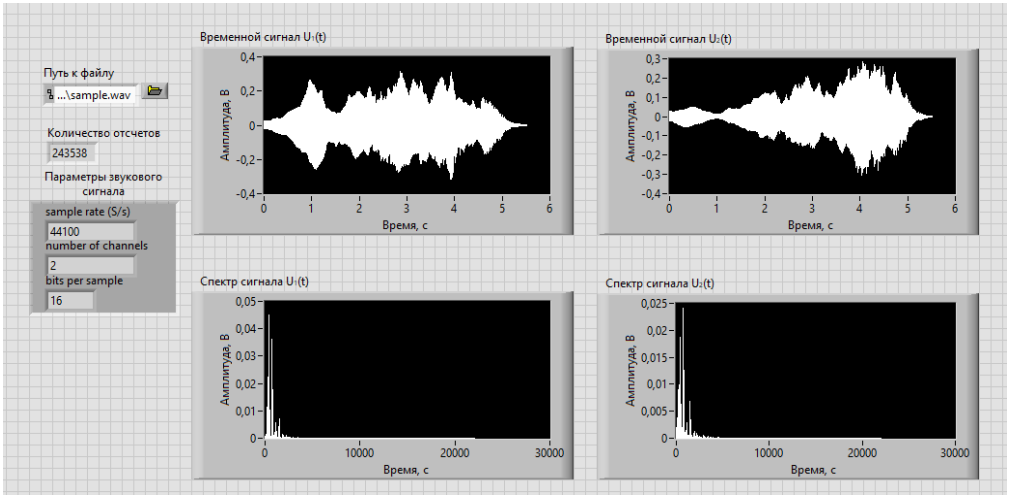


Рис. 92

Блок-диаграмма разделения каналов звукового сигнала



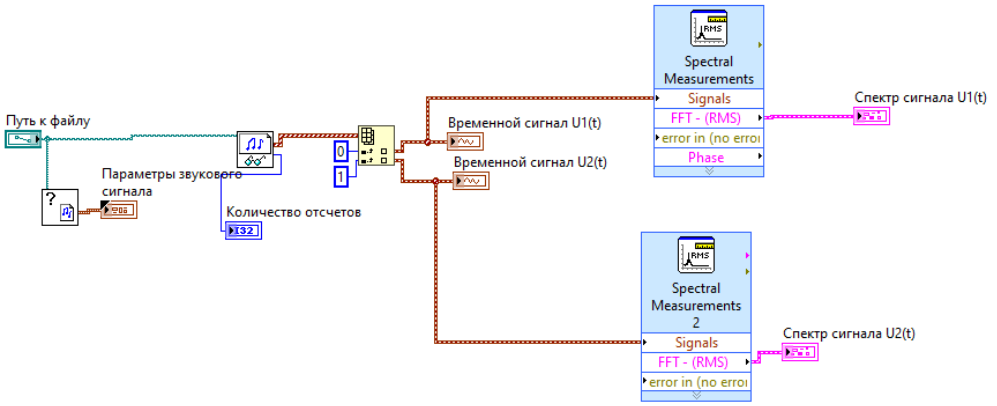


Рис. 93

ВП считывания звукового сигнала и проведения его спектрального анализа

Задание:

1. Доработайте ВП таким образом, чтобы воспроизводился звуковой файл (проигрывался).

2. Изменяя максимальное значение по оси времени, проанализируйте частотный состав сигналов с первого и второго каналов. Сделайте выводы о частотных компонентах и их амплитуде.

3.16. Вибродиагностика

Вибродиагностика — метод неразрушающего контроля, основанный на измерении и анализе параметров вибрации вращающихся механизмов. Вибрация относится к второстепенным сигналам, при этом несет в себе много информации об оборудовании и позволяет строить на основе анализа данного сигнала системы диагностики и мониторинга.

Вибродиагностический сигнал представляет собой совокупность гармонической, импульсной и случайной составляющих. Традиционным методом анализа вибрации является спектральный анализ.

Разработаем ВП, выполняющий спектральный анализ реального сигнала вибрации гидрогенератора, входящего в состав гидроагрегата Нижнекамской гидроэлектростанции. Вибрация измерялась у стальных конструкций и лобовых частей обмоток статора гидрогенератора.

Одним из обязательных этапов оценки технического состояния гидроагрегатов на российских ГЭС является контроль вибрационного состояния его конструктивных узлов, проводящийся в соответствии с нормативными документами. Вибрационные испытания агрегата, согласно технической документации, обязательно проводятся до и после каждого капитального ремонта. Испытания, проводимые до ремонта, позволяют определить отклонения от нормального режима работы оборудования, выявить скрытые дефекты, не обнаруженные в период эксплуатации, и уточнить объем работ по ремонту. Испытания после ремонта определяют качество выполненных работ. По результатам проведения вибрационных испытаний должно быть составлено заключение с оценкой технического состояния узла.

При анализе вибрации гидрогенератора необходимо определить значение спектральной составляющей частоты 100 Гц, называемой высокочастотной вибрацией. Низкочастотная составляющая вибрации представляет собой сумму пяти составляющих спектра 1,4 Гц, 2 Гц, 3 Гц, 4 Гц, 5 Гц, первая из них определяет частоту вращения вала, остальные кратны ей.

3.17. Пример виртуального прибора: спектральный анализ вибрационного сигнала гидрогенератора

Для разработки ВП будем использовать чтение данных из файла и элементы спектрального анализа, рассмотренные выше. Для исследования будут применяться данные виброскорости стальных конструкций гидрогенератора, снятые до и после ремонта. Данные представляют собой колебания виброскорости за 20 оборотов вращения гидротурбины при работе под нагрузкой 20 МВт.

1. Создайте ВП, поместите на лицевую панель два элемента управления выбора пути размещения файлов *File Path* (*Controls* → *Sting & Path* → *File Path Control*), хранящихся в памяти компьютера. Поместите две осциллограммы

Waveform Graph, на которых будут отображаться значения виброскорости до и после ремонта и две осциллограммы *Waveform Graph*, на которых будут представляться данные спектрального анализа сигналов.

2. Предусмотрим возможность анализа не всего сигнала, а только его части. Это бывает необходимо в случае считывания файлов большого объема и длительности. Для этого добавьте на лицевую панель два цифровых элемента управления «Начальный отсчет» и «Длина выборки».

3. Для считывания данных из файла добавьте два элемента чтения *Read Delimited Spreadsheet*, спецификация приведена на рисунке 94.

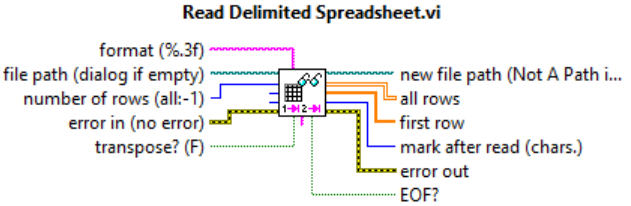


Рис. 94

Спецификация элемента считывания данных из файла

4. Сигнал с выхода элемента управления выбора пути размещения файлов *File Path* подайте на вход *file path* (*dialog if empty*) элемента считывания данных из файла, сигнал с выхода *first row* подайте на элемент разделения массива *Array Subset* (*Functions* → *Programming* → *Array*) для обеспечения возможности выбора части данных для анализа и отображения.

5. На вход *index 0* элемента разделения массива *Array Subset* подключите элемент управления «Начальный отсчет», на вход *length* — элемент «Длина выборки». С выхода *subarray* подайте сигнал на осциллограмму «До ремонта».

6. Для выполнения спектрального анализа на блок-диаграмму добавьте экспресс ВП *Spectral Measurements* (*Functions* → *Signal Processing* → *Waveform Measurements*). В диалоговом окне выберите Представление амплитудного спектра *Magnitude (RMS)* в линейном масштабе *Linear*. Сигнал с выхода амплитудного спектра *FFT-(RMS)* экспресс ВП подайте на осциллограмму.

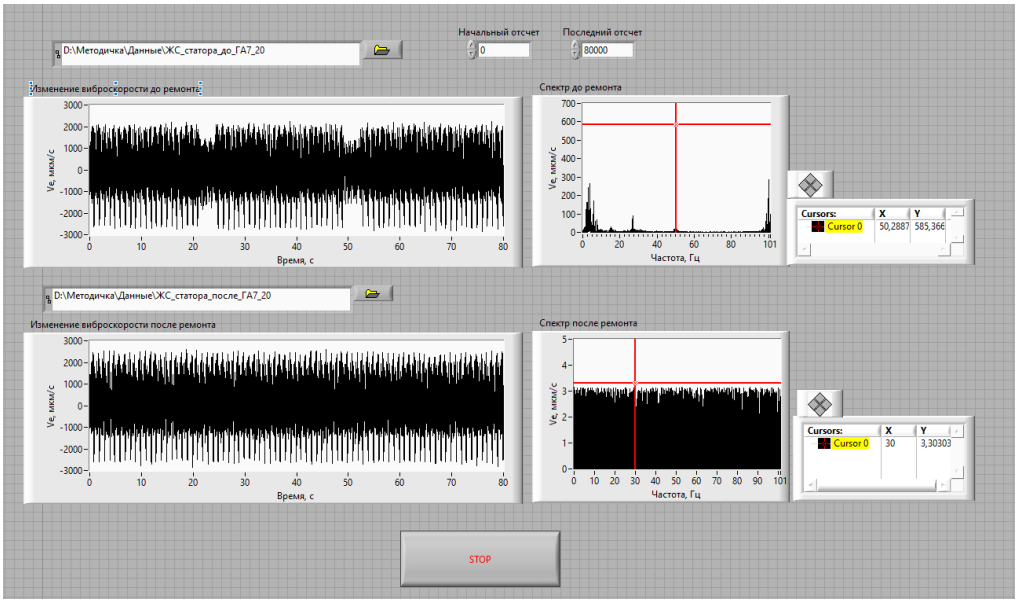
7. Выполните аналогичные действия для проведения спектрального анализа сигнала виброскорости, полученного после ремонта. Лицевая панель и блок-диаграмма ВП приведены на рисунке 95.

8. Нажмите на значок папки справа у элемента управления к пути файла и выберите необходимые файлы, они располагаются в папке «ЦОС» на рабочем столе (ЖС_статора_до_ГА7_20, ЖС_статора_после_ГА7_20).

9. Добавьте для осциллограмм спектра виброскорости возможность измерения амплитуды частотной составляющей по графику, для этого щелкните ПКМ по графику и выберите *Visible Item* → *Cursor Legend*. В появившемся поле курсора *Cursor* щелкните ПКМ и выберите *Create Cursor* → *Free*. Поменяйте на красный цвет, выполните перемещение курсора по области осциллограммы.

10. Сигнал измерялся с частотой дискретизации 1000 Гц, поэтому для представления по оси *X* временной оси откройте свойства *Properties* осцилло-

грамм до и после ремонта, на вкладке *Scales*, в поле *Scaling Factors* → *Multiplier* введите значение 0,001. На осциллограммах спектров в аналогичных полях введите значение 1000.



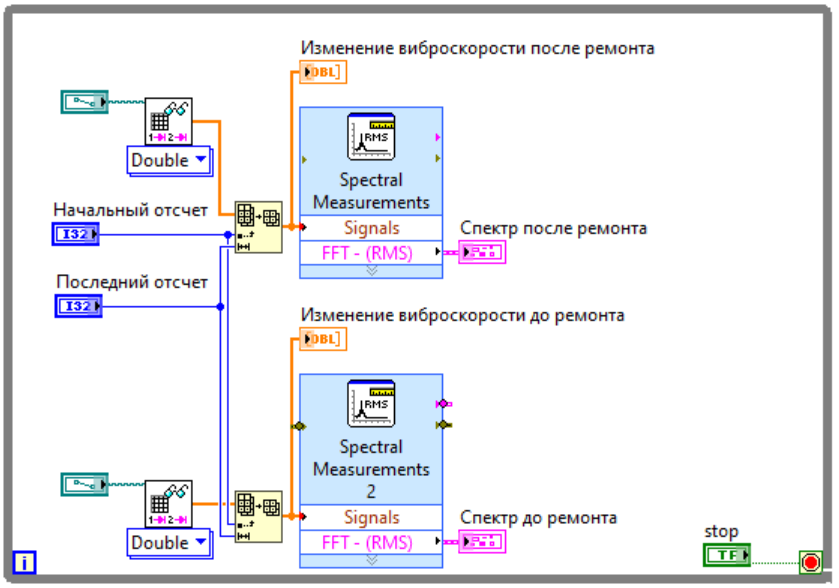


Рис. 95

Спектральный анализ вибродиагностического сигнала

11. Запустите ВП, наблюдайте результаты чтения данных из файла и полученный спектральный анализ.

Задание:

1. Вычислите уровень низкочастотной вибрации и уровень высокочастотной вибрации до и после ремонта гидрогенератора. Уровень высокочастотной вибрации определяется составляющей вибрации 100 Гц, уровень НЧ вибрации определяется как сумма составляющих спектра 1,4 Гц, 2 Гц, 3 Гц, 4 Гц, 5 Гц. На графиках приведено изменение виброскорости в зависимости от частоты, величина вибрации определяется значением виброперемещения. Доработайте ВП таким образом, чтобы можно было выполнить перевод числового значения виброскорости в виброперемещение. Преобразование осуществляется по формуле:

$$S = \frac{V}{2 \cdot \pi \cdot f}.$$

2. С помощью курсора определите значение виброскорости, выполните перевод в виброперемещение и занесите данные в таблицу:

Режим до ремонта	**Частота, Гц, 100**	**Значение виброскорости, мкм/с**	**Значение виброперемещения, мкм**
	1,4		
	2		
	3		
	4		
	5		

ГЛАВА 4
ПЛАТФОРМА NI ELVIS III

4.1. Назначение платформы NI ELVIS III, архитектура аппаратных средств

NI ELVIS (*Education Laboratory Virtual Instrumentation*) — учебная лабораторная измерительная станция, объединяющая в себе типовые лабораторные измерительные приборы, гибкий аналоговый и цифровой ввод и вывод, а также высокопроизводительный встраиваемый контроллер.

NI ELVIS имеет открытую программную архитектуру, которая позволяет выполнять широкий спектр экспериментальных исследований, от изучения основных принципов с помощью готовых приложений и базового программирования *API* до возможности перехода к низкоуровневому программированию встраиваемого процессора и *FPGA*. В состав платформы входят съемные платы, позволяющие создавать устройства с использованием широкого поля возможностей.

NI ELVIS III обеспечивает функциональность и удобное непосредственное подключение к прикладной плате с помощью разъемов с высокой плотностью контактов для управления вводом и выводом, стандартных промышленных разъемов для входов и выходов измерительных приборов и стандартных коммуникационных портов для управления измерительной станцией. Внешний вид *NI ELVIS III* приведен на рисунке 96.

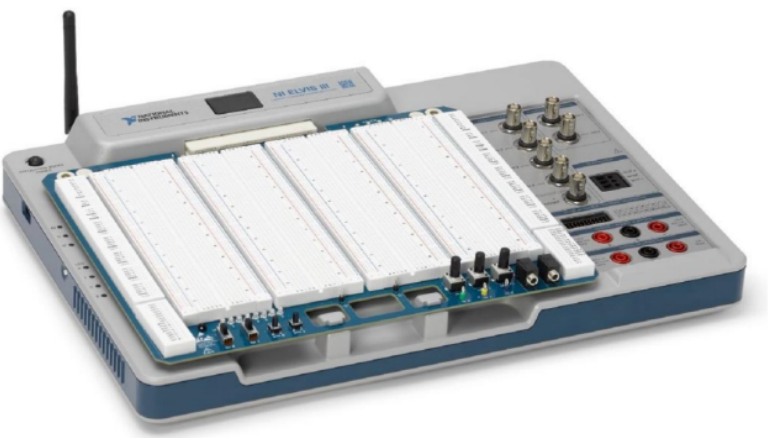


Рис. 96
Учебная лабораторная измерительная станция NI ELVIS III

Текущая версия лабораторной станции управляется с помощью компьютера через *USB*, *Ethernet*, *Wi-Fi*.

В состав платформы входит полный набор лабораторных измерительных приборов:

• осциллограф (*Oscilloscope*);

• генератор сигналов стандартной и произвольной формы (*Function Generator*);

• цифровой мультиметр (*Digital Multimeter*);

• регулируемые источники питания (*VPS*);

• логический анализатор и генератор цифровых последовательностей (*Logic Analyzer*);

• анализатор вольтамперных характеристик (*IV Analyzer*);

• анализатор амплитудно-частотных характеристик (анализатор Боде — *Bode Analyzer*).

Гибкий ввод-вывод для управления и измерения:

• аналоговый ввод (*Analog Input*);

• аналоговый вывод (*Analog Output*);

• цифровой ввод/вывод (*Digital Input/output*);

• нерегулируемые источники питания (*Power Supply*).

Кроме того, в состав входят модульные прикладные платы для различных областей применения. Детализированный вид *NI ELVIS III* с обозначением лабораторных измерительных приборов приведен на рисунке 97.

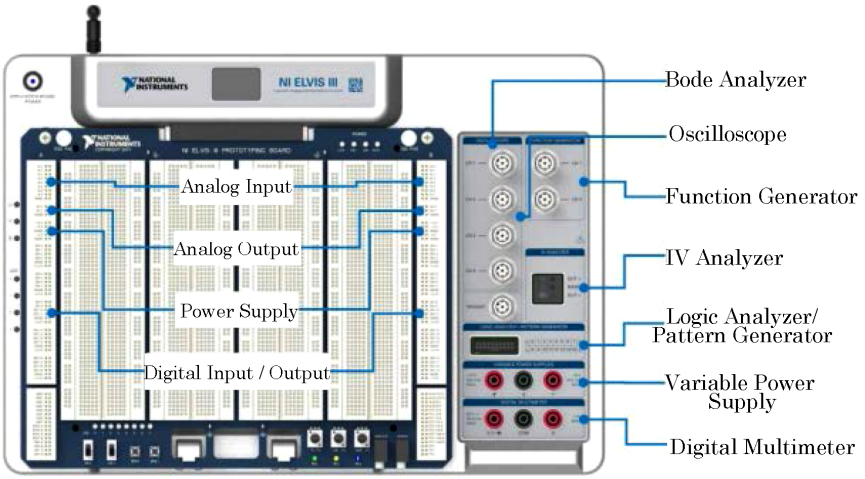


Рис. 97

Детализированный вид учебной лабораторной измерительной станции NI ELVIS III

Архитектура аппаратных средств *NI ELVIS III* (рис. 98) основана на *FPGA* (*Field-programmable gate array*) и *RT* (*Real-Time*) процессоре, которые программно доступны пользователю. Пользователь может управлять различными аналоговыми и цифровыми линиями ввода/вывода из прикладной платы, и вводом/выводом измерительных приборов через специальные разъемы.

Для быстрого доступа к измерительным приборам платформы используются программные лицевые панели (*SFP*), каждая из которых представляет собой законченный прибор. Они обеспечивают доступ к приборам без программирования.

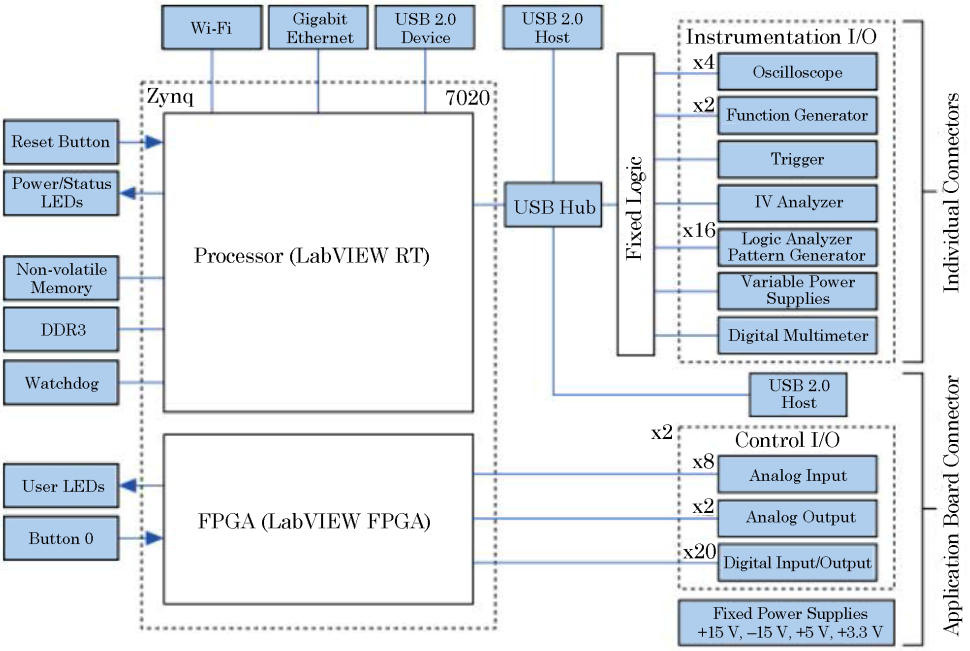


Рис. 98

Архитектура аппаратных средств NI ELVIS III

Для создания приложения на *NI ELVIS III* используется специальное дополнение к *LabVIEW* — тулкит *LabVIEW ELVIS III*. В случае низкоуровневого доступа к управлению вводом/выводом можно использовать тулкит *LabVIEW FPGA*.

NI ELVIS III основан на архитектуре, которая объединяет систему *LabVIEW Real-Time* (*RT*), программируемую пользователем *FPGA,* и программируемый пользователем ввод-вывод.

Система *LabVIEW Real-Time* включает в себя программные и аппаратные компоненты. Программные компоненты — это *LabVIEW* с расширениями для работы с *NI ELVIS III* и созданные вами проекты и виртуальные приборы. Соответственно, аппаратные компоненты — хост-компьютер, *NI ELVIS III* и прикладные платы.

Кроме того, доступно использовать индивидуальные прошивки *FPGA* для программирования *NI ELVIS III*. Тулкит *LabVIEW ELVIS III* предоставляет прошивку *FPGA* по умолчанию.

В состав пакета программ *NI ELVIS III Software Bundle* входит основное программное обеспечение, необходимое для разработки встраиваемых приложений для *NI ELVIS III*, в том числе *LabVIEW*, модуль *LabVIEW Real-Time*, тулкит *LabVIEW ELVIS III* и *Multisim*.

Доступ к программным лицевым панелям (*SFP*) осуществляется из приложения *MeasurementsLive* (рис. 99).

Вкладка *Manage device connection* — для управления подключением устройства.

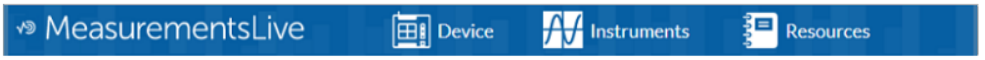


Рис. 99

Панель инструментов приложения MeasurementsLive

Вкладка *Instruments* — для запуска программных лицевых панелей (*SFP*).

Вкладка *Resources* — доступ к общим ресурсам, поддерживающим работу с *NI ELVIS III*: документам, формам, инструкциям пользователя.

4.2. Генерация сигнала в NI ELVIS III

Выполним генерацию сигнала с помощью встроенного генератора сигналов (*Function Generator*) и получим его отображение на осциллографе (*Oscilloscope*).

1. Выполните подключение платформы к источнику электропитания, убедитесь, что выключатель питания на задней панели *NI ELVIS III* находится в выключенном положении *OFF*.

2. Подключите платформу *NI ELVIS III* к компьютеру посредством *USB*-кабеля.

3. Переведите выключатель питания на задней панели *NI ELVIS III* во включенное положение *ON*.

4. Запустите программу MeasurementsLive, диалоговое окно при открытии программы изображено на рисунке 100.

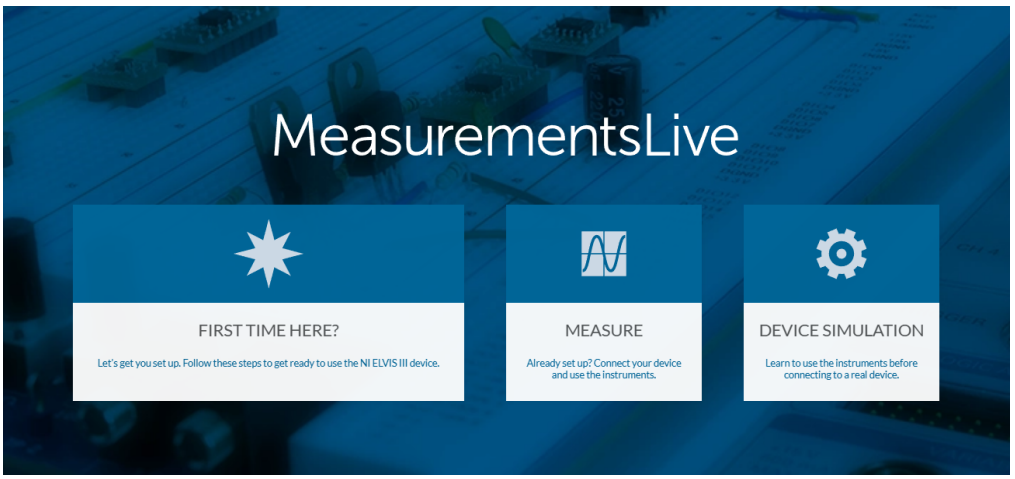


Рис. 100

Диалоговое окно программы MeasurementsLive

5. Для работы с платформой *NI ELVIS III* выберите функцию измерений *Measure*.

6. Автоматически на панели *Manage device connection* появится окно для соединения *NI ELVIS III* с компьютером. Убедитесь, что *USB*-кабель соединен, нажмите *Connect* (рис. 101). Появится всплывающее окно с сообщением о со-

единении *Connected*, которое сразу можно закрыть, на панели отобразится номер подключенной платформы.

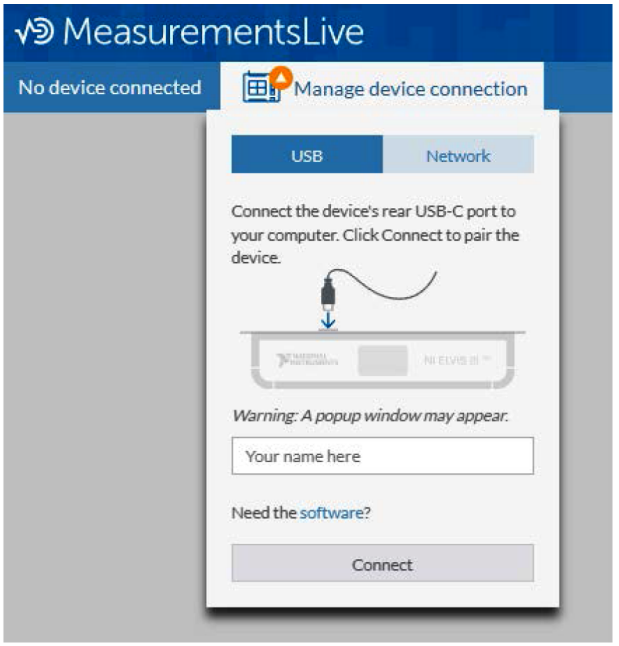


Рис. 101

Диалоговое окно подключения платформы NI ELVIS III

7. Выполните соединение канала осциллографа 1 (*CH 1*) с каналом генератора 1 (*CH 1*) с помощью кабеля *BNC*, как показано на рисунке 102. В результате генерируемый сигнал будет отображаться на осциллографе.

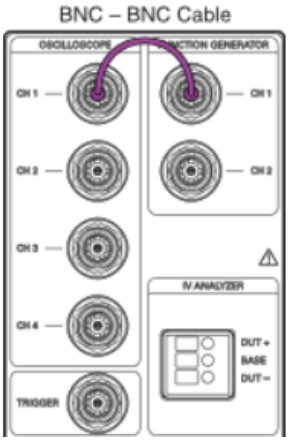


Рис. 102

Соединение функционального генератора с осциллографом

8. На вкладке *Instruments* (рис. 103) выберите функциональный генератор (*Function and Arbitrary Waveform Generator*).

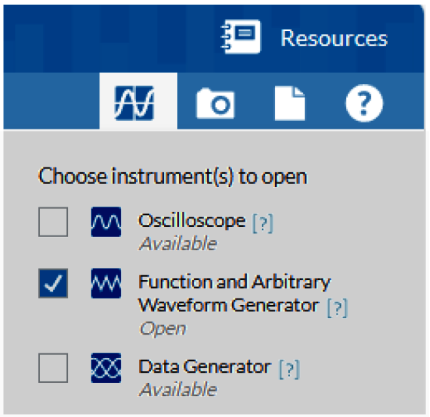


Рис. 103

Вкладка Instruments программы MeasurementsLive

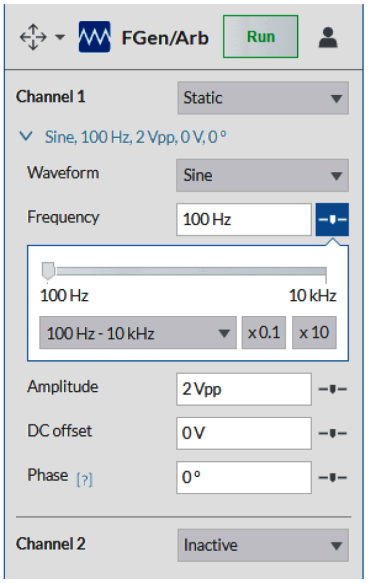


Рис. 104

Настройка канала 1 (CH 1) функционального генератора

9. Настройте канал 1 (*CH 1*) функционального генератора согласно рисунку 104.

10. По умолчанию значение частоты выставлено на величину 1 кГц. Нажмите на область справа от значения частоты и щелкните кнопку x0.1.

11. Запустите генератор нажатием на кнопку *Run*.

12. Выберите на вкладке *Instruments* программную лицевую панель Осциллограф (*Oscilloscope*). Запустите осциллограф нажатием на кнопку *Run*. На экране виртуального осциллографа вы можете наблюдать синусоидальный сигнал заданной частоты (рис. 105). Выполните настройку осциллографа, как показано на рисунке.

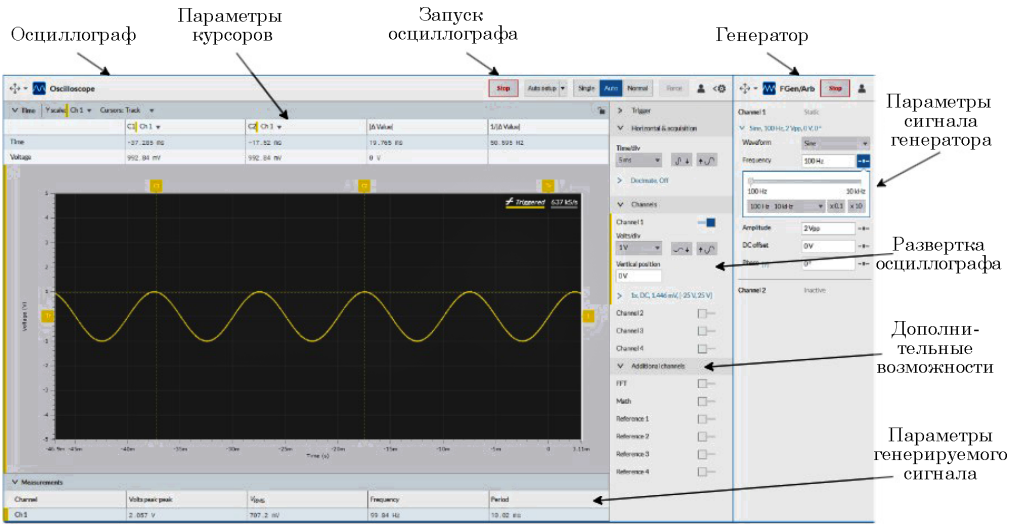


Рис. 105

Окно программы MeasurementsLive в режиме генерации сигнала

Задание:

1. Измените тип сигнала, его частоту и амплитуду. Зафиксируйте изменения. Выполните сравнение сигналов, сгенерированных в предыдущем ВП, с сигналами, генерируемыми с помощью *NI ELVIS III*, отметьте изменения, если они имеются.

2. В окне осциллографа подключите определение спектра (*FFT*), настройте его параметры, наблюдайте спектр сигнала.

ПРИЛОЖЕНИЯ

Приложение 1
Типы данных

Элемент управления	Тип данных
	Числовой с плавающей запятой расширенной точности
	Числовой с плавающей запятой двойной точности
	Числовой с плавающей запятой одинарной точности
	Число с фиксированной точкой
	64-разрядное целое число со знаком
	32-разрядное целое число со знаком
	16-разрядное целое число со знаком
	8-разрядное целое число со знаком
	Беззнаковое 64-битовое целое число
	Беззнаковое 32-битовое целое число
	Беззнаковое 16-битовое целое число
	Беззнаковое 8-битовое целое число
	Комплексный с плавающей запятой одинарной точности
	Комплексный с плавающей запятой двойной точности
	Комплексный с плавающей запятой расширенной точности
	Массив
	Перечисление
	Логический
	Строковый
	Путь
	Кластер

Приложение 2
Математические функции

Название	Изображение	Описание
Add		Сумма двух операндов
Subtract		Разность двух операндов
Multiply		Произведение двух операндов
Divine		Частное двух операндов
Increment		Значение входа увеличивается на 1
Decrement		Значение входа уменьшается на 1
Quotient & Remainder		Деление x/y: целое и остаток
Add Array Elements		Сумма элементов массива
Multiply Array Elements		Произведение элементов массива
Absolute Value		Модуль
Square Root		Квадратный корень числа
Square		Возведение в квадрат
Negative		Отрицание значения на входе
Reciprocal		Деление 1 на входное значение

Приложение 3
Операции структуры Formula Node

Обозначение операции	Описание
**	Возведение в степень
+	Суммирование
-	Вычитание
*	Умножение
/	Деление
>>	Арифметический сдвиг вправо
<<	Арифметический сдвиг влево
&	Битовое И
|	Битовое ИЛИ
&&	Логическое И
||	Логическое ИЛИ
ceil(x)	Округление до целого числа
floor(x)	Округление до меньшего целого *x*
pow(x,y)	Возведение *x* в степень *y*

Приложение 4
Классификация вейвлетов

Строение функций вейвлетов различно. Некоторые вейвлеты состоят лишь из одной функции ψ — материнского вейвлета. Другие строятся при помощи двух функций: материнского вейвлета — ψ и функции φ — масштабирующей функции.

1. Ортогональные вейвлеты с компактным носителем.

Это *вейвлеты Добеши* (db N), *симлеты* (sym N), *койфлеты* (coif N).

Общие свойства:

- функция φ существует;
- функции ψ и φ имеют компактный носитель;
- функция ψ имеет некоторое число нулевых моментов.

Применяются для дискретного преобразования.

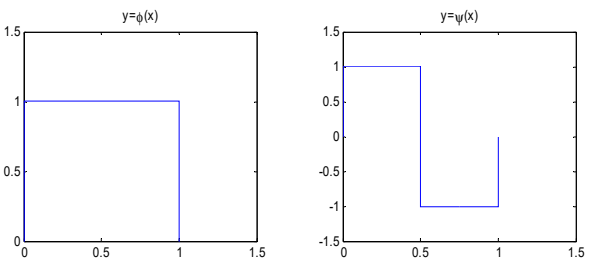
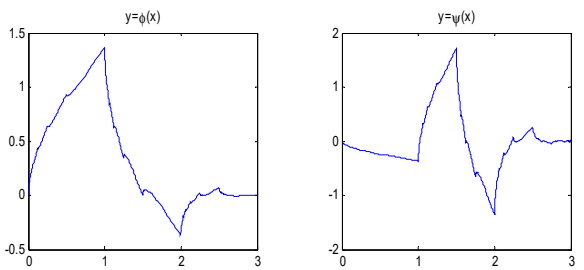
2. Биортогональные пары вейвлетов с компактным носителем.

Это *биортогональные вейвлеты* (biorNr.Nd).

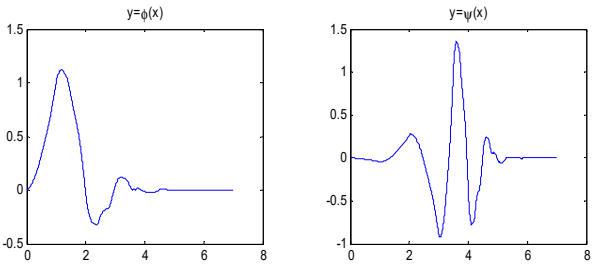
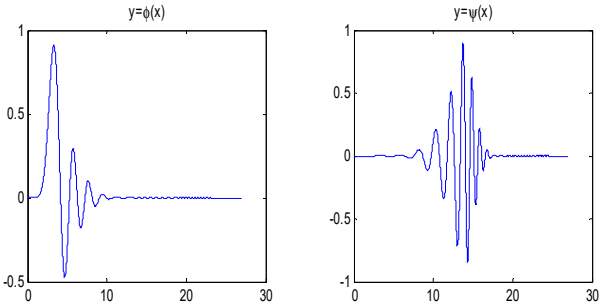
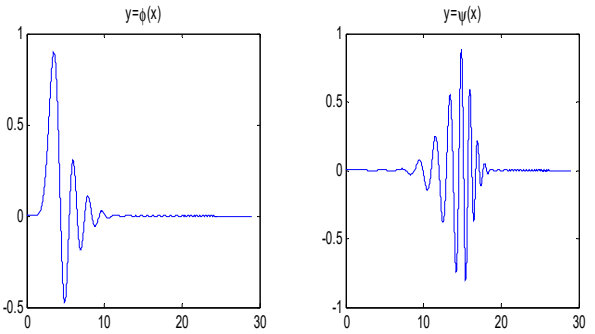
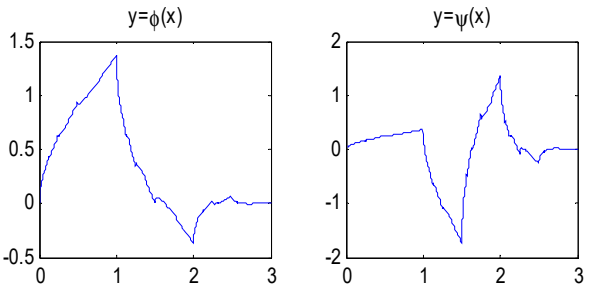
Свойства:

- функция φ существует;
- функции ψ, φ подходят для восстановления сигнала;
- функции ψ, φ не имеют аналитического выражения.

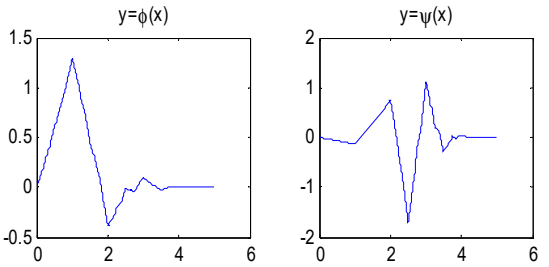
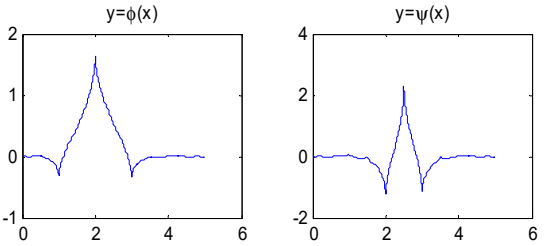
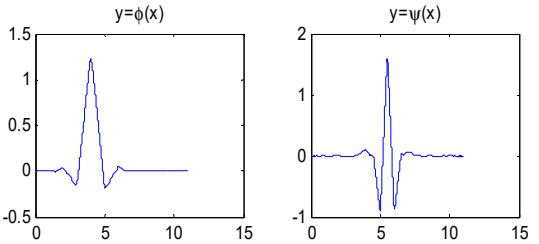
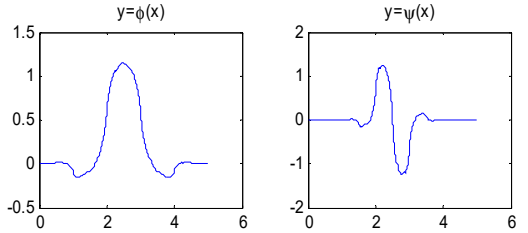
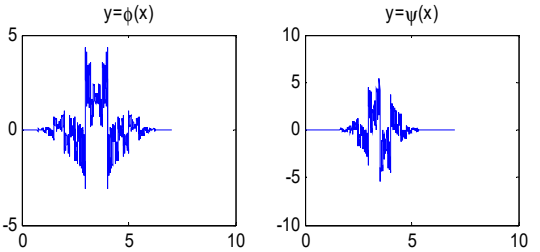
Используются для дискретного преобразования. Обладают хорошей симметрией. Изображение материнского вейвлета и масштабирующей функции приведены в таблице ниже.

Название	Вейвлет
db 1 (вейвлет Хаара — haar)	
db 2 Добеши 2	

Продолжение табл.

Название	Вейвлет
db 4 Добеши 4	
db 13 Добеши 13	
db 14 Добеши 14	
sym 2 Симлет 2	

Продолжение табл.

Название	Вейвлет
sym 3 Симлет 3	
coif 1 Койфлет 1	
coif 2 Койфлет 1	
bior 1.3 Биор 1.3	
bior 3.3 Биор 3.3	

ЛИТЕРАТУРА

1. *Федосов*, *В. П.* Цифровая обработка сигналов в LabVIEW : учеб. пособие / В. П. Федосов, А. К. Нестеренко ; под ред. В. П. Федосова. — М. : ДМК Пресс, 2007. — 456 с.

2. NI Educational Laboratory Virtual Instrumentation Suite (NI ELVIS). — National Instruments, 2018.

3. Комплект виртуальных измерительных приборов для учебных лабораторий NI ELVIS. Технические средства. Руководство пользователя. — Новосиб. гос. техн. ун-т, Российский филиал корпорации National Instruments, 2006.

4. NI ELVIS III 1.2. Комплект виртуальных измерительных приборов для учебной лаборатории. Руководство по эксплуатации / NATIONAL INSTRUMENTS. — URL: https://nitec.nstu.ru/upload/lib/2020_Translated/ELVIS%20III_%D0%A0%D1%83%D0%BA%D0%BE%D0%B2%D0%BE%D0%B4%D1%81%D1%82%D0%B2%D0%BE%20%D0%BF%D0%BE%20%D1%8D%D0%BA%D1%81%D0%BF%D0%BB%D1%83%D0%B0%D1%82%D0%B0%D1%86%D0%B8%D0%B8%20(%D0%9D%D0%B0%D1%87%D0%B0%D0%BB%D0%BE%20%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D1%8B).pdf (дата обращения: 20.02.2024).

5. *Щепетов*, *А. Г.* Преобразование измерительных сигналов : учебник и практикум для вузов / А. Г. Щепетов, Ю. Н. Дьяченко ; под ред. А. Г. Щепетова. — М. : Юрайт, 2024. — 270 с.

6. *Лысенко*, *О. В.* Учебная лаборатория электроники на аппаратно-программном комплексе ELVIS-LabVIEW-Multisim : учеб. пособие / О. В. Лысенко, П. П. Гавриш, Ю. А. Мелешкин. — Самара : Самар. гос. техн. ун-т, 2009. — 76 с.

7. Электроника: часть первая. Лабораторный практикум по аналоговой электронике в программно-аппаратной среде NI ELVIS II : учеб. пособие / Э. И. Цимбалист, П. Ф. Баранов, С. В. Силушкин, Ю. М. Фомичев ; Томск. Политех. Ун-т. — Томск : Изд-во Томск. политехн. ун-та, 2013. — 302 с.

8. *Суранов*, *А. Я.* LabVIEW 8.20: Справочник по функциям / А. Я. Суранов. — М. : ДМК Пресс, 2007. — 536 с.

9. *Лупов*, *С. Ю.* LabVIEW в примерах и задачах. Учебно-методические материалы по программе повышения квалификации «Обучение технологиям National Instruments» / С. Ю. Лупов, С. И. Муякшин, В. В. Шарков. — Н. Новгород, 2007. — 101 с.

10. Автоматизированный сбор и цифровая обработка данных в измерительных системах / Ю. К. Евдокимов, Е. С. Денисов, Д. В. Шехтурин, О. В. Шиндор. — Казань : Изд-во Казан. гос. техн. ун-та, 2012.

11. *Тревис*, *Дж.* LabVIEW для всех / Дж. Тревис ; пер. с англ. Н. А. Клушин. — М. : ДМК Пресс; ПриборКомплект, 2005. — 544 с.

12. *Васильев*, *А. М.* Основы программирования в среде LabVIEW / А. М. Васильев, О. Ю. Лашманов. — СПб. : Ун-т ИТМО, 2015. — 82 с.

Ольга Владимировна ШИНДОР,
Дмитрий Евгеньевич ЧИКРИН,
Петр Анатольевич КОКУНИН

LabVIEW
В ЦИФРОВОЙ ОБРАБОТКЕ СИГНАЛОВ

УЧЕБНОЕ ПОСОБИЕ

Зав. редакцией литературы
по информационным технологиям
и системам связи *О. Е. Гайнутдинова*
Ответственный редактор *Т. С. Спирина*
Подготовка макета *Т. С. Симонова*
Корректор *Е. П. Филатова*
Выпускающий *В. А. Иутин*

ЛР № 065466 от 21.10.97
Гигиенический сертификат 78.01.10.953.П.1028
от 14.04.2016 г., выдан ЦГСЭН в СПб

Издательство «ЛАНЬ»
lan@lanbook.ru; www.lanbook.com
196105, Санкт-Петербург, пр-кт Ю. Гагарина, д. 1, лит. А.
Тел./факс: (812) 336-25-09, 412-92-72.
Бесплатный звонок по России: 8-800-700-40-71

Подписано в печать 28.11.24.
Бумага офсетная. Гарнитура Школьная. Формат 70×100 $^{1}/_{16}$.
Печать офсетная/цифровая. Усл. п. л. 8,13. Тираж 30 экз.

Заказ № 1309-24.

Отпечатано в полном соответствии
с качеством предоставленного оригинал-макета
в АО «Т8 Издательские Технологии».
109316, г. Москва, Волгоградский пр-кт, д. 42, к. 5.